



AKTU

B.Tech I-Year



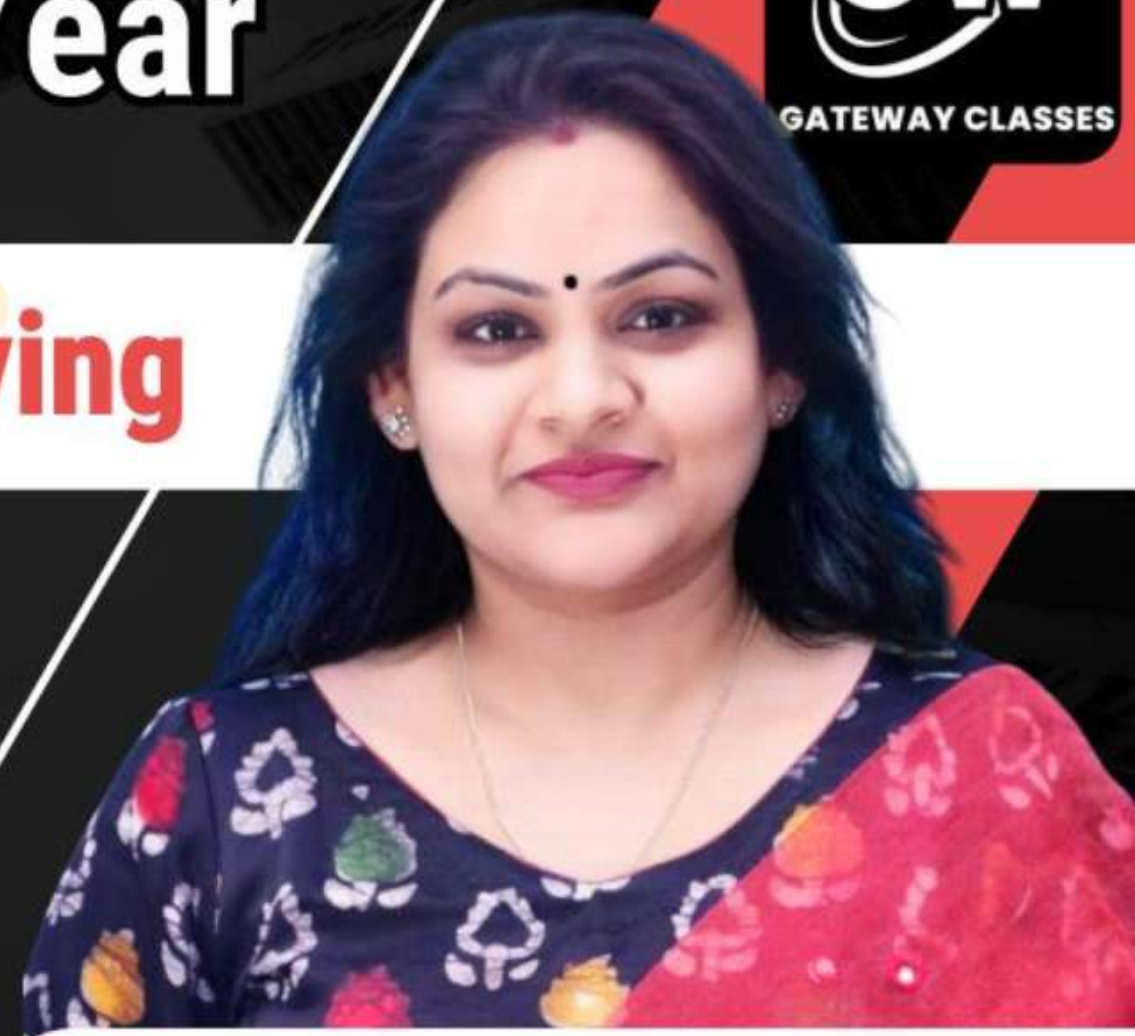
PPS: Programming For Problem Solving

Crash Course

(ONE SHOT Revision)

Unit-3

Pdf Notes | AKTU PYQs | Imp. Questions



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

PROGRAMMING FOR PROBLEM SOLVING

UNIT-3

AKTU SYLLABUS

Iteration and Loops: Use of While, do While and for Loops, Multiple Loop Variables, Use of Break , Goto and Continue Statements.

Arrays: Array Notation and Representation, Manipulating Array Elements, using

Multi Dimensional Arrays. Character Arrays and Strings, Structure, union, Enumerated Data types, Array of Structures, ✓ Passing Arrays to Functions.



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Lecture-1

Topics to be Covered

- while loop
- do-while loop
- Difference between while and do-while loop
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Iterative Statements (Loops)

A loop statement allows us to execute a statement or group of statements multiple times.

1. while loop
2. do while loop
3. for loop

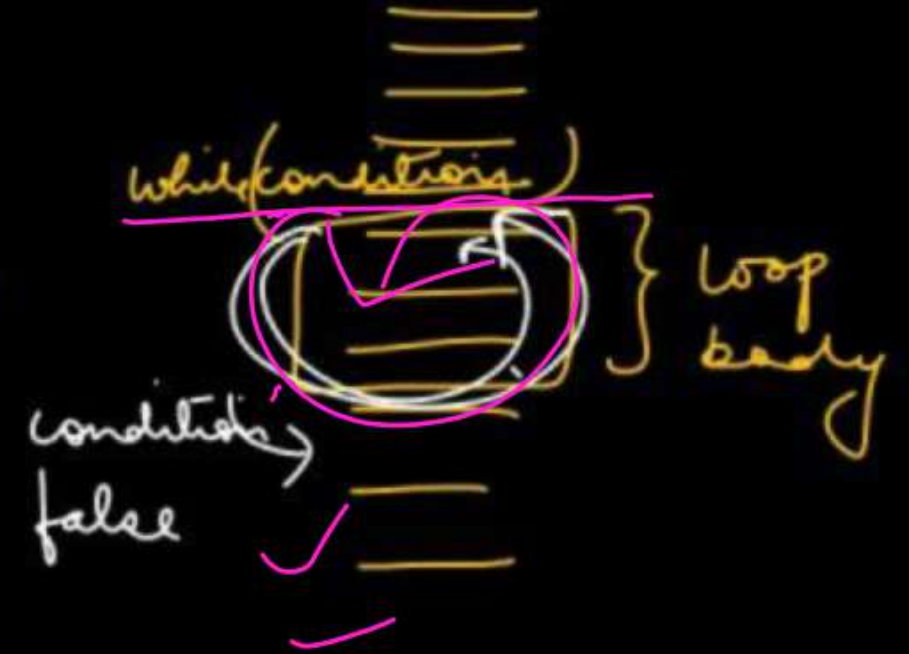
Sequential



Selection



Iterative (loop)



- Its initialization is done before the loop
- Update condition is present in the body of the loop.
- Body of the loop execute while the condition is true.
- When the condition becomes false, the program control passes to the line immediately following the loop.

Syntax Keyword

while (test expression)

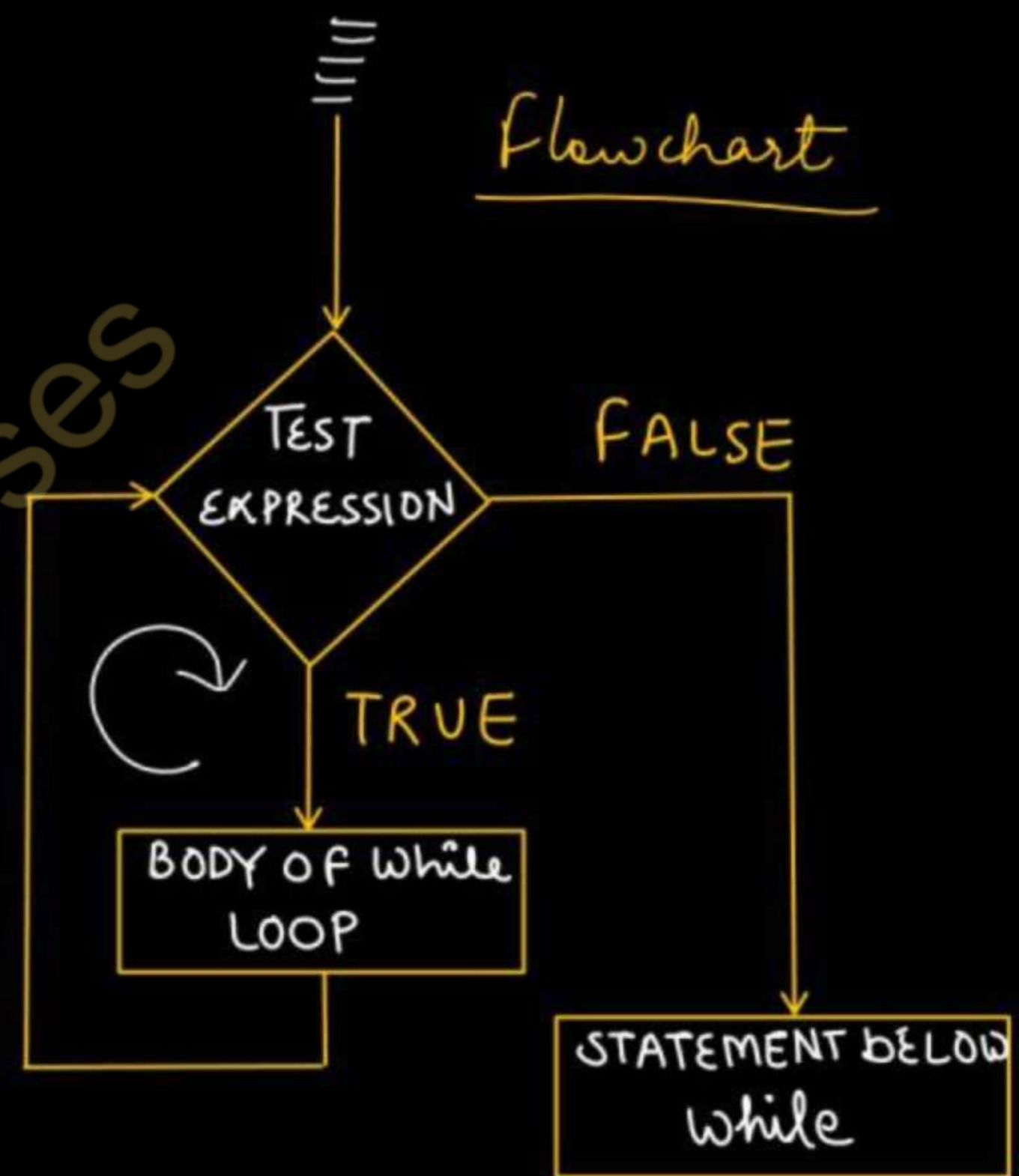
{
 ≡ } LOOP BODY

statement below while

```

  |||
  |||
  |||
  
```

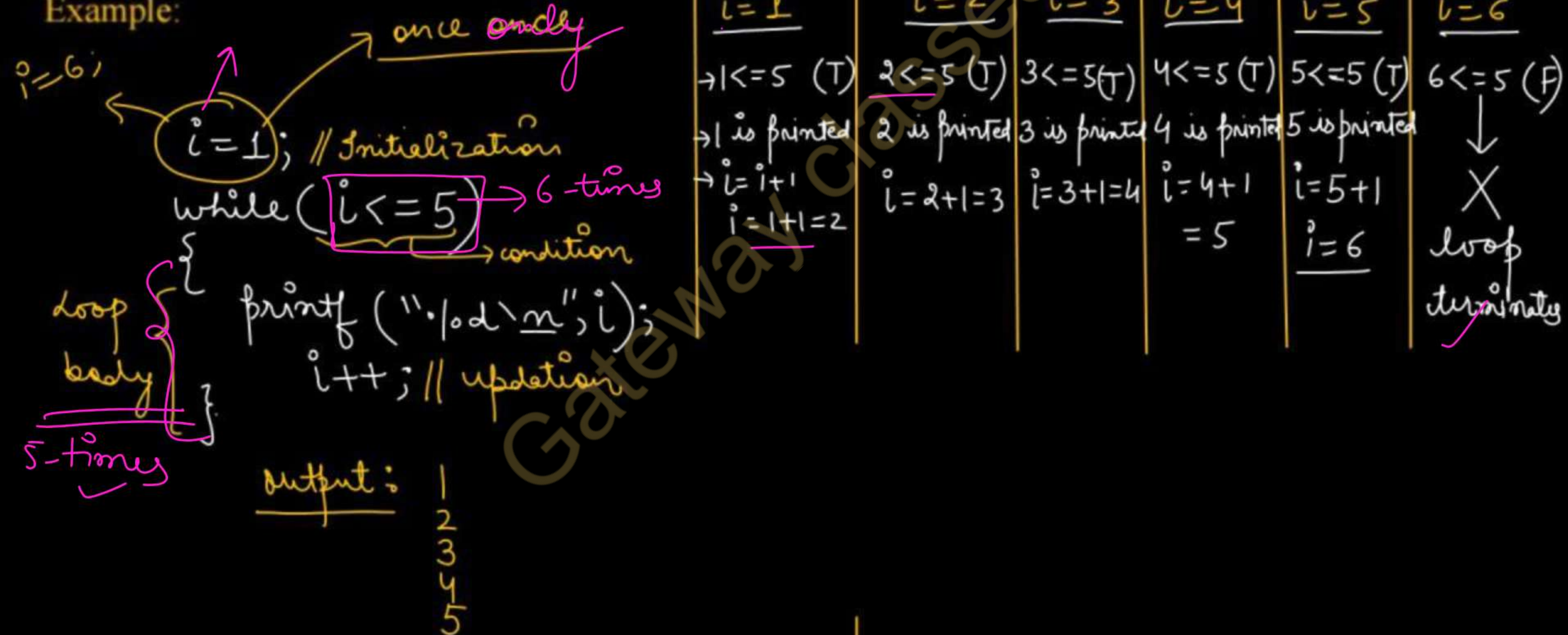
Flowchart



Why it is called ENTRY CONTROLLED LOOP?

Before entering the loop, the condition is tested and if the result is false, the loop body will be skipped and loop will not run at all. This is why it is also called entry controlled loop.

Example:



Write a program to print all natural numbers from 1 to n.

```
#include <stdio.h>
int main()
{
    int i, n;
    printf("Enter the value of n");
    scanf("%d", &n);
```

i = 1; // Initialization

```
while (i <= n) → condition
{
    printf("%d\n", i);
    i++; // updation
}
```

return 0;

}

while (i < (n + 1))

⇒

Write a program to print all natural numbers from n to 1 in reverse order.

```
#include <stdio.h>
int main()
{
    int i, n;
    printf("Enter the value of n");
    scanf("%d", &n); // 3
    i = n;
    while(i >= 1)
    {
        printf("%d\n", i);
        i--;
    }
    return 0;
}
```

n=3

i=3

$(3 \geq 1) (T)$

→
 $i = i - 1 = 2$

i=2

$(2 \geq 1) (T)$

→
 $i = 2 - 1 = 1$

i=1

$(1 \geq 1) (T)$

→
 $i = 1 - 1 = 0$

i=0

$(0 \geq 1) (False)$

↓
X

output :

3
2
1

Infinite while loop

①

i=5;

```
while (i <= 50)  
{  
    printf("%d", i);  
}
```

loop condition
will never become
false $\Rightarrow$ It will run
infinitely

5 5 5 5
infinite times

②

i=5; ✓

```
while (i <= 5)  
{  
    printf("%d", i);  
    i++;  
}
```

will be
treated as
empty loop body

i=5; ✓
while (i <= 5)
{
 ;
}

```
{  
    printf("%d", i);  
    i++;  
}
```

$\Rightarrow$ Infinite
loop

Empty while loop

```
i = 1;  
while (i <= 5)  
{  
}  
}
```

no loop body

while loop without condition

```
i = 1  
while (  
{  
=  
=  
=  
}  
}
```

It is necessary to write condition

Rules for Parenthesis

while (1) True
{
 ===
}
}

⇒ All nonzero values are True.

⇒ zero is false

do-while loop

- It is similar to the while loop.
- The only difference is that the test condition here is checked at the end of the loop.
- It is called exit controlled loop.
- It is guaranteed to execute at least one time.
- If the condition is true, the flow of control jumps back up to do, and the statement(s) in the loop executes again. This process repeats until the given condition becomes false.

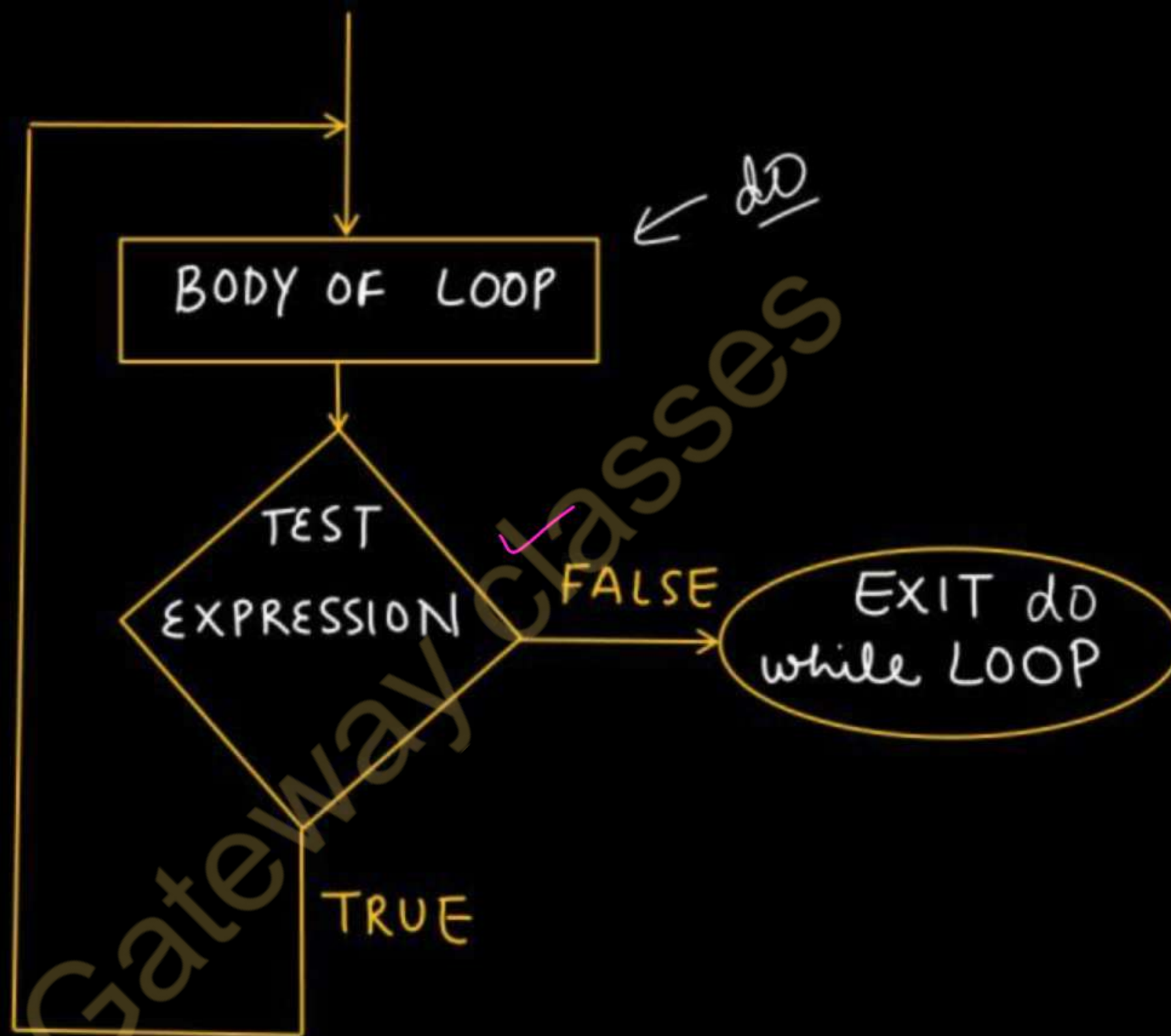
Syntax:

initialization

```
do  
{  
    statement block;  
} while (condition);  
  
statement y;
```

loop body

FLOWCHART OF do-while loop



Example: Program to print natural numbers ^{from 1 to 5} using do-while loop.

```
#include <stdio.h>
int main()
{
    int i;
    i = 1;
    do
    {
        printf("%d\n", i);
        i++; // updation
    } while (i <= 5) // condition is checked
                    // at loop exit
    return 0;
}
```

V. Imp Difference between while Loop and do-while loop (AKTU)

Feature	while Loop	do-while Loop
Syntax	while (condition) { } ✓	do { } while (condition); ✓
Semicolon Place	No semicolon after the while condition.	There must be a semicolon after the while condition.
First Execution	Condition is checked before the loop block is executed.	Loop block is executed at least once before checking the condition.
Iterations	Loop body does not execute if, the condition at the first iteration appears false.	Loop body executes <u>at least once even</u> if the condition is false at the first iteration.
Use Cases	Suitable when the loop block should be executed only if the condition is initially true.	Useful when the loop block must be executed at least once, regardless of the initial condition.

AKTU PYQ

1. Differentiate between while and do-while loop. (AKTU 2022-23)
2. Explain while loop.
3. Write short note on do-while loop.

Gateway classes



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture-2

Topics to be Covered

- for loop
- program to print all even numbers from 1 to n
- program to add all natural numbers from 1 to n
- program to find factorial of a given number
- program to find sum of all even numbers between 1 to n
- program to print multiplication table of a given number
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

for loop

- Repeat a set of statements a defined number of times.
- A form of an **entry-controlled loop**.
- Unlike the while loop and do-while loop, for loop contains the initialization, condition, and updating statements as part of its syntax.

✓
Syntax:

for (initialization; test expression; updation)

{

// body consisting of multiple statements

}

Ex.

printing Numbers from 1 to 5.

for ($i = 1$; $i \leq 5$; $i++$)
 { printf("%d\n", i); }

}
loop body

executes only 1 time

6. times

executes 5 times

```
for (i = 1; i <= 5; i++)  
{  
    printf("%d\n", i); // executes 5 times  
}
```

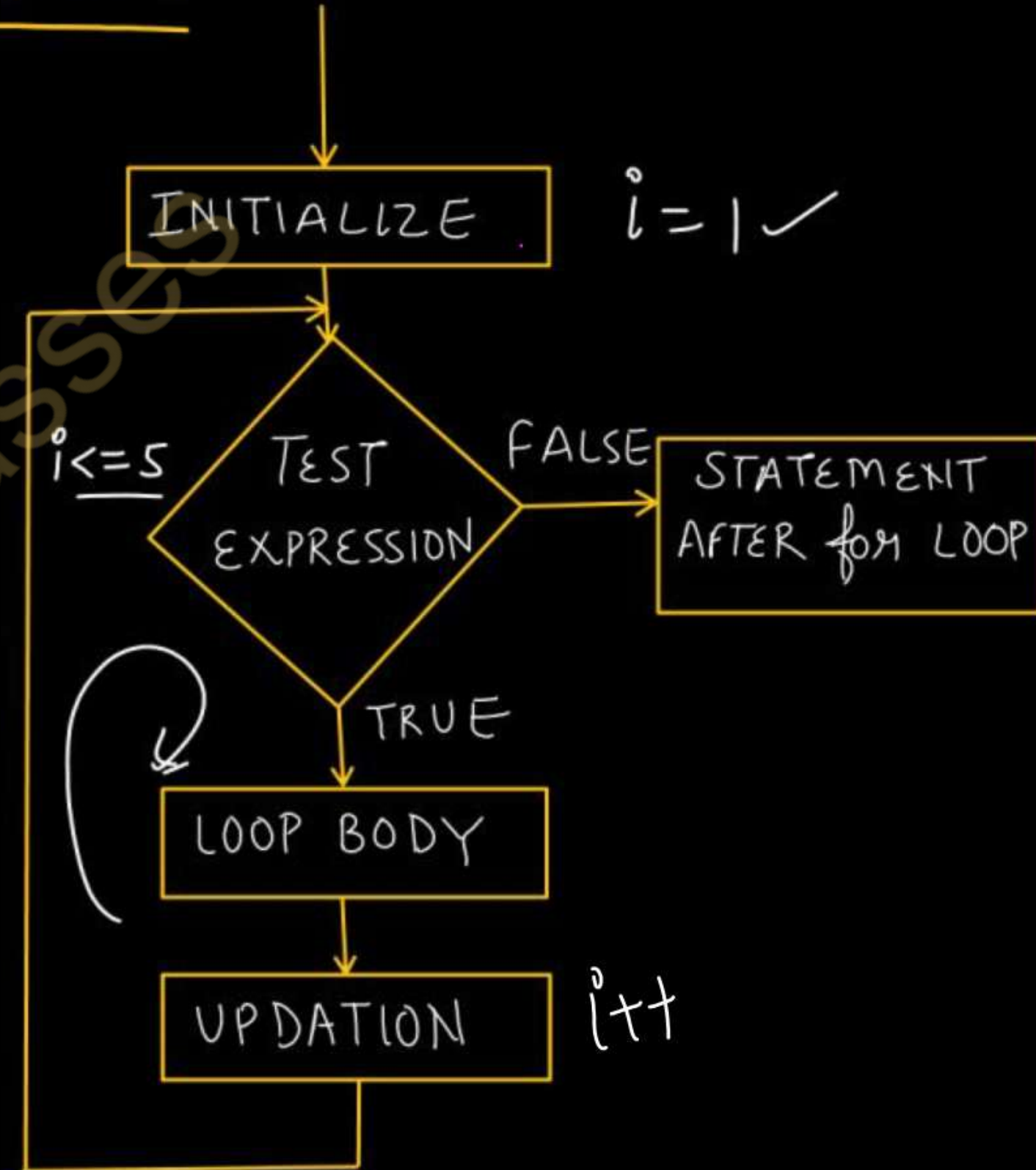
output:—

1
2
3
4
5

FLOWCHART OF for LOOP

- The initialization statement is executed only once.
- If the test expression is false, the loop terminates.
- if the test expression is true, statements inside the body of the for loop are executed, and the update expression is executed.
- Again the test expression is evaluated.

This process goes on until the test expression is false.



Write a program to print all even numbers from 1 to n. Let's take $n=5$

```
#include <stdio.h>
```

```
int main()
```

```
{ int n, i;
```

```
printf("Enter the value of n");
```

```
scanf("%d", &n);
```

```
for (i=1; i<=n; i++)
```

```
{ if (i%2 == 0)
```

```
    if (i%2 != 0)
```

→ for odd nos only

```
        printf("%d\n", i);
```

```
    }
```

```
return 0;
```

output: 2
4

$i=1$	$i=2$	$i=3$	$i=4$	$i=5$	$i=6$
$1 \leq 5 (T)$	$2 \leq 5 (T)$	$3 \leq 5 (T)$	$4 \leq 5 (T)$	$5 \leq 5 (T)$	$6 \leq 5 (F)$
if $\times$	if $\checkmark$	if $\times$	if $\checkmark$	if $\times$	$\downarrow$
$i=2$	2 is printed	$i=4$	4 is printed	$i=6$	$\times$
	$i=3$		$i=5$		

Write a program to print all even numbers from 1 to n.

```
#include <stdio.h>
int main()
{
    int n, i;
    printf("Enter the value of n");
    scanf("%d", &n);
```

```
    for (i = 1; i <= n; i++)
    {
        if (i % 2 == 0)
        {
            printf("%d\n", i);
        }
    }
```

```
    return 0;
}
```

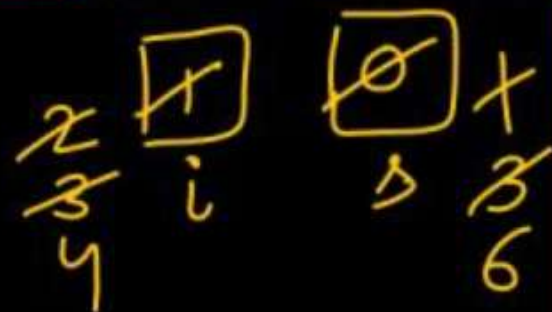


$i=1 \Rightarrow$ for odd nos

```
for (i = 2; i <= n; i = i + 2)
{
    printf("%d", i);
}
```

Write a program to add all natural numbers from 1 to n.

```
#include <stdio.h>
int main()
{
    int n, i, s=0;
    printf("Enter how many natural numbers you want to add");
    scanf("%d", &n);
    for(i=1; i<=n; i++)
    {
        s=s+i;
    }
    printf("The sum is=%d", s);
    return 0;
}
```

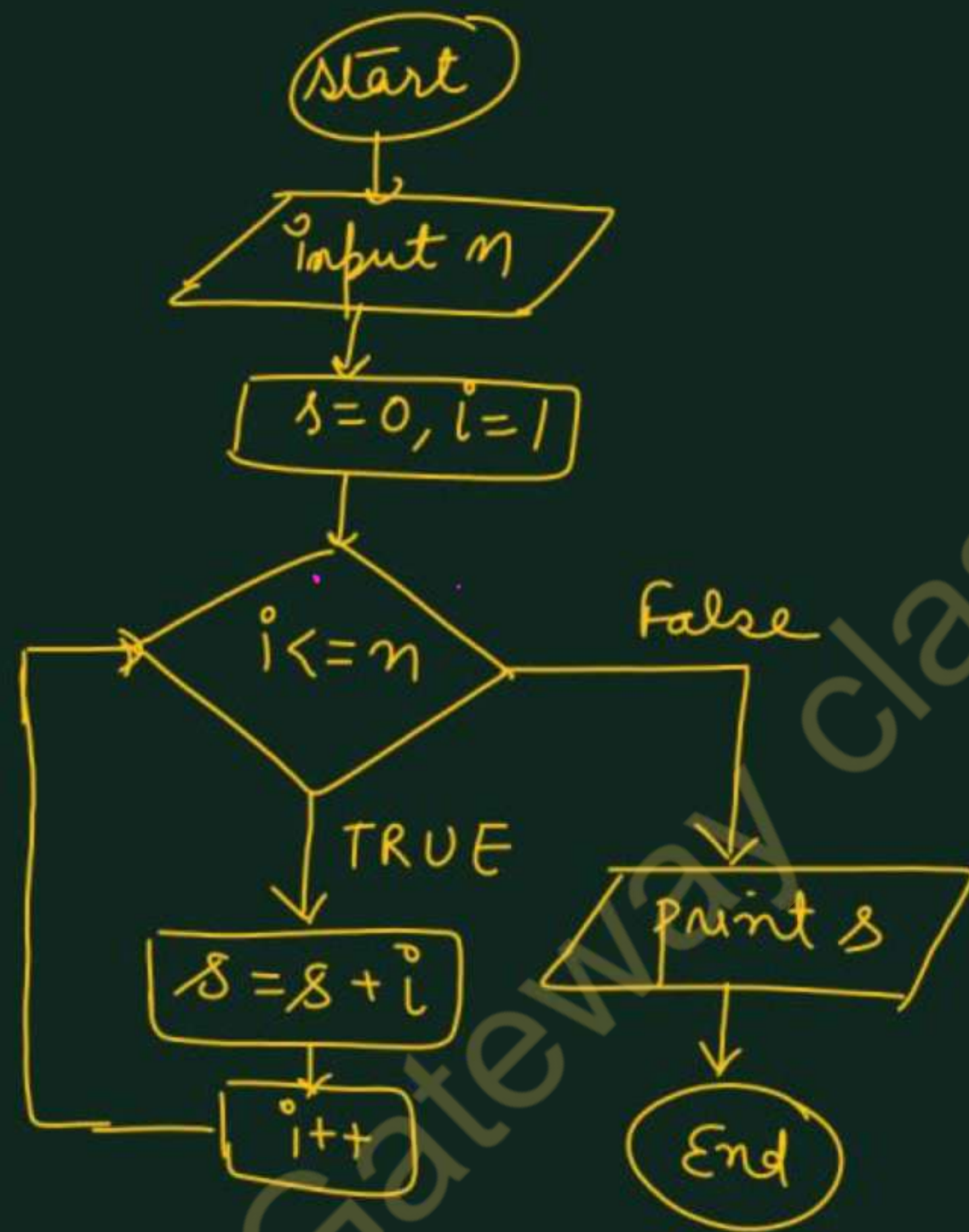


n=3

Lets take n=3

<u>i=1</u>	<u>i=2</u>	<u>i=3</u>	<u>i=4</u>
1<=3 (T)	2<=3 (T)	3<=3 (T)	4<=3 (F)
s=s+i	s=s+i	s=3+3	↓
s=0+1	s=1+2	<u>s=6</u>	X
<u>s=1</u>	<u>s=3</u>	i=4	
i++ ⇒ i=2	i=3		

output: The sum is=6



⇒ flowchart of sum of natural numbers from 1 to n .

Program to calculate factorial of a number.

```
#include <stdio.h>
int main()
```

```
{ int n, i, fact = 1;
```

```
printf("Enter the number");
```

```
scanf("%d", &n);
```

```
if (n < 0)
```

```
printf("factorial is not defined");
```

```
else if (n == 0 || n == 1)
```

```
printf("factorial is 1");
```

```
else {
```

```
for (i = 1; i <= n; i++)
```

```
{
    fact = fact * i;
}
```

```
printf("The factorial is = %d", fact);
return 0;
}
```

4!

Algo

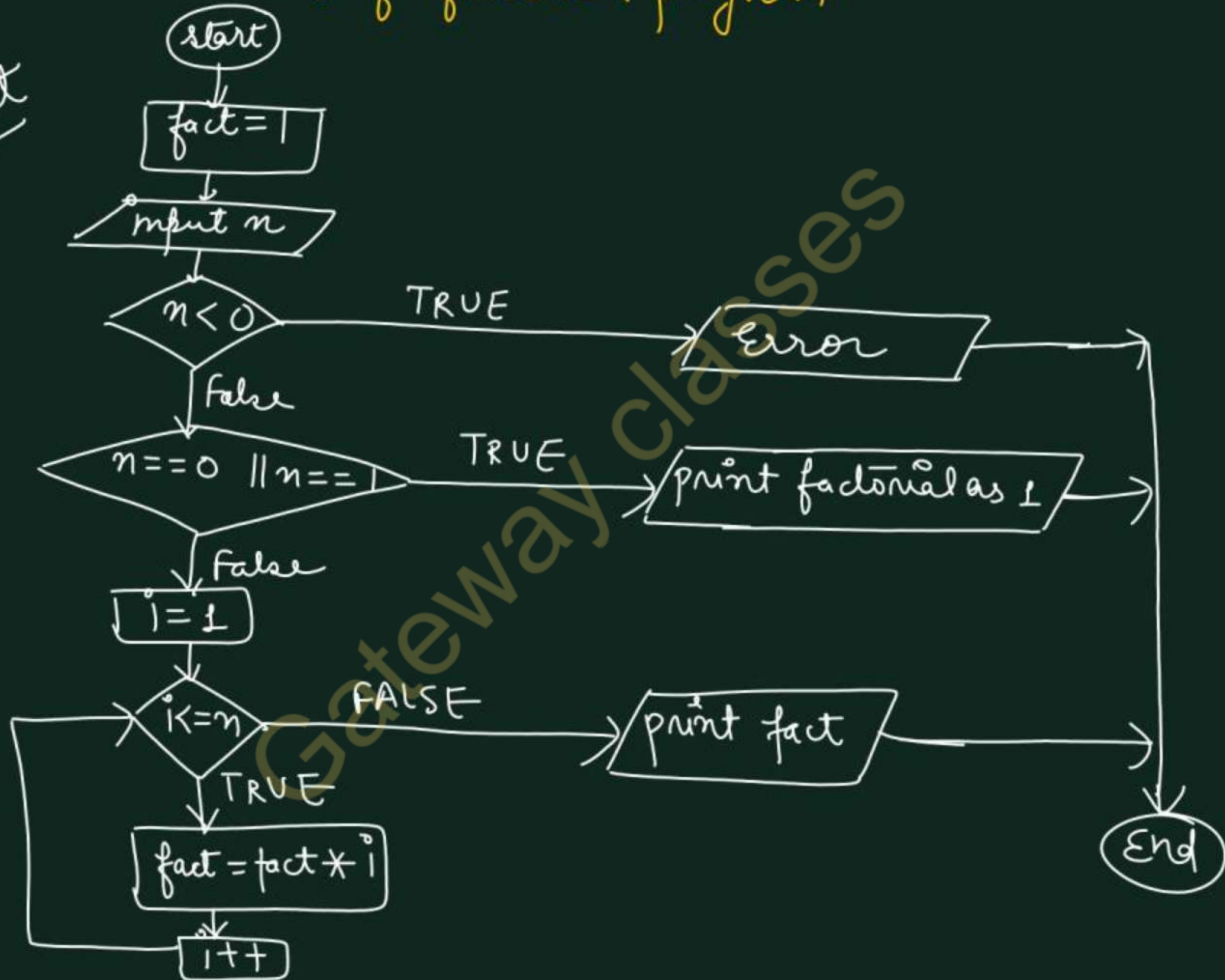
- 1) Start
- 2) Initialize $fact = 1, i = 1$
- 3) Input n
- 4) if $(n < 0)$
then print error and goto step ⑦
- 5) if $n = 0$ or $n = 1$
then print $fact = 1$ and goto step ⑦.

{
 else repeat steps (5.1, 5.2) till $i \leq n$
 5.1 Calculate $fact = fact \times i$
 5.2 calculate $i = i + 1$
}
- 6) Print fact
- 7) End

Algorithm to
find factorial
of a number

Flowchart of factorial program

Important



Write a program to find sum of all even numbers between 1 to n.

```
#include <stdio.h>
```

```
int main()
```

```
{ int n, i, s=0;
```

```
printf("Enter n");
```

```
scanf("%d", &n);
```

```
for(i=1; i<=n; i++)
```

```
{ if(i%2==0)
```

```
{ s=s+i;
```

```
}
```

```
printf("The sum of evens = %d", s);
```

```
return 0;
```

```
}
```

n=5 , s=0

i=1

if X

s=0

i=2

if ✓

s=0+2

s=2

i=3

if X

s=2

i=4

if ✓

s=2+4

i=5

if X

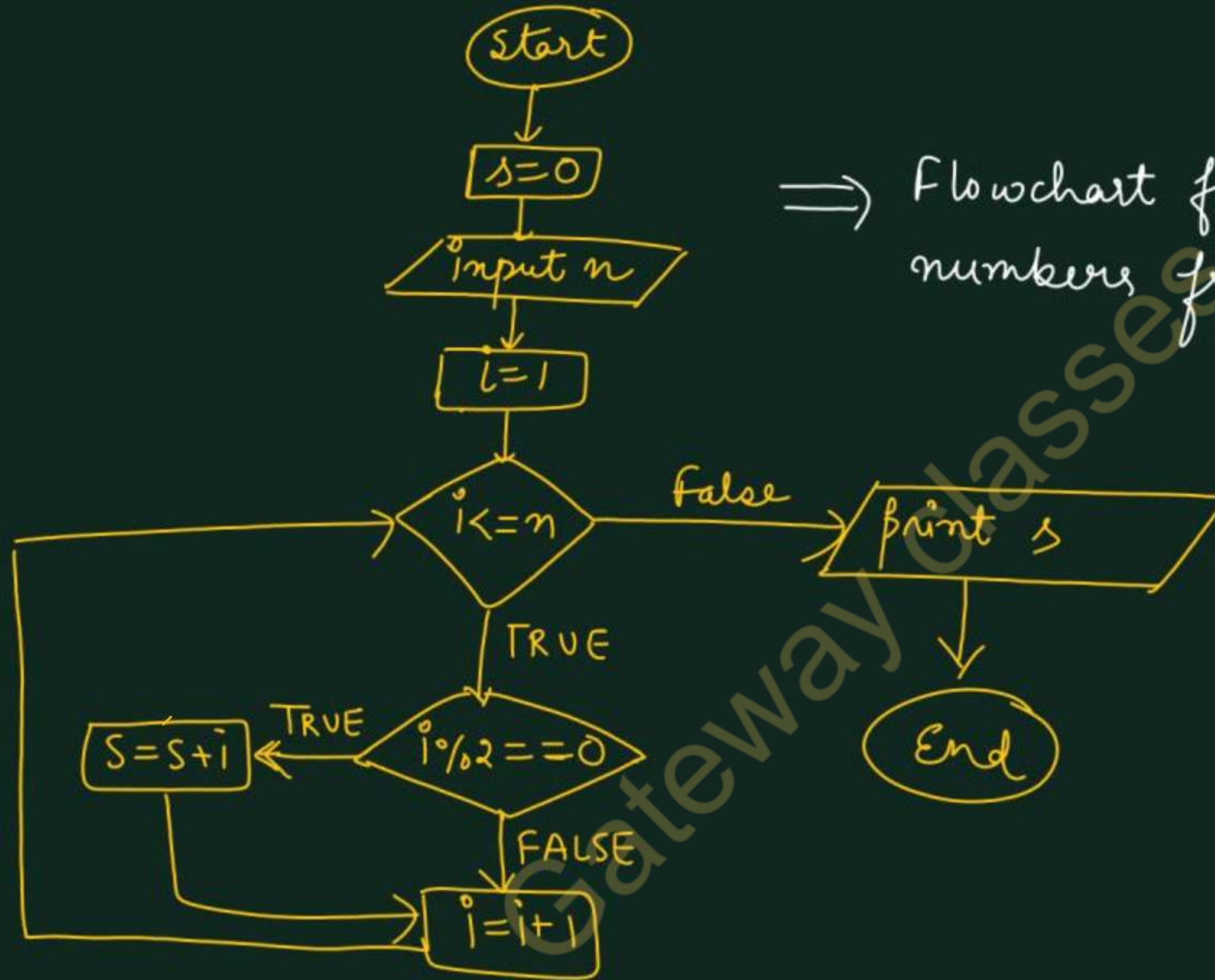
s=2+4

i=6

output:-

The sum of evens = 6

✓



$\Rightarrow$ Flowchart for sum of all even numbers from 1 to n .

C program to print multiplication table of a given number (AKTU)

```
#include <stdio.h>
int main()
{
    int n, i;
    printf("Enter the number to print multiplication\n table ");
    scanf("%d", &n);
```

```
for(i=1; i<=10; i++)
{
    printf("%d * %d = %d\n", n, i, (n*i));
}

return 0;
}
```

Expected Output

$$n=5$$

$5 \times 1 = 5$

$5 \times 2 = 10$

$5 \times 3 = 15$

$5 \times 4 = 20$

$5 \times 5 = 25$

$5 \times 6 = 30$

$5 \times 7 = 35$

$5 \times 8 = 40$

$5 \times 9 = 45$

$5 \times 10 = 50$

Important points

these two ; are mandatory

```
for( i=1, i<=n, i++)
```

```
{  
    ==  
}
```

same as
↓

```
i=1;
```

```
for( ; i<=n; )  
{  
    ==  
    i++;  
}
```

```
for( i=1, i<=5; i++ );  
{  
    printf( "%d", i );  
}
```

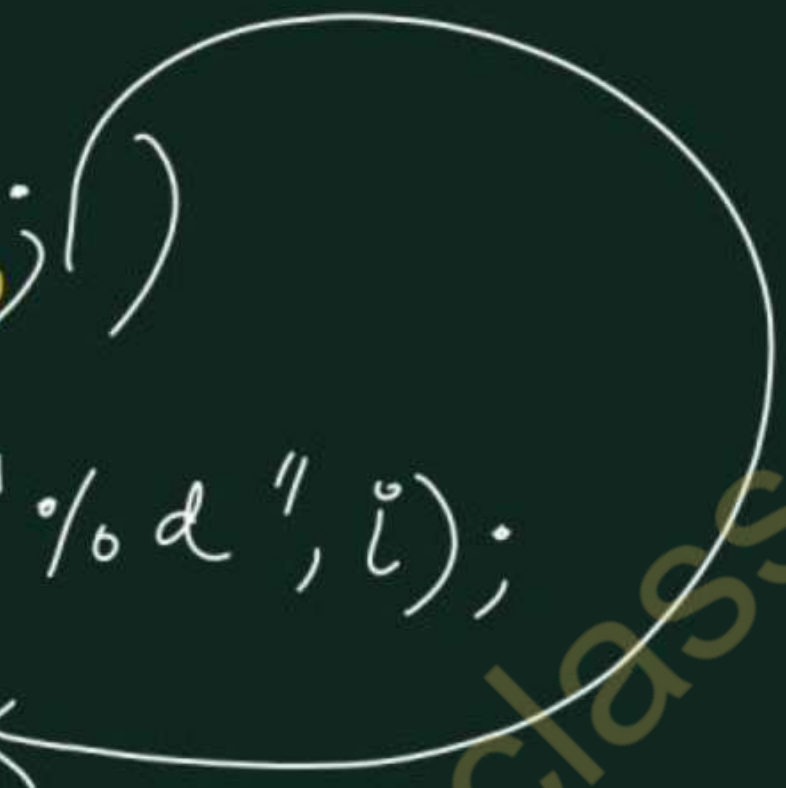
↓

```
for( i=1; i<=5; i++ )  
empty loop body ;
```

```
{ printf( "%d", i ); // 6 is printed.  
}
```



```
i = 1;  
for(1; 1; )  
{  
    printf("%d", i);  
    i++;  
}
```



output :- 1 2 3 4 5 - - - - - infinite times



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture-3

Topics to be Covered

- Program to count number of digits in an integer
- Program to find sum of first and last digit of any number
- Program to find sum of digits of a number with algo and flowchart
- Program to find reverse of a number with algo and flowchart
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Algorithm to find factorial of a given number

1: Start

2: Read number from User

3: Initialize Variable fact=1 and i=1

4: Repeat Until $i \leq \text{number}$

4.1 fact=fact*i

4.2 i=i+1

5: Print fact ✓

6: Stop ✓

$$n = 4! \quad \text{fact} = 1$$

$$i = 1$$

$$\text{fact} = 1 \times 1 \\ = 1$$

$$i = 2$$

$$i = 2$$

$$\text{fact} = 1 \times 2 \\ = 2$$

$$i = 3$$

$$i = 3$$

$$\text{fact} = 2 \times 3 \\ = 6$$

$$i = 4$$

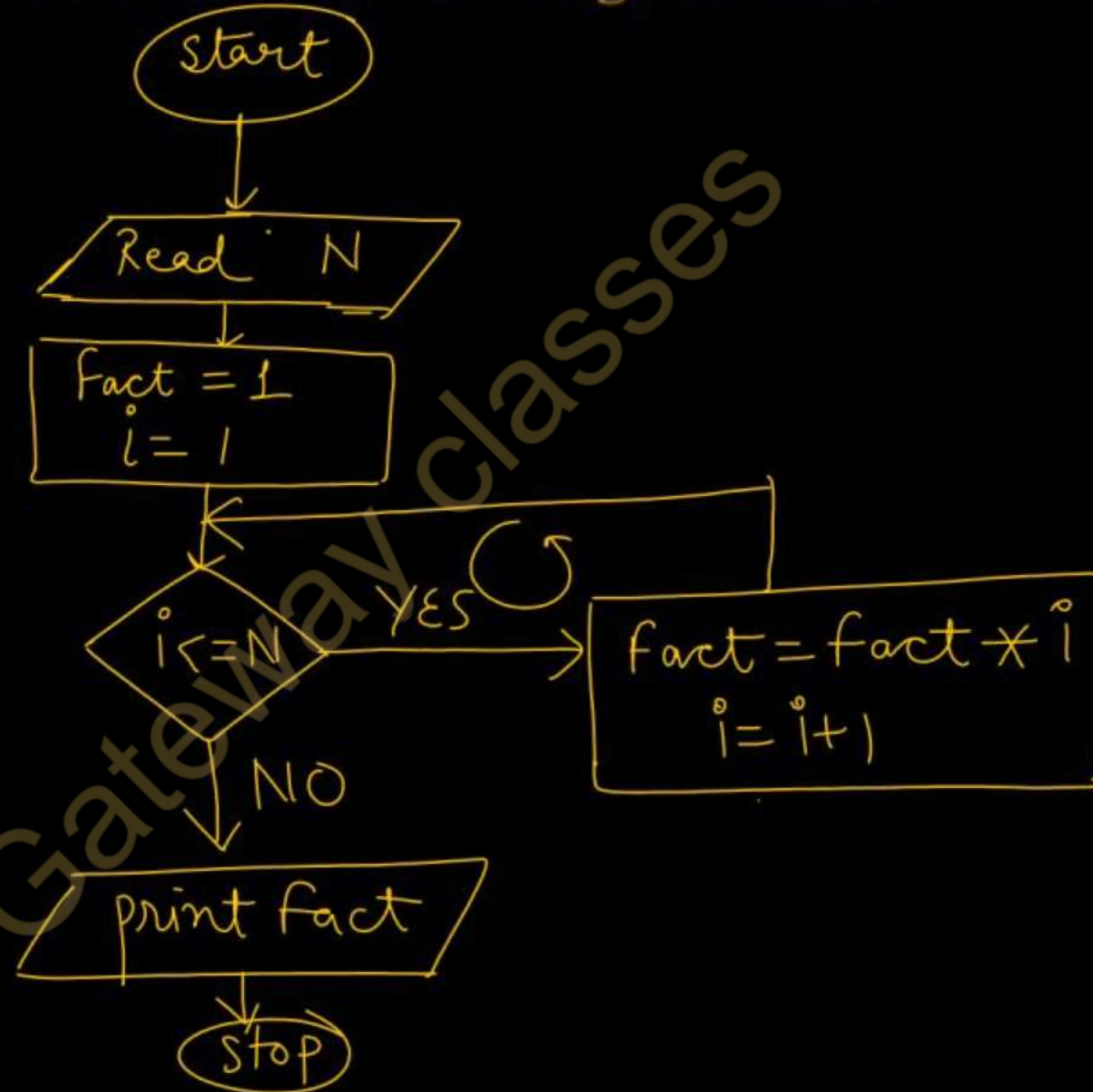
$$i = 4$$

$$\text{fact} = 6 \times 4$$

$$\text{fact} = 24$$

$$i = 5$$

Flowchart to find factorial of a given number



Algorithm to find sum of n numbers and their average

(AKTU)
 $n = 5$ ✓

(Add all nos. from 1 to 5)

Sum = 0, $i = 1$

II

IV

(IV)

(V)

I

$i = 1$

$i = 2$

$i = 3$

$i = 4$

$i = 5$

Sum = Sum + i

Sum = Sum + i

Sum = Sum + i

Sum = Sum + i

Sum = Sum + i

Sum = 0 + 1

Sum = 1 + 2

Sum =

Sum =

Sum =

Sum = 1

$i = 3$

1 + 2 + 3

1 + 2 + 3 + 4

1 + 2 + 3 + 4 + 5

$i = i + 1$

$i = 4$

$i = 5$

$i = 6$

$i = 2$

1. Start

2. Initialize sum=0 and $i=0$

3. Read the number n

4. Repeat steps 4.1 and 4.2 until $i=n$ reached

4.1 Compute sum = sum + i

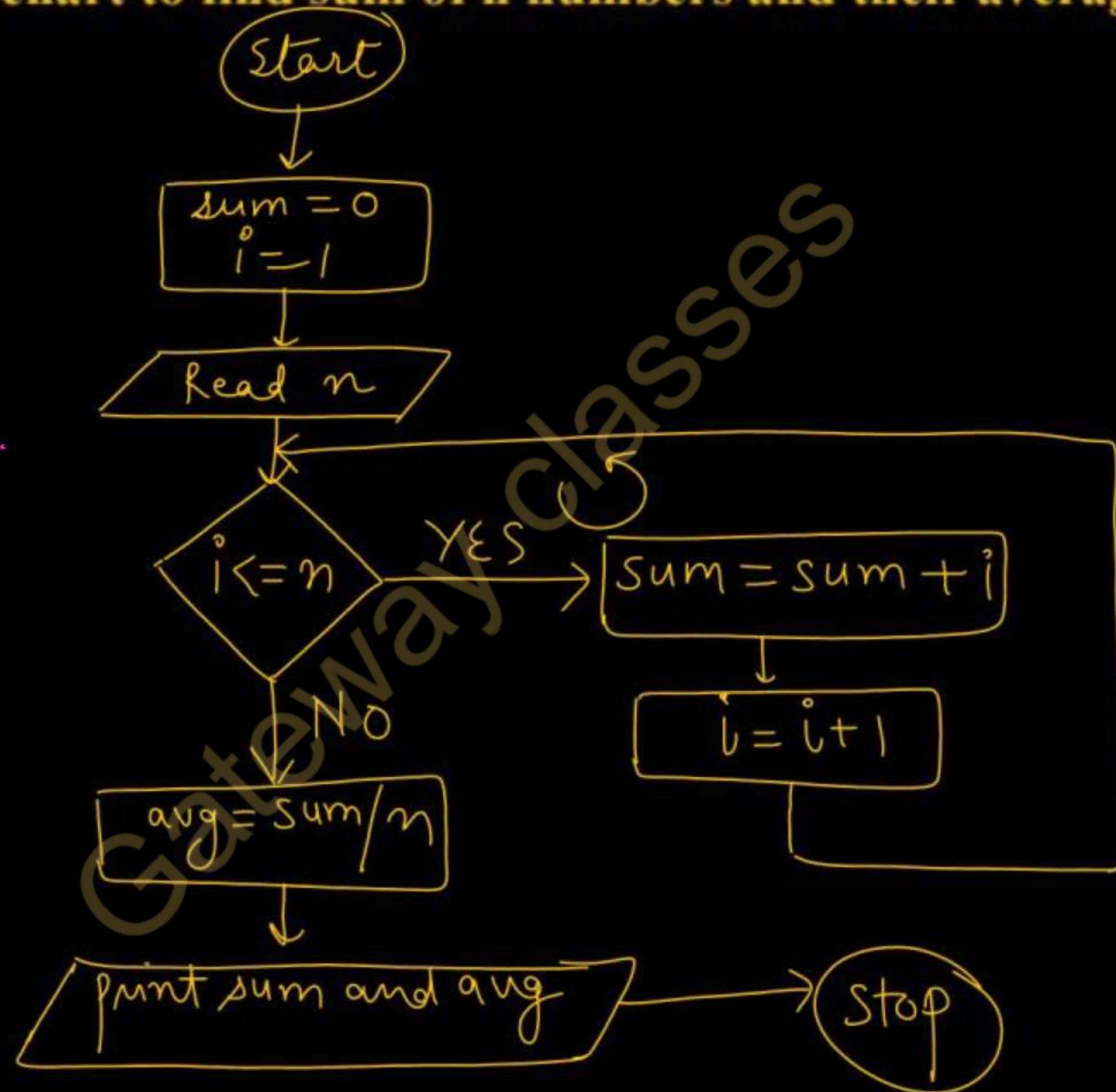
4.2 Compute $i = i + 1$

5. Calculate avg = sum / n

6. Print sum and avg

7. Stop

Flowchart to find sum of n numbers and their average



AKTU PYQs

1. Explain flow chart and benefits of using the flow chart in programming. Draw a flow chart to find the sum and average of n integers. (AKTU 2022-23)
2. Explain various symbols used in flow chart and benefits of using the flow chart in programming. Draw a flow chart to find the sum and average of n integers. (AKTU 2021-22)
3. Draw flowchart of swapping two numbers. (AKTU 2021-22)

C program to count number of digits in an integer.

$$\boxed{n = 325} \checkmark$$

$$\text{count} = 0$$

n ←
while ($n \neq 0$) ← while ($n > 0$)
{
 $n = n / 10$;
 count++;
}

(I) $\frac{325}{10} = 32$ count = 1

(II) $\frac{32}{10} = 3$ count = 2

(III) $\frac{3}{10} = 0$ count = 3

~~0~~

C program to count number of digits in an integer.

```
#include <stdio.h>
int main()
{
    int n, count = 0;
    printf("Enter the number");
    scanf("%d", &n); // 325
```

```
    while (n > 0)
    {
        n = n / 10;
        count = count + 1;
    }
```

⇒ logic to count
no. of digits

```
    printf("The no. of digits are = %d", count);
    return 0;
}
```

Let's take $n = 325$ count = 0

$n = 325$
 $(32570) T$
 $n = 325 / 10$
 $n = 32$
 $count = 0 + 1$
 $count = 1$

$n = 32$
 $(3270) T$
 $n = 32 / 10$
 $n = 3$
 $count = 1 + 1$
 $count = 2$

$n = 3$
 $(370) T$
 $n = 3 / 10$
 $n = 0$
 $count = 2 + 1$
 $count = 3$

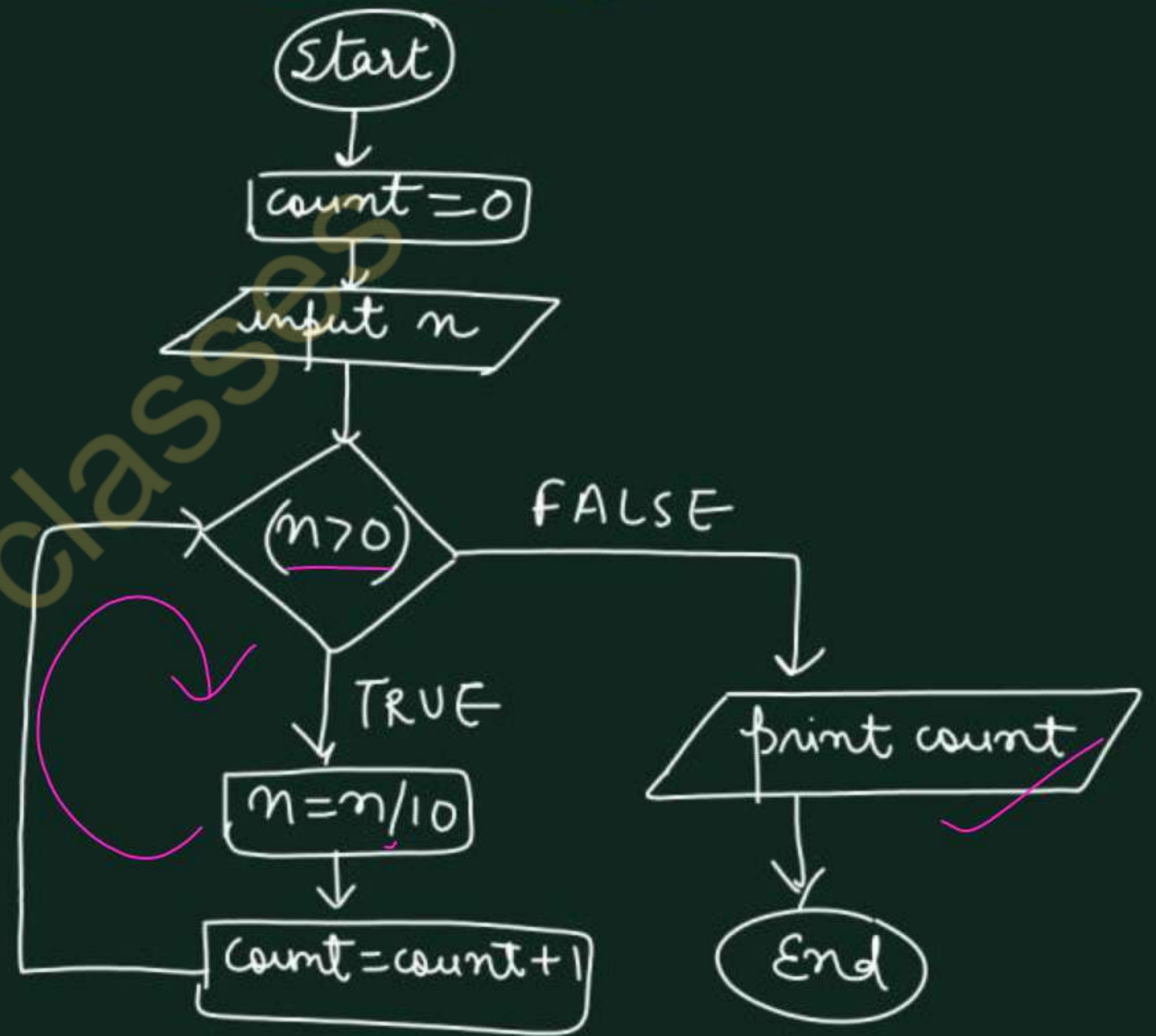
$n = 0$
 $(070) F$
↓
X

n	count
325	0
32	1
3	2
0	3

Algorithm

- 1) Start
- 2) Initialize count = 0
- 3) Input n
- 4) Repeat steps 5 and 6 while (n > 0)
- 5) Calculate $n = n/10$
- 6) Calculate $\text{count} = \text{count} + 1$
- 7) Print count.
- 8) End.

Flowchart



C program to find sum of first and last digit of any number.

Extracting last digit

$$n = 1328$$

$$d = n \% 10$$

$$d = 8 \Rightarrow \text{last digit}$$

$$\begin{array}{r} 132 \rightarrow \\ 10 \overline{) 1328} \\ \underline{10} \\ 32 \\ \underline{30} \\ 28 \\ \underline{20} \\ 8 \end{array}$$

Extracting last digit

$$n = 1328$$

$$(I) \quad n = n / 10 \quad n \Rightarrow 132$$

$$(II) \quad 132 / 10 \Rightarrow 13$$

$$(III) \quad 13 / 10 \Rightarrow 1$$

$$(IV) \quad 1 / 10 \Rightarrow 0$$

First digit
 $n = 1328$
Last digit

$(n \geq 10)$
condition

$$n \geq 0$$

$$n \geq 10$$

$$n \geq 103$$

C program to find sum of first and last digit of any number.

$$n = \underline{5836}$$

```
#include <stdio.h>
int main()
```

```
{ int n, fd, ld;
  printf("Enter the number");
  scanf("%d", &n);
```

```
  ld = n % 10;
```

```
  while (n >= 10)
  {
    n = n / 10;
  }
```

⇒ Logic to find first digit of a number.

```
  fd = n;
```

```
  printf("The sum of first and last digits is = %d", (fd + ld));
}
```

$$ld = 5836 \% 10 = \underline{6}$$
$$n = 5836$$

$$\begin{array}{l} \underline{n = 5836} \\ 5836 > 10 \text{ (T)} \\ n = 5836 / 10 \\ n = 583 \end{array}$$

$$\begin{array}{l} \underline{n = 583} \\ 583 > 10 \text{ (T)} \\ n = 583 / 10 \\ n = 58 \end{array}$$

$$\begin{array}{l} \underline{n = 58} \\ 58 > 10 \text{ (T)} \\ n = 58 / 10 \\ \boxed{n = 5} \end{array}$$

$$\begin{array}{l} \underline{n = 5} \\ 5 > 10 \text{ (F)} \\ \downarrow \\ \times \end{array}$$

This n=5 will be the first digit of the number.

C program to find sum of digits of a number.

```
#include <stdio.h>
int main()
```

```
{ int n, d, s=0;
  printf("Enter the number");
  scanf("%d", &n);
```

```
  while(n>0)
  {
    d = n%10;
    s = s + d;
    n = n/10;
  }
```

```
  printf("The sum of digits is = %d", s);
  return 0;
}
```

V. Imp
for AKTU

logic to find
sum of digits
of a number

n=513
513 > 0 (T)
d = 513 % 10
d = 3
s = 0 + 3
s = 3
n = 513 / 10
n = 51

n=513 s=0

n=51
51 > 0 (T)
d = 51 % 10
d = 1
s = 3 + 1

n=51/10
n=5

n=5
5 > 0 (T)
d = 5 % 10
d = 5
s = 3 + 1 + 5

n=5/10
n=0

n=0
0 > 0 (F)
X

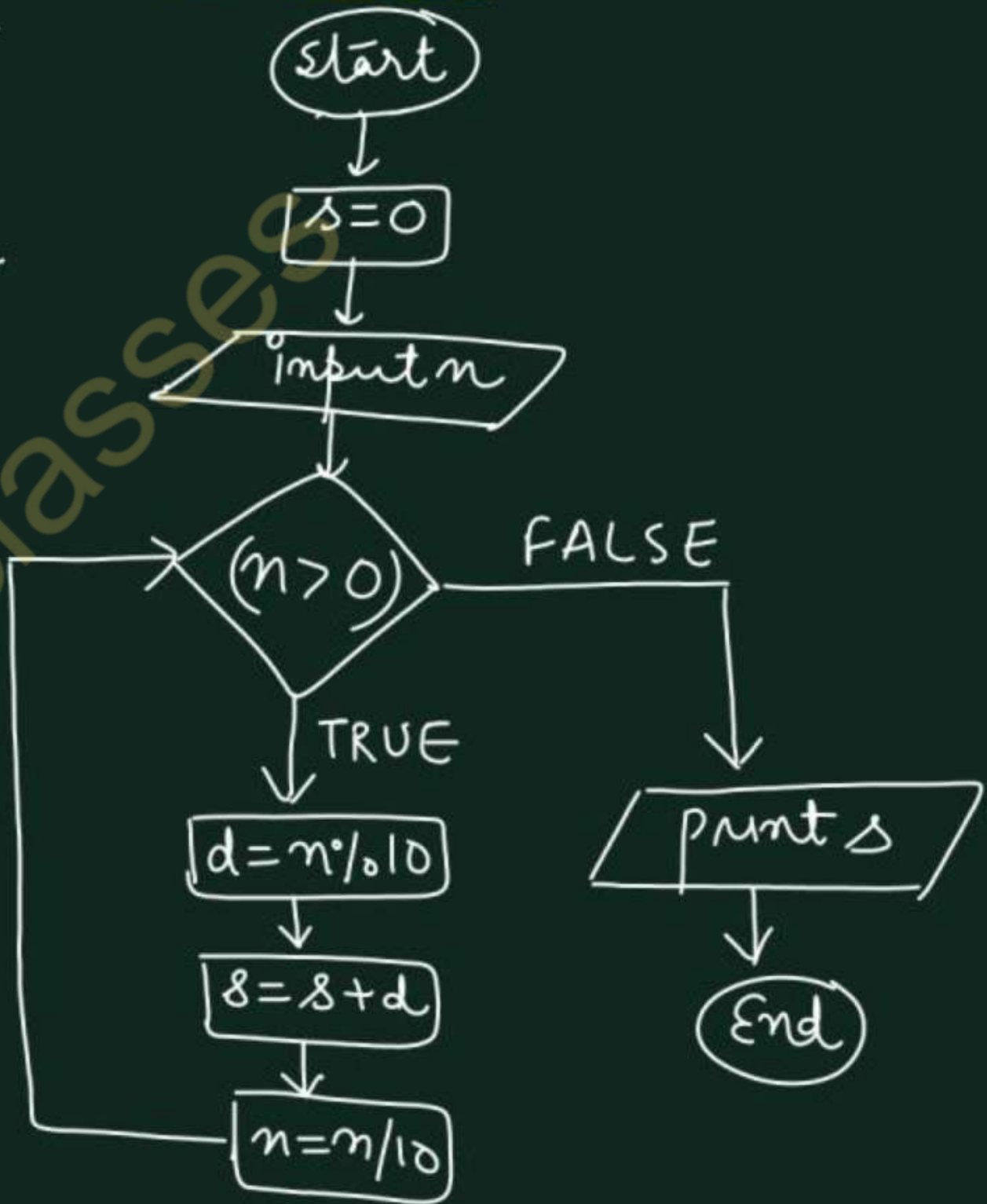
required
sum of digits

Algo

- 1) start
- 2) Initialize $s=0$
- 3) Input n .
- 4) Repeat steps 5, 6, 7 while $(n > 0)$
- 5) Calculate $d = n \% 10$
- 6) Calculate $s = s + d$
- 7) Set $n = n / 10$
- 8) Print s
- 9) End

Important
for
AKTU

Flowchart



C program to find reverse of a number.

$$\underline{n = 513}$$

$$\text{last digit} = \underline{3}$$

(I)

$$3 \times 10 = 30 + 1$$

$$31 \times 10$$

$$310 + 5$$

$$\underline{315}$$

C program to find reverse of a number. V. Imp

```
#include <stdio.h>
int main()
{
    int n, d, rev = 0;
    printf("Enter the number");
    scanf("%d", &n);
```

```
    while(n > 0)
    {
        d = n % 10;
        rev = rev * 10 + d;
        n = n / 10;
    }
```

⇒ logic
to find
reverse
of a number

```
    printf("The reverse is = %d", rev);
    return 0;
}
```

n = 513 rev = 0

n = 513

513 > 0 (T)

d = 513 % 10

d = 3

rev = rev * 10 + d
= 0 * 10 + 3

rev = 3

n = 513 / 10

n = 51

n = 51

51 > 0 (T)

d = 51 % 10

d = 1

rev = rev * 10 + d
= 3 * 10 + 1

rev = 31

n = 51 / 10

n = 5

n = 5 ✓

5 > 0 (T)

d = 5 % 10

d = 5

rev = rev * 10 + d
rev = 31 * 10 + 5

rev = 315

n = 5 / 10

n = 0

n = 0

0 > 0 (F)

↓

X

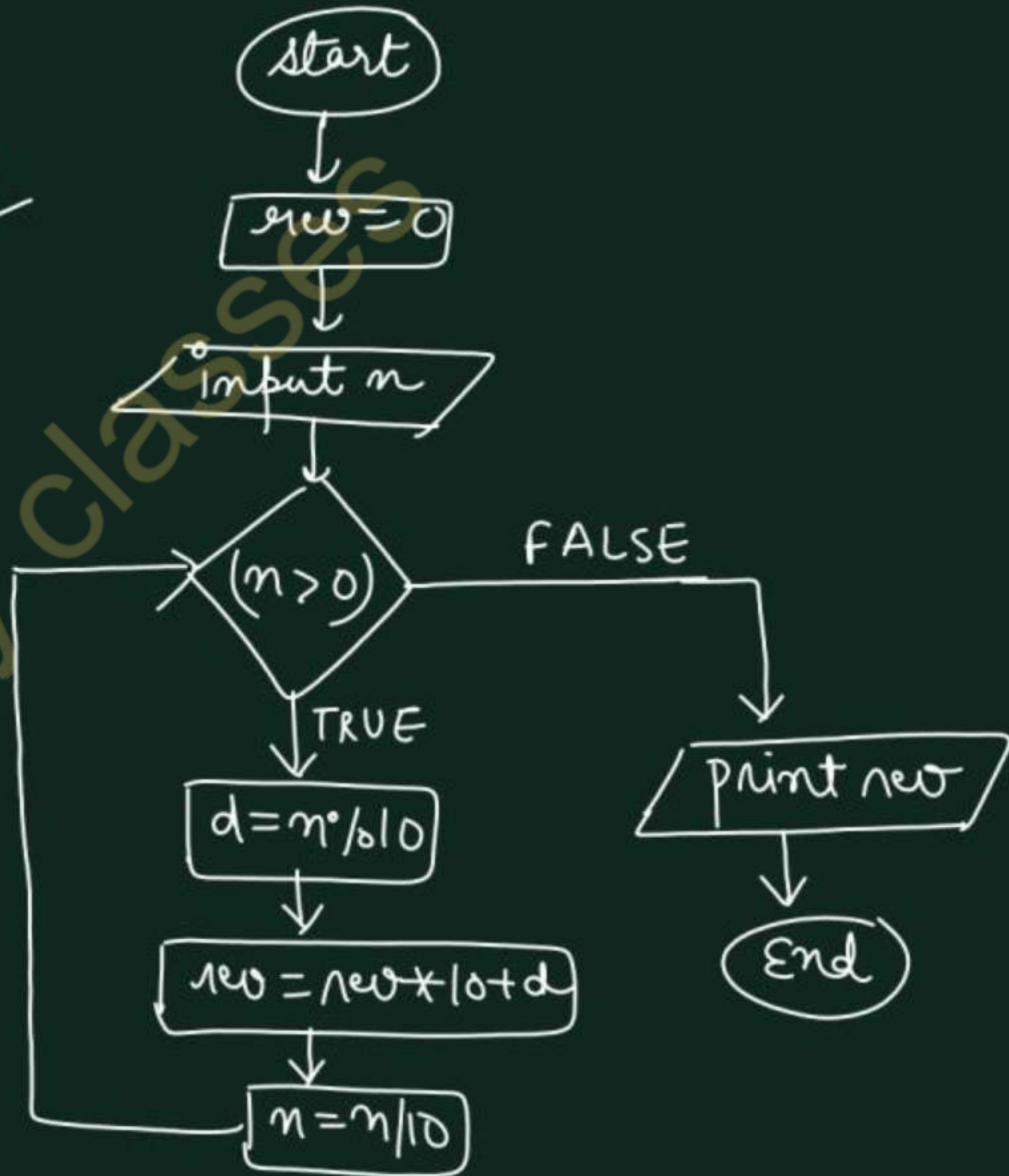
Required reverse
of number 513

Algorithm

- 1) start
- 2) Initialize new = 0
- 3) Input n.
- 4) Repeat steps 5, 6, 7 while $(n > 0)$
- 5) $d = n \% 10$
- 6) $new = new * 10 + d$
- 7) $n = n / 10$
- 8) Print new
- 9) End.

Important
for
AKTU

Flowchart



AKTU PYQs

1. ✓ WAP to print all natural numbers from 10 to 100. (AKTU 2023-24)
2. Program to print Fibonacci series up to n terms with algo and flowchart
3. ✓ Write an algorithm to calculate sum of digits of a number entered by user. (AKTU 2022-23)
4. ✓ Write a program in C to reverse a given number N having any number of digits. (AKTU 2022-23)
5. ✓ Write an algorithm and draw a flowchart to reverse a given number. (AKTU 2021-22)
6. Write a program to check a given number is Armstrong or not. (AKTU 2021-22)



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture-4

Topics to be Covered

- Program to check if a number is palindrome or not with algo and flowchart
- Program to check whether a number is Armstrong or not with algo and flowchart
- Program to print Fibonacci series up to n terms with algo and flowchart
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

program to check whether a number is palindrome or not.

$n = 5225$

reverse of $n = 5225$
 $n \leftarrow$

$temp = n;$

while ($n > 0$)

$n = n/10;$

if ($temp == new$)

$\{$

After completion
of this loop
 n becomes 0

A number which is
same after reversal

program to check whether a number is palindrome or not. (Very Important for AKTU)

```
#include <stdio.h>
int main()
```

```
{ int n, rev=0, d, temp;
  printf("Enter the number");
  scanf("%d", &n);
  temp = n;
```

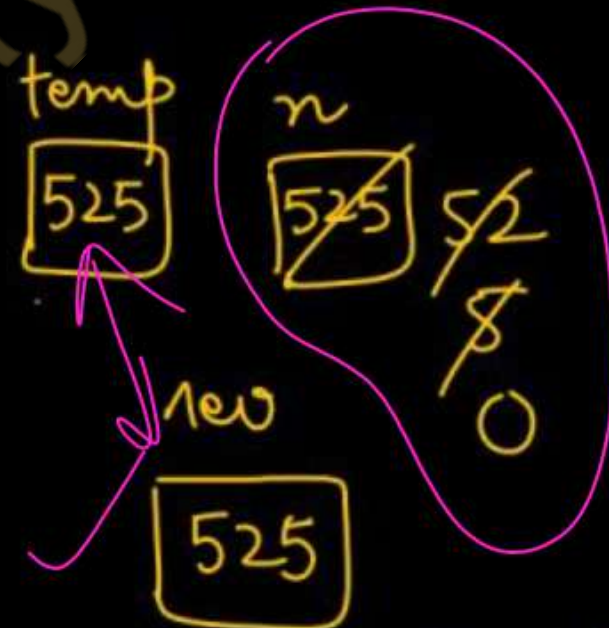
```
  while(n > 0)
  { d = n/10;
    rev = rev*10 + d;
    n = n/10;
  }
```

⇒ calculates reverse of a number

```
  if(rev == temp)
  { printf("The no. is palindrome");
  }
  else { printf("The no. is not palindrome");
  }
  return 0;
}
```

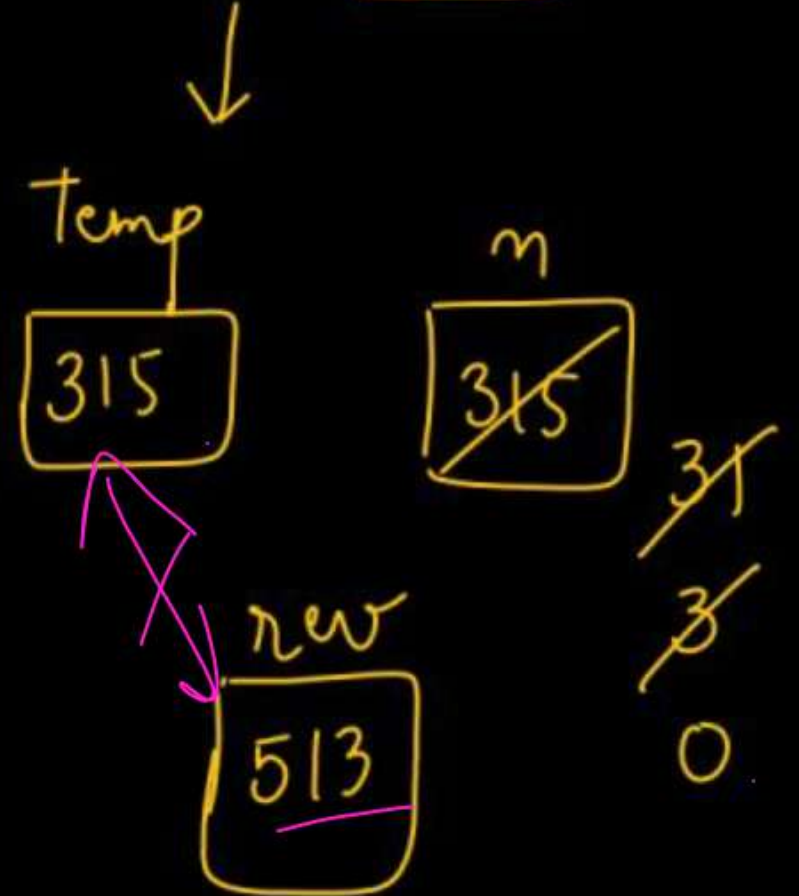
let n = 525

temp = 525



⇒ PALINDROME

let n = 315



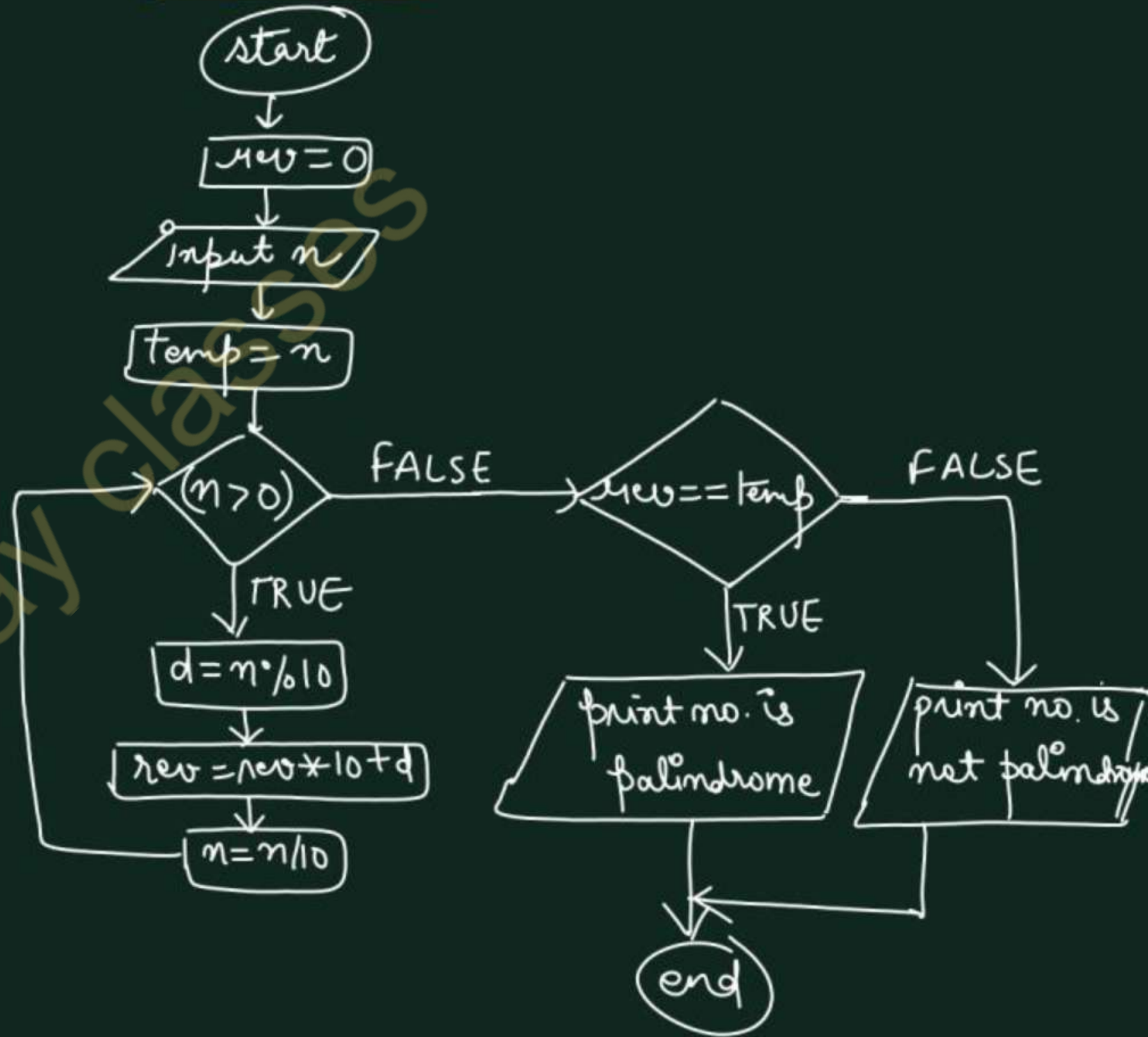
⇒ NOT PALINDROME

Algo

- 1) Start
- 2) Initialize $rev = 0$.
- 3) Input n .
- 4) Set $temp = n$.
- 5) Repeat steps 6, 7, 8 while($n > 0$)
- 6) $d = n \% 10$
- 7) $rev = rev * 10 + d$
- 8) $n = n / 10$
- 9) if ($temp == rev$)
 print "number is palindrome."
 else
 print "number is not palindrome"
10. End.

V. Imp
for
AKTU

Flowchart



program to check whether a number is Armstrong or not. (Very Important of AKTU)

$$\begin{array}{l} \text{3-digit} \\ \hline n = 153 \end{array}$$

$$\text{no. of digits} = \underline{\underline{3}}$$

$$\boxed{n = 153}$$

$$= 1^3 + 5^3 + 3^3$$

$$= 1 + 125 + 27$$

$$= \underline{\underline{153}}$$

Gateway classes

program to check whether a ^{three-digit} number is Armstrong or not. (Very Important of AKTU)

```
#include <stdio.h>
int main()
{
    int n, s=0, d, temp;
    printf("Enter the number");
    scanf("%d", &n);
    temp=n;
    while(n>0)
    {
        d=n%10;
        s=s+d*d*d;
        n=n/10;
    }
    if(s==temp)
    {
        printf("No. is Armstrong");
    }
    else
    {
        printf("No. is not Armstrong");
    }
    return 0;
}
```

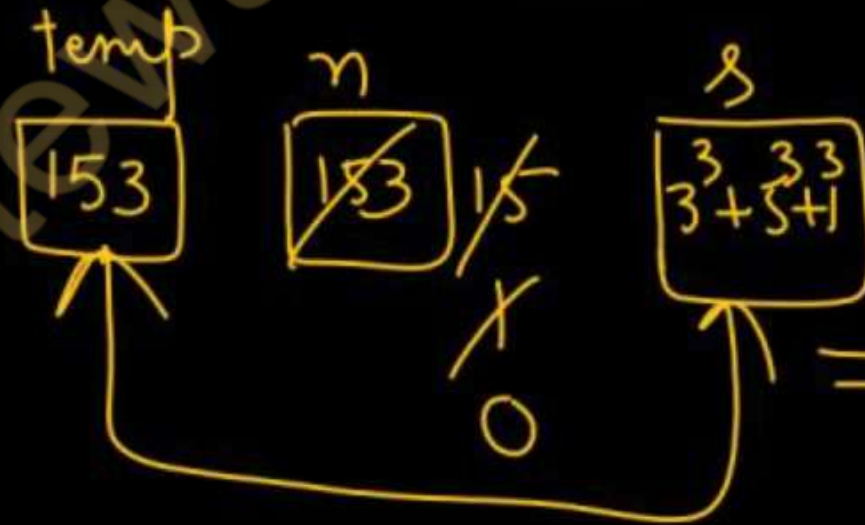
for 3-digit no.

n=153
 $153 > 0$ (T)
 $d = 153 \% 10$
 $d = 3$ ✓
 $s = 0 + 3 \times 3 \times 3$
 $s = 3^3$ ✓
 $n = 153 / 10$
n=15

n=153, s=0, temp=153 ✓
n=15
 $15 > 0$ (T)
 $d = 15 \% 10$
 $d = 5$ ✓
 $s = 3^3 + 5 \times 5 \times 5$ ✓
 $s = 3^3 + 5^3$
 $n = 15 / 10$
n=1

n=1
 $1 > 0$ (T)
 $d = 1 \% 10$
 $d = 1$
 $s = 3^3 + 5^3 + 1 \times 1 \times 1$
 $s = 3^3 + 5^3 + 1^3$
 $n = 1 / 10$
n=0

n=0
 $0 > 0$ (F)
 |
 X



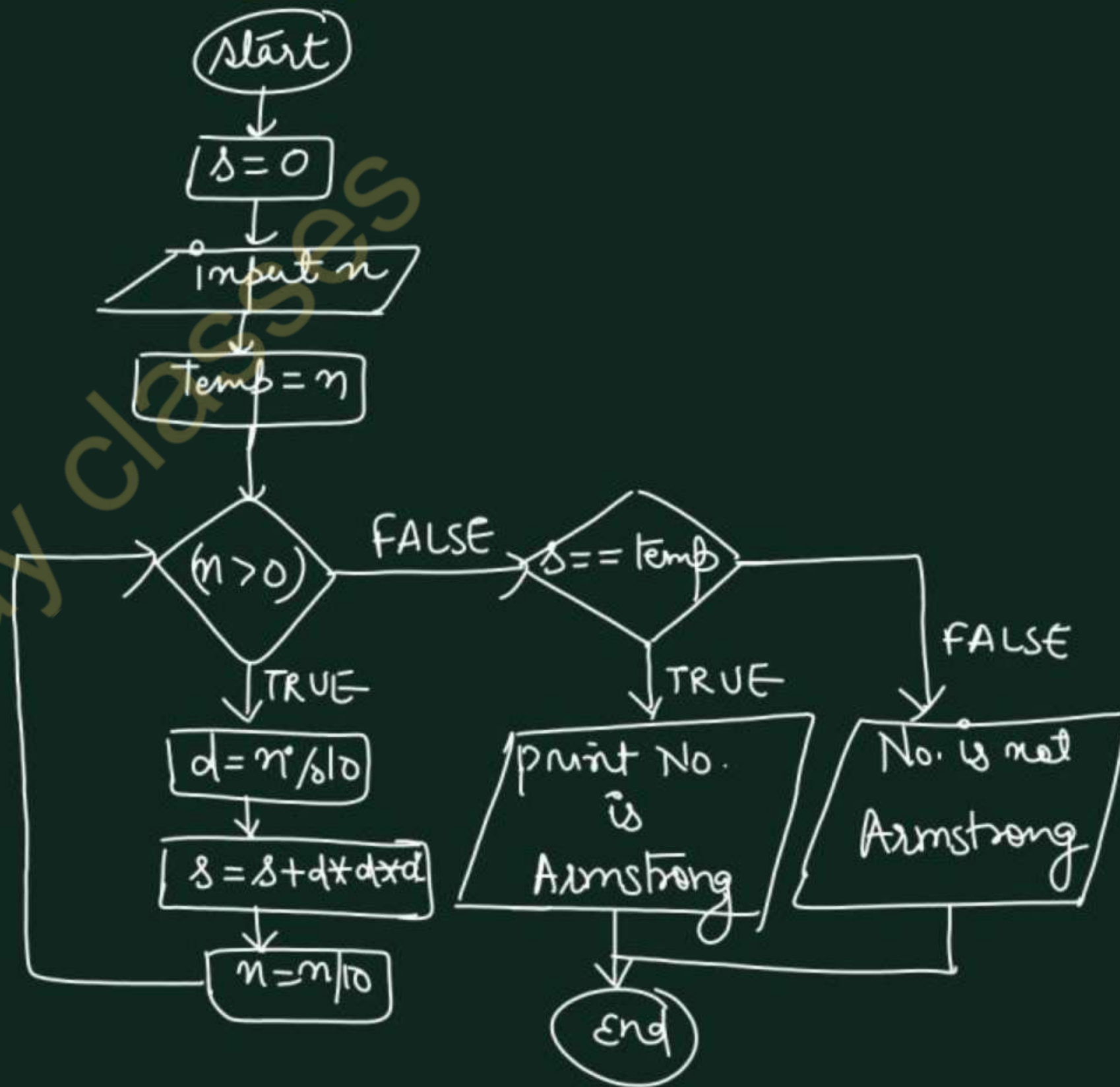
⇒ No is Armstrong

Algo

- 1) start.
- 2) Initialize $s=0$
- 3) Input n .
- 4) $temp = n$.
- 5) Repeat steps 6,7,8 while($n > 0$)
- 6) $d = n \% 10$
- 7) $s = s + d * d * d$
- 8) $n = n / 10$
- 9) if ($s == temp$)
 print "No. is Armstrong"
 else
 print "No. is not Armstrong"
- 10) End

V. Imp

Flowchart



Program to print Fibonacci series up to n terms



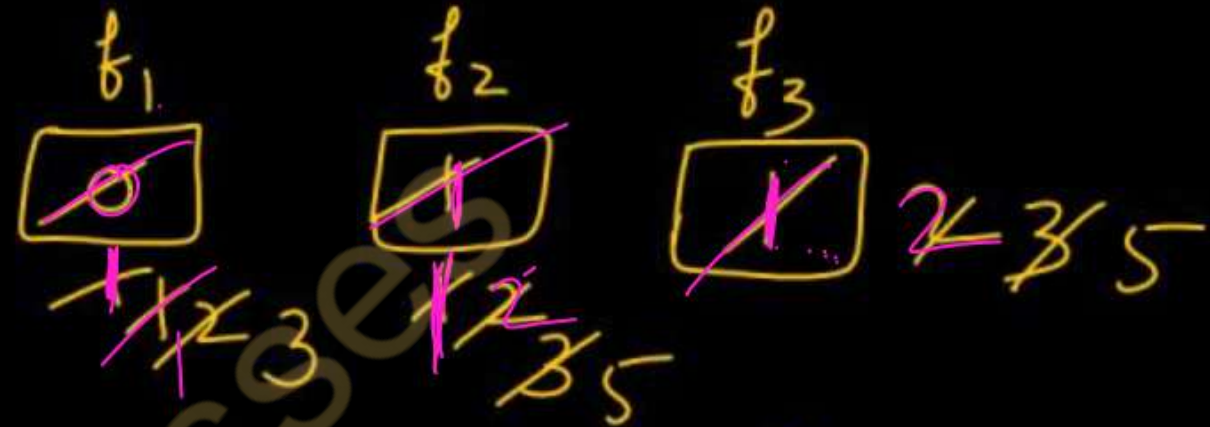
Program to print Fibonacci series up to n terms

```
#include <stdio.h>
int main()
{
    int n, i, f1, f2, f3;
    printf("Enter no. of terms");
    scanf("%d", &n);
    f1 = 0;
    f2 = 1;
    printf("%d\t%d", f1, f2);
    for (i = 1; i <= n - 2; i++)
    {
        f3 = f1 + f2;
        printf("\t%d", f3);
        f1 = f2;
        f2 = f3;
    }
    return 0;
}
```

Imp

Let $n = 6$ ✓

0 1 1 2 3 5 - - -

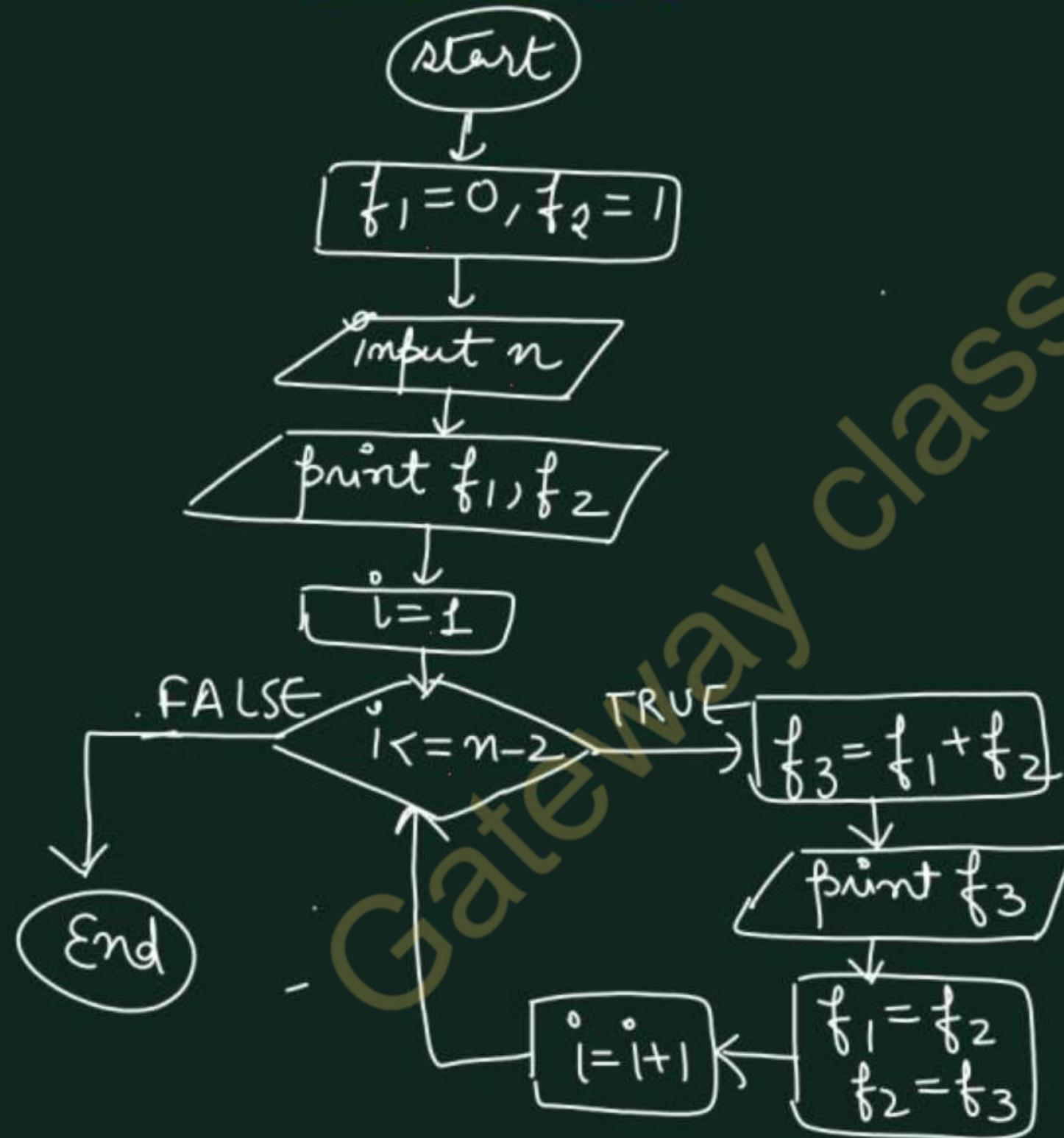


$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
$f_3 = 0 + 1 = 1$	$f_3 = 1 + 1 = 2$	$f_3 = 1 + 2 = 3$	$f_3 = 2 + 3 = 5$	$\downarrow$ X

Output:

0 1 1 2 3 5 ✓✓

Flowchart



AKTU PYQs

1. Write an algorithm and draw a flow chart to check whether the number entered by user is palindrome or not. (AKTU 2022-23)

Gateway classes



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture- 5

Topics to be Covered

- Loop Control Statements
 - break statement ✓
 - continue statement ✓
- Difference between break and continue
- Program to check whether a number is Prime or not
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Loop Control Statements in C

The C programming language supports several control statements. These include:

- break statement ✓
- continue statement
- goto statement

```
i = 1; ✓  
while (i <= 10) ✓  
{  
    printf("%d", i);  
    i++;  
}
```

break —
continue —

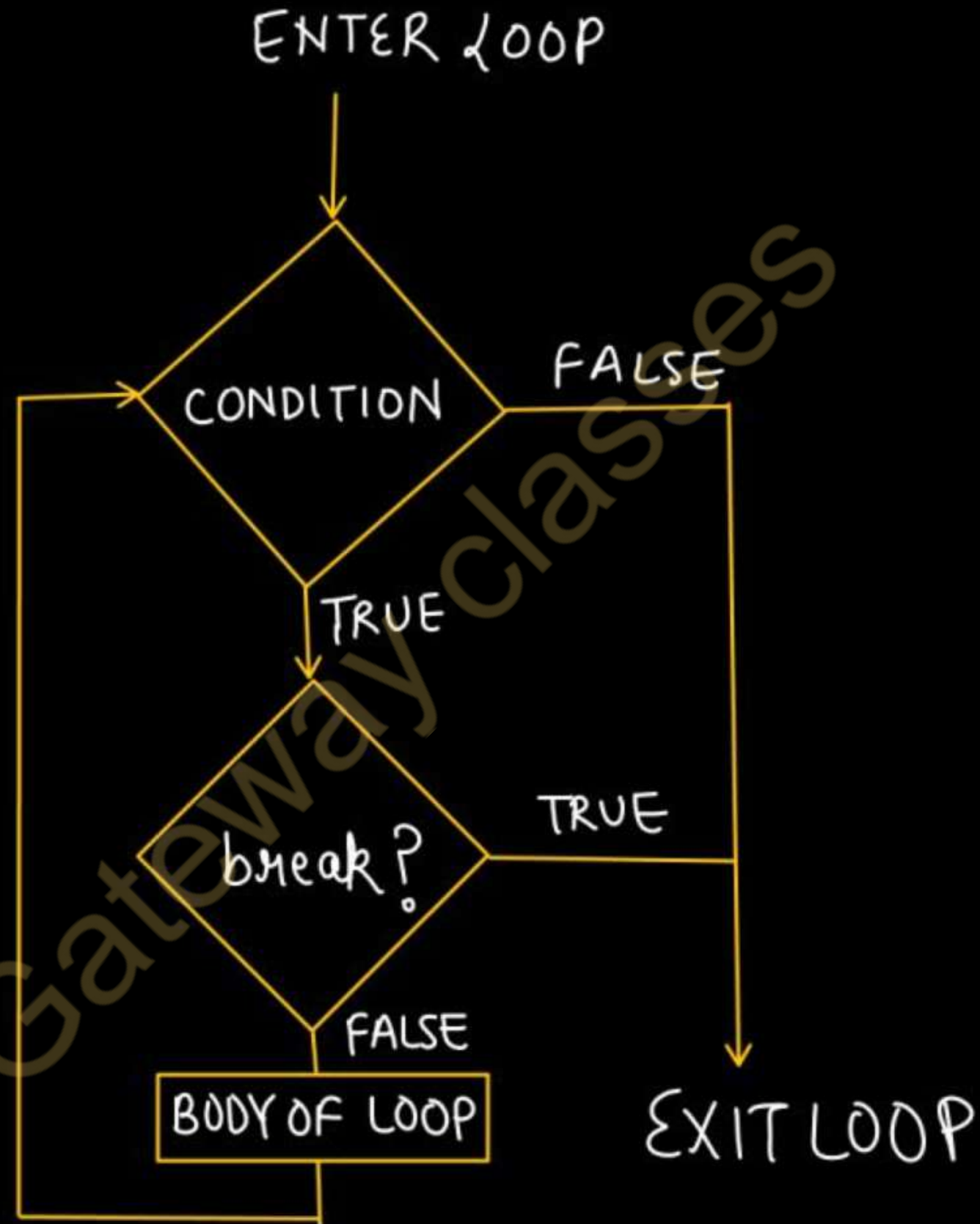
break Statement (Imp)

The break statement terminates the execution of the nearest enclosing loop in which it appears and brings the control to the next statement outside the loop.

It has two primary uses:

- When a break statement is encountered within a loop, the loop is immediately terminated and program control resumes at the next statement following the loop.
- It can be used to terminate a case in the switch statement.
- In case of nested loops, it terminates the innermost loop and begins executing the next line of code following the ~~block~~. *loop block*.

FLOWCHART OF break statement



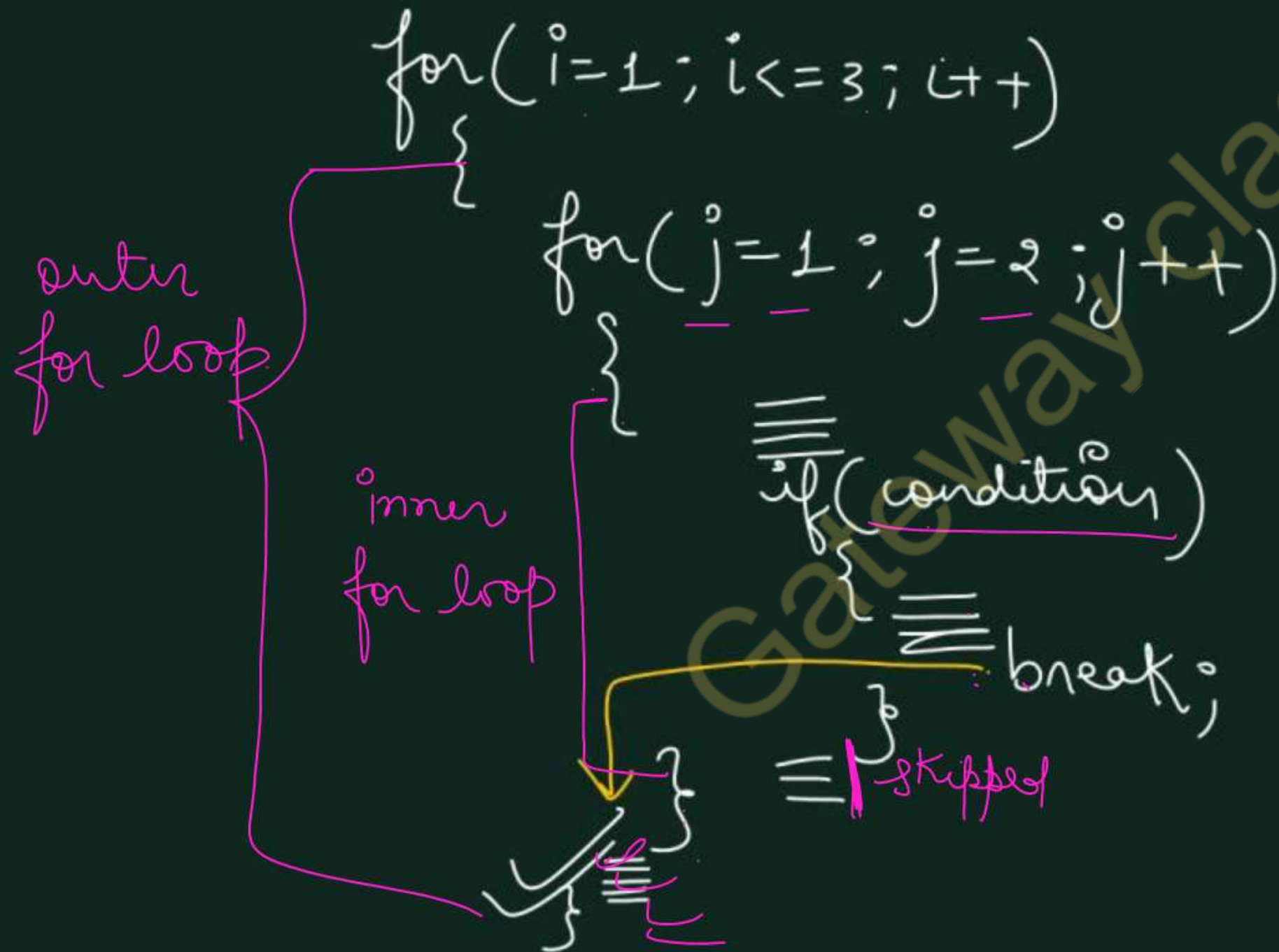
How break statement works?

```
while( condition )  
{  
    ≡  
    if( condition )  
    {  
        ≡  
        break;  
    }  
    ≡  
    } } skipped  
} }  
↓  
loop is terminated
```

```
for( init; condition; update )  
{  
    ≡  
    if( condition )  
    {  
        ≡  
        break;  
    }  
    ≡  
    } } skipped  
}  
↓  
loop terminates
```

```
do  
{  
    ≡  
    if( condition )  
    {  
        ≡  
        break;  
    }  
} while( condition );  
↓  
loop terminates
```


In Nested loops



Example: (AKTU)

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int a = 10;
```

```
    while( a < 20 )
```

```
    {
```

```
        printf("value of a: %d\n", a);
```

```
        a++;
```

```
        if( a > 15 )
```

```
        {
```

```
            break;
```

```
        }
```

```
    }  
    return 0;
```

```
}
```

output

value of a : 10

value of a : 11

value of a : 12

value of a : 13

value of a : 14

value of a : 15

Find the output of following code written in C-Language (AKTU 2022-23)

```
#include<stdio.h>
main()
```

```
{
    int i=1;
    for(;;)
    {
        printf("%d", i);
        if(i=5)
            break;
    }
}
```

i
 $\boxed{1}$ 5

output

```
int i=1;
for(;;)
{
    printf("%d", i);
    if(i==7)
    {
        break;
    }
    i++;
}
```

i
 $\boxed{1}$ 2 3 4 5 6 7 ✓

output

1 2 3 4 5 6 7

program to check whether a number is Prime or not. (AKTU)

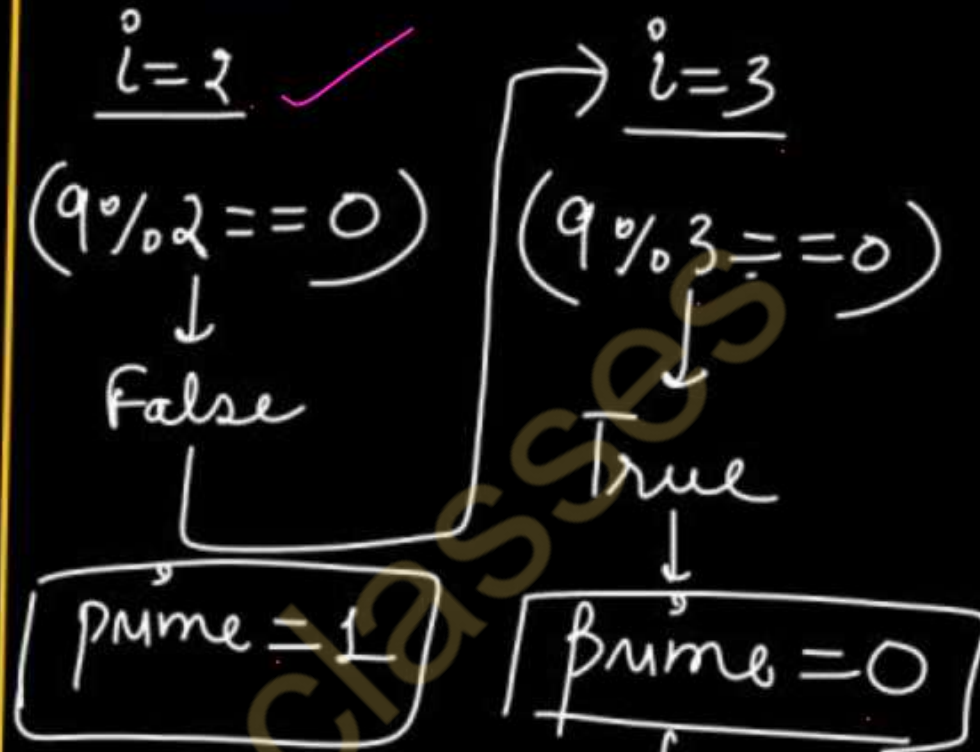
prime number — $\left\{ \begin{array}{l} \rightarrow \text{divisible by itself} \\ \rightarrow \text{divisible by 1} \end{array} \right.$

Gateway classes

program to check whether a number is Prime or not.

let take $n=9$, $prime=1$

```
#include <stdio.h>
int main()
{
    int n, prime=1, i;
    printf("Enter the number greater than 1");
    scanf("%d", &n);
    for( $i=2$ ;  $i \leq n/2$ ;  $i++$ )
    {
        if( $n \% i == 0$ )
        {
             $prime=0$ ;
            break;
        }
    }
    if( $prime == 1$ )
        printf("%d is prime", n);
    else
        printf("%d is not prime", n);
}
```



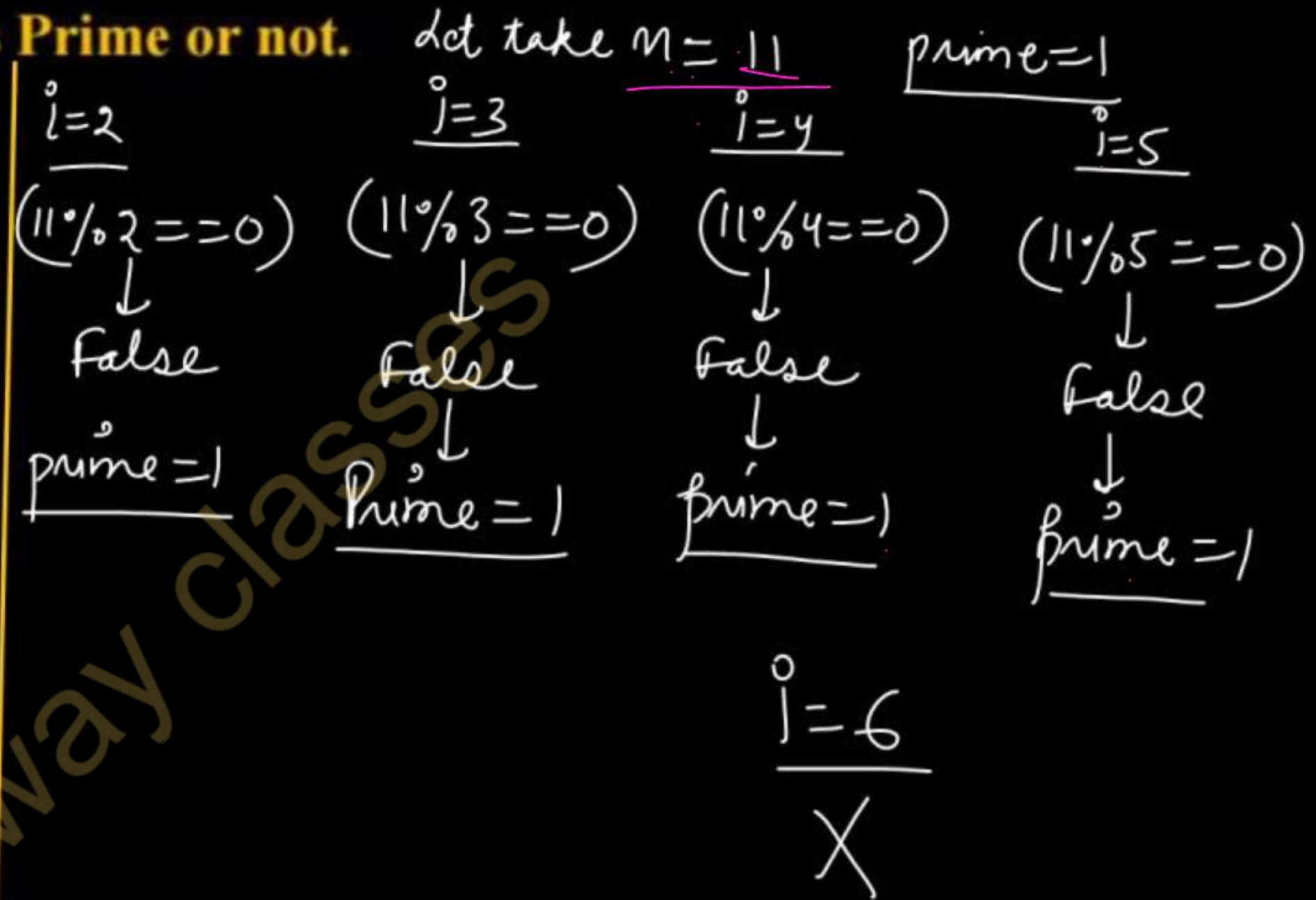
~~$i=4$~~

$9/2$
 $\Rightarrow 4.5$

Loop is terminated

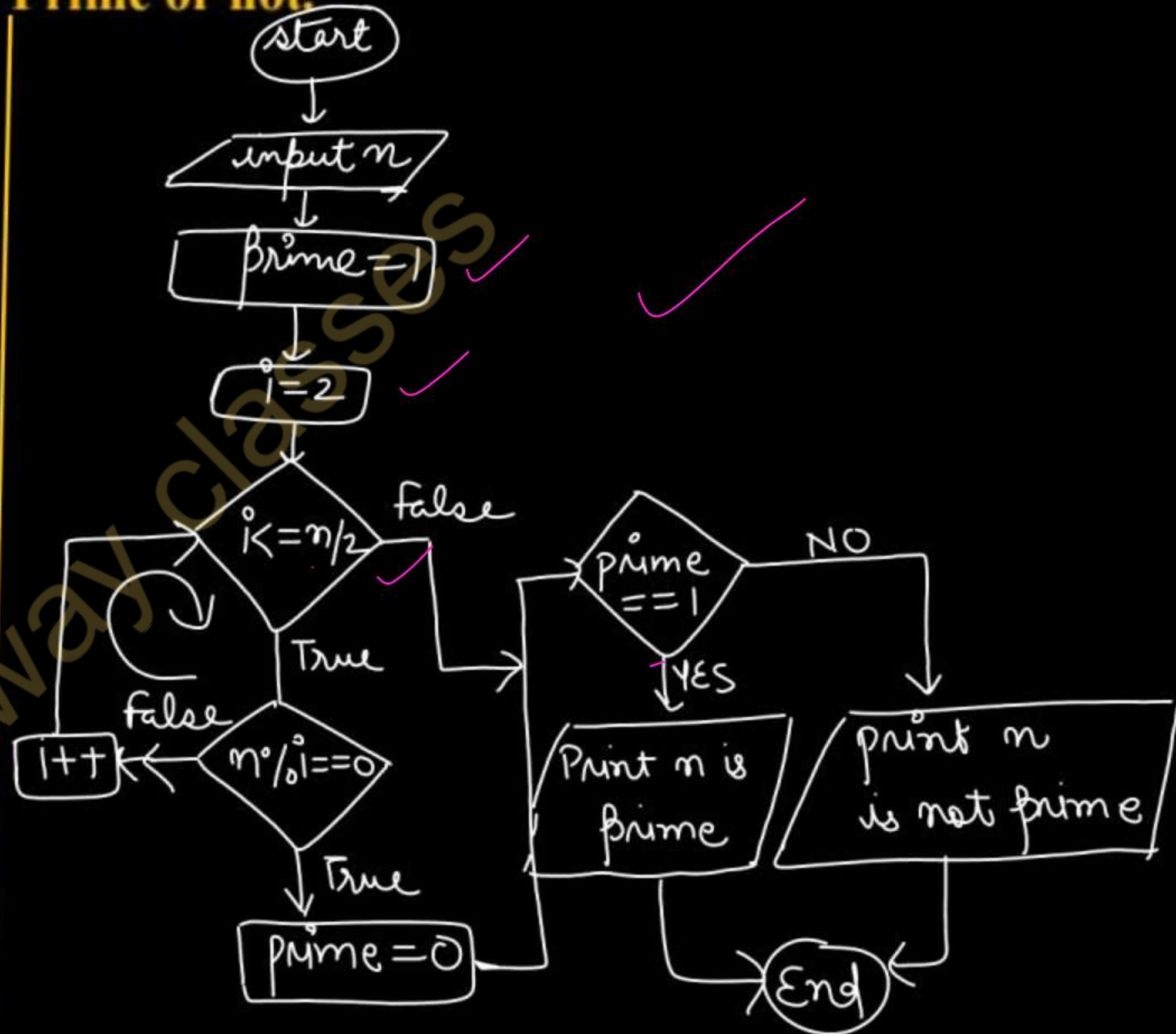
program to check whether a number is Prime or not.

```
#include <stdio.h>
int main()
{
    int n, prime=1, i;
    printf("Enter the number greater than 1");
    scanf("%d", &n);
    for(i=2; i<=n/2; i++)
    {
        if(n%i==0)
        {
            prime=0;
            break;
        }
    }
    if(prime==1)
        printf("%d is prime", n);
    else
        printf("%d is not prime", n);
}
```



program to check whether a number is Prime or not.

```
#include <stdio.h>
int main()
{
    int n, prime=1, i;
    printf("Enter the number greater than 1");
    scanf("%d", &n);
    for(i=2; i<=n/2; i++)
    {
        if(n%i==0)
        {
            prime=0;
            break;
        }
    }
    if(prime==1)
        printf("%d is prime", n);
    else
        printf("%d is not prime", n);
}
```



Give loop statements to get the following output

(AKTU 2017-18)

-8 -6 -4 -2 0 2 4 6 8

```
#include <stdio.h>
int main()
{
    int i;
    for (i = -8; i <= 8; i = i + 2)
    {
        printf("%d", i);
    }
    return 0;
}
```

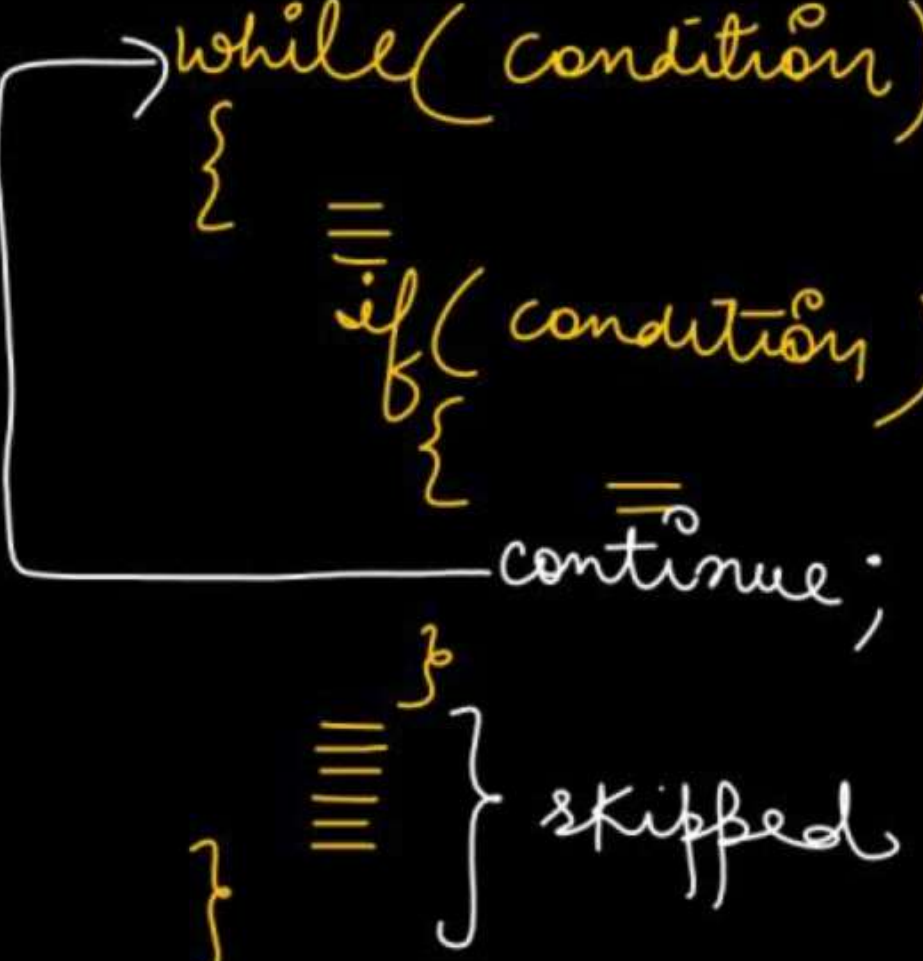

Continue statement:

- **This statement appears only in the body of the loop.**
- **Whenever the compiler encounters this statement the rest of the statements are skipped and control is transferred to the loop beginning of the nearest enclosing loop.**

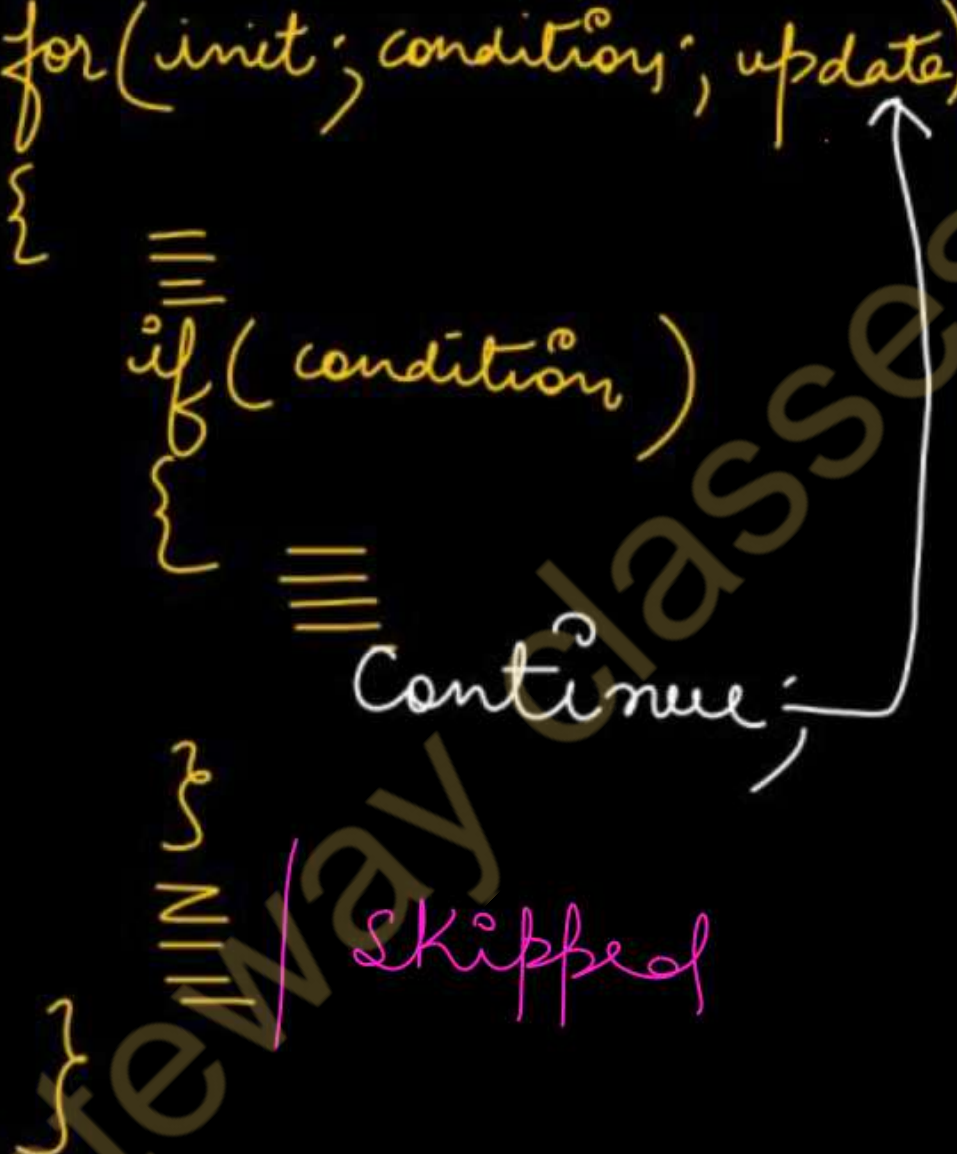
Gateway classes

How break statement works?

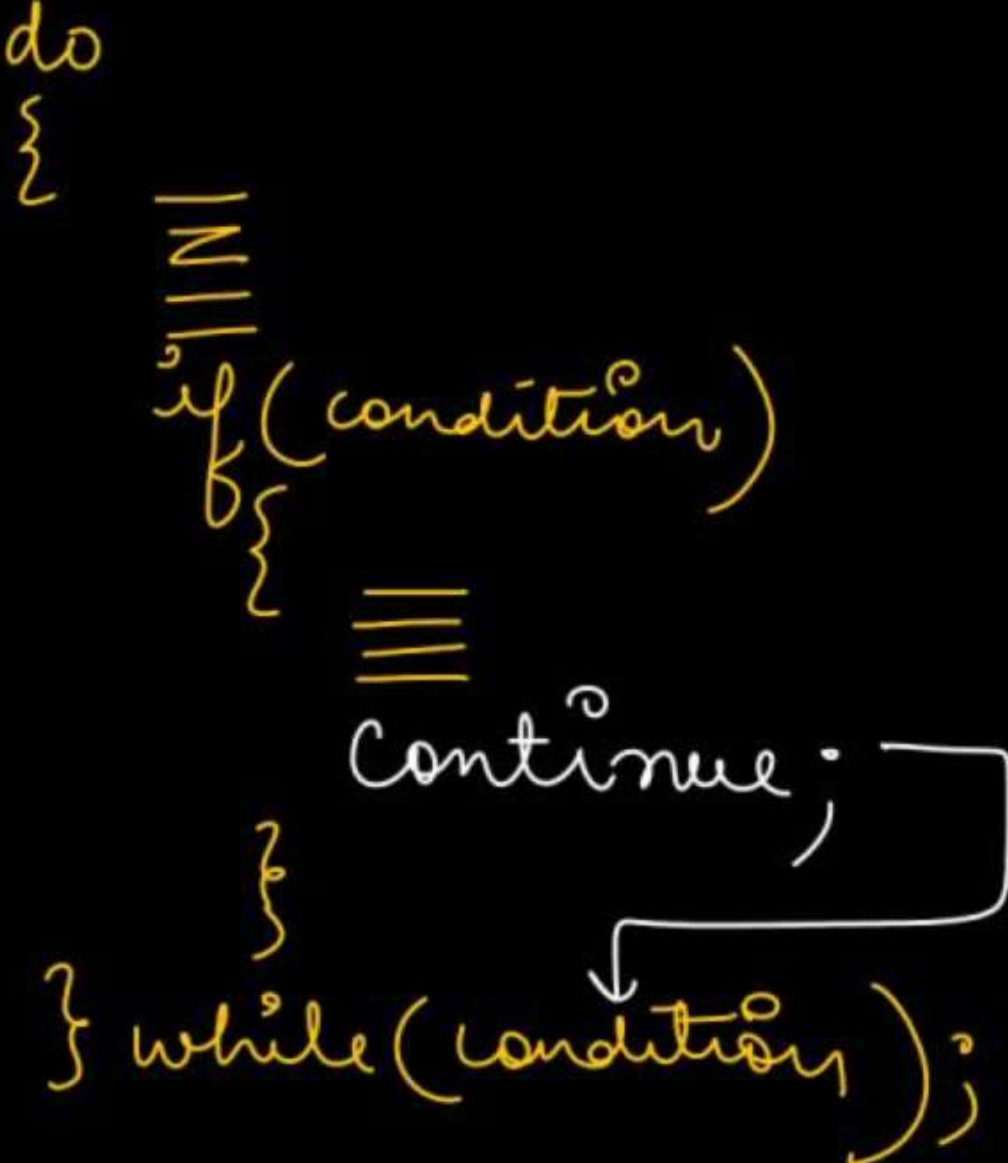
```
while( condition )  
{  
    if( condition )  
    {  
        continue;  
    }  
    } skipped  
}
```



```
for( init; condition; update )  
{  
    if( condition )  
    {  
        continue;  
    }  
    } skipped  
}
```

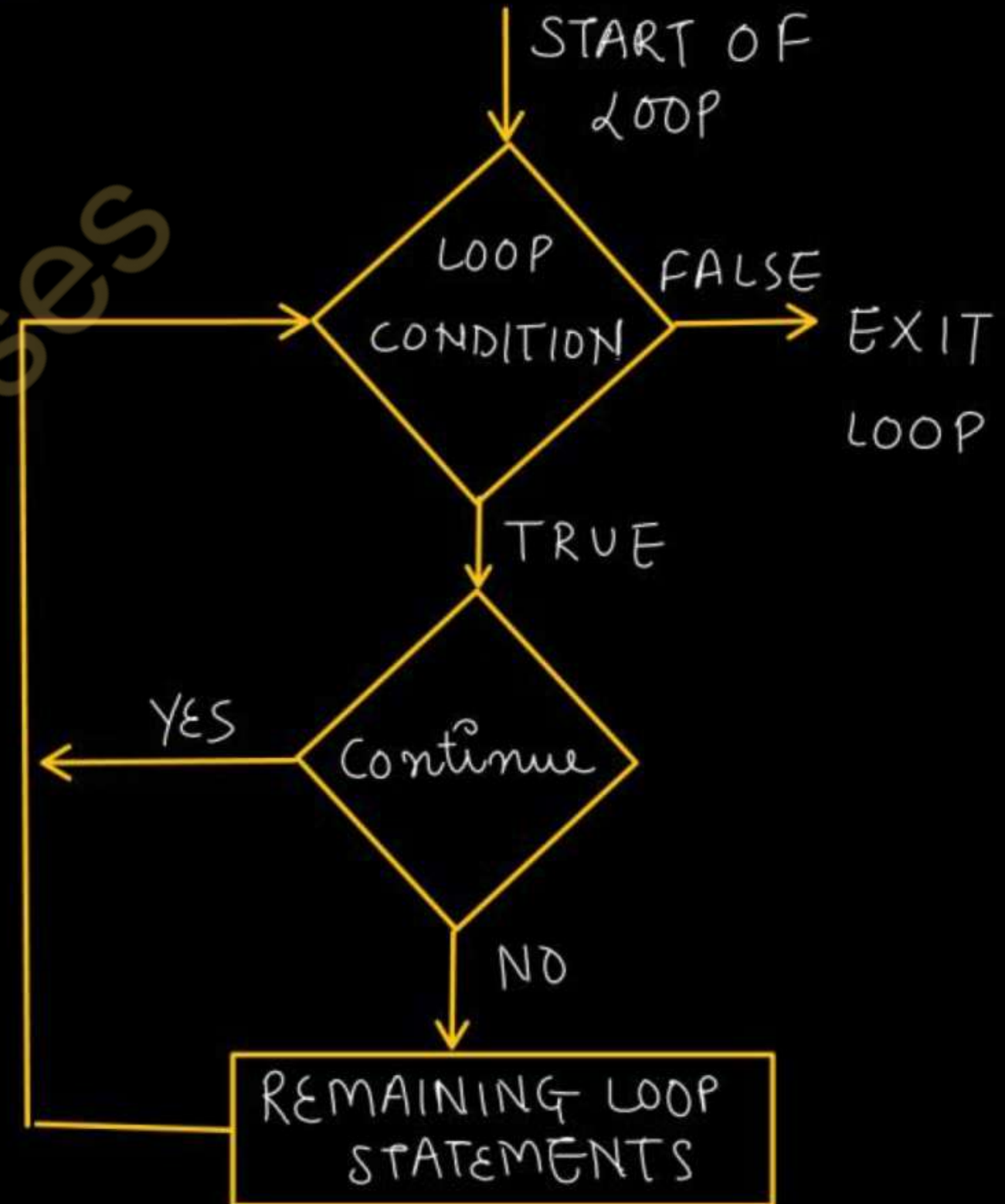


```
do  
{  
    if( condition )  
    {  
        continue;  
    }  
} while( condition );
```



How continue statement works?

FLOWCHART



Example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    for (int j=0; j<=8; j++)
```

```
    {
```

```
        if (j==4) ✓
```

```
        {
```

```
            continue;
```

```
        }
```

```
        printf("%d ", j); ✗
```

```
    } ✓
```

```
    return 0;
```

```
}
```



0 1 2 3 | 5 6 7 8

4 ✗



Difference between break and continue (AKTU) Imp

Basis For Comparison	break	continue
Task	It <u>terminates the execution of</u> remaining iteration of the loop.	It terminates only the <u>current iteration</u> of the loop.
Control after break/continue	Transfer control to the ^{next statement} <u>end of</u> ^{after the} loop enclosing that 'break'.	Transfer control of the program to the next <u>iteration of that loop</u> enclosing 'continue'.
Causes	causes <u>early termination</u> of loop.	causes early execution of next iteration.
Other uses	'break' can be used with 'switch'	'continue' can not be executed with 'switch'.

AKTU PYQs

1. Write an algorithm in C to check a given number is prime or not. AKTU(2023-24)(2021-22)
2. Differentiate between break and continue. (AKTU)
3. Write syntax of continue statement. (AKTU 2020-21)
4. How break statement work? (AKTU)
5. Explain the utility of break statement in loops with the help of a program. (AKTU)

✓ prime no. program



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture- 6 ✓

Topics to be Covered

- Nested loop
- Program and flowchart to print all Prime numbers from 1 to n
- Program and flowchart to print all Armstrong numbers from 1 to n
- Program and flowchart to print all Palindrome numbers from 1 to n
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Nested Loops

- Nesting of loops allows the looping of statements inside another loop.
- Any number of loops can be defined inside another loop, i.e., there is no restriction for defining any number of loops.
- We can define any type of loop inside another loop; for example, you can define 'while' loop inside a 'for' loop.

Gateway classes

Example

outer loop {
 for (i = 1; i <= 2; i++)
 {
 inner loop {
 for (j = 1; j <= 2; j++)
 {
 printf("%d %d", i, j);
 }
 }
 printf("\n");
 }
}

output

1	1	1	2
2	1	2	2

i = 1 {
 j = 1
 j = 2

output
1 1

1 2

i = 2 {
 j = 1
 j = 2

2 1

2 2

Procedure

- First, the 'i' variable is initialized to 1 and then program control passes to the $i \leq 2$.
- The program control checks whether the condition ' $i \leq 2$ ' is true or not.
- If the condition is true, then the program control passes to the inner loop.
- The inner loop will get executed until the condition is true.
- After the execution of the inner loop, the control moves back to the update of the outer loop, i.e., $i++$.
- After incrementing the value of the loop counter, the condition is checked again, i.e., $i \leq 2$.
- If the condition is true, then the inner loop will be executed again.
- This process will continue until the condition of the outer loop is true.

program to print all Prime numbers from 1 to n. (Imp for AKTU)
100 to 500

```
#include <stdio.h>
int main()
{
    int i, n, prime;
```

```
for (n = 100; n <= 500; n++)
```

↗ This code checks
if a no. is
prime or not

```
    prime = 1;
    for (i = 2; i <= n/2; i++)
    {
        if (n % i == 0)
        {
            prime = 0;
            break;
        }
    }
    if (prime == 1)
        printf("%d\t", n);
}
```

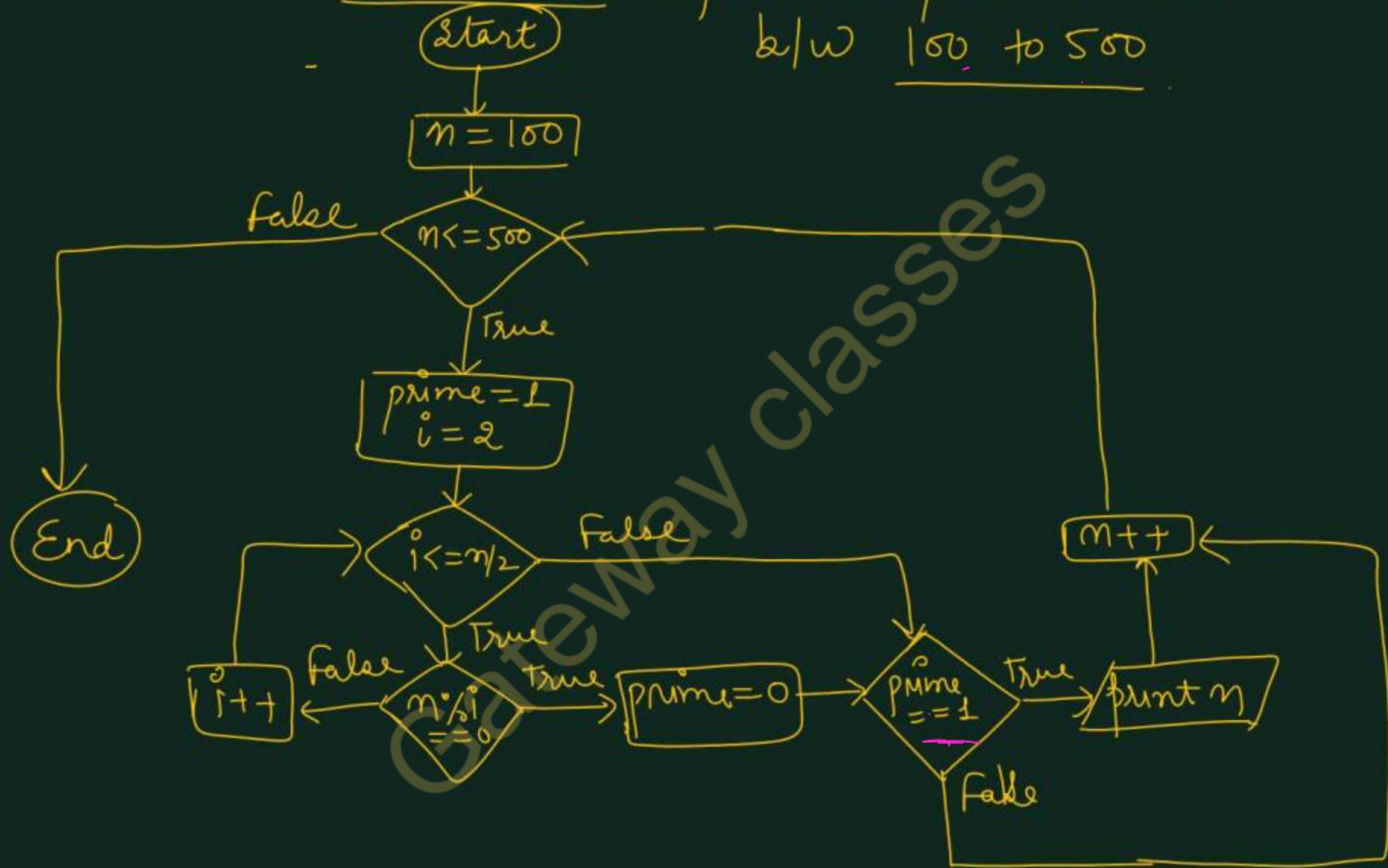
```
for (i = 1; i <= n; i++)
```

```
    prime = 1;
```

```
    for (j = 2; j <= i/2; j++)
```

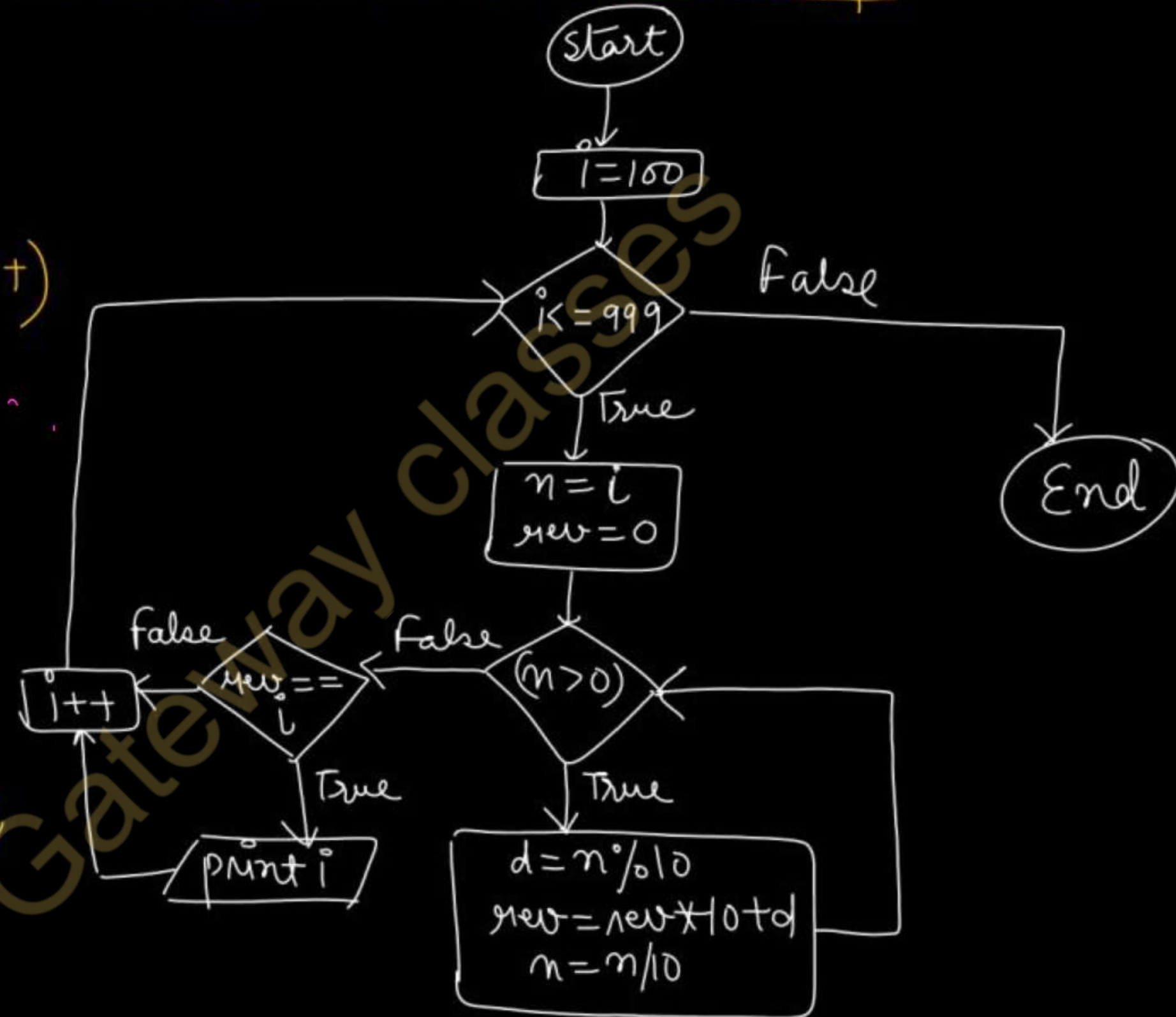
```
    {
        if (i % j == 0)
```

Flowchart to print all prime number
b/w 100 to 500



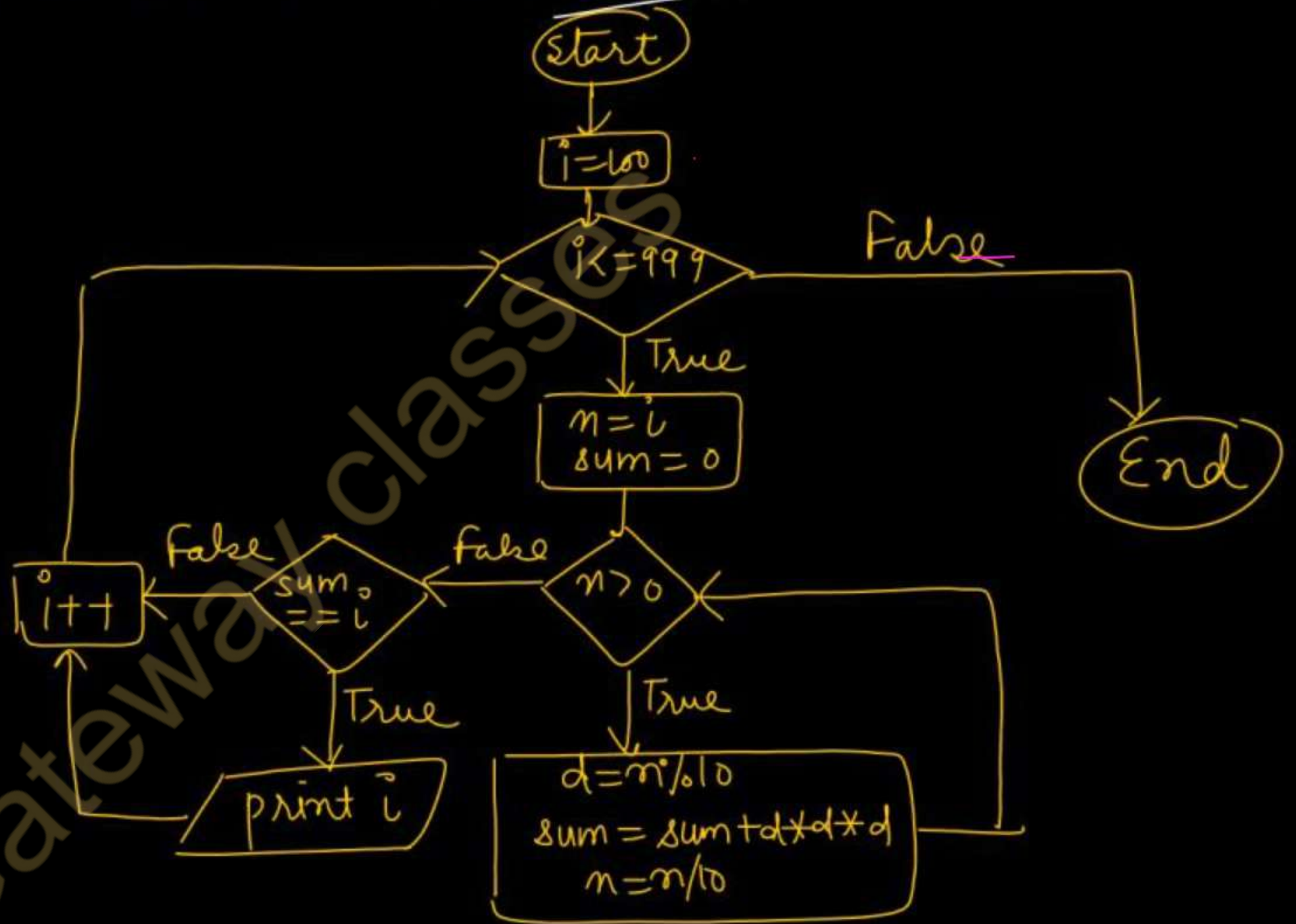
program to print all Palindrome numbers from 1 to n. 100 to 999

```
#include <stdio.h>
int main()
{
    int n, i, d, new;
    for (i = 100; i <= 999; i++)
    {
        n = i; new = 0;
        while (n > 0)
        {
            d = n % 10;
            new = new * 10 + d;
            n = n / 10;
        }
        if (new == i)
            printf("%d\n", i);
    }
}
```



program to print all Armstrong numbers from 1 to 500. 100 to 999

```
#include <stdio.h>
int main()
{
    int i, n, sum, d;
    for(i=100; i<=999; i++)
    {
        n=i;
        sum=0;
        while(n>0)
        {
            d=n%10;
            sum=sum+d*d*d;
            n=n/10;
        }
        if(sum==i)
            printf("%d\n", i);
    }
}
```



AKTU PYQs

1. Write a program to print All prime numbers from 1 to 100. (AKTU 2022-23)
2. Write a program to print All Armstrong numbers from 1 to 500 and draw its flowchart. (AKTU)
3. Write a program to print All palindrome numbers from 1 to n. (AKTU)

Gateway classes

DPP

1. Write an algorithm to print All prime numbers from 1 to 100.
2. Write algorithm to print All Armstrong numbers from 1 to 500.
3. Write algorithm to print All palindrome numbers from 1 to n.

Gateway classes



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture- 7

Topics to be Covered

- Introduction to Array
 - One dimensional Array
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Array

stored at adjacent memory locations

- An array is a collection of similar type of data items and each data item is called an element of the array.

`int a[5];`

- The data type of the elements may be any valid data type like char, int or float.
- The elements of array share the same variable name but each element has a different index number known as subscript.
- Arrays can be single dimensional or multidimensional.
 1-D 2-D, 3-D
- The number of subscripts determines the dimensions of array. A one-dimensional array has one subscript, two-dimensional array has two subscripts and so on.
- In C, the subscripts start from zero.

Accessing elements in One Dimensional Array

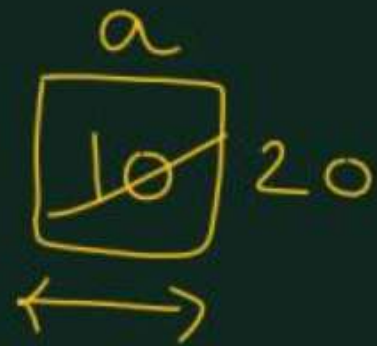
- The elements of an array can be accessed by specifying the array name followed by subscript in brackets.
 $a[0]$ $a[1]$
- In C, the array subscripts start from 0. Hence if there is an array of size 5 then the valid subscripts will be from 0 to 4.
- here 0 is the lower bound and 4 is the upper bound of the array.
- In C there is no check on bounds of the array.
- If we have an array `arr[5]`, the valid subscripts are only 0, 1, 2, 3, 4 and if someone tries to access elements beyond these subscript like `arr[5]`, `arr[10]` or `arr[-1]`, the compiler will not show any error message.

Single variable

Declaration

int a = 10;

a = 20



Array

int a[5];

1-D
Array

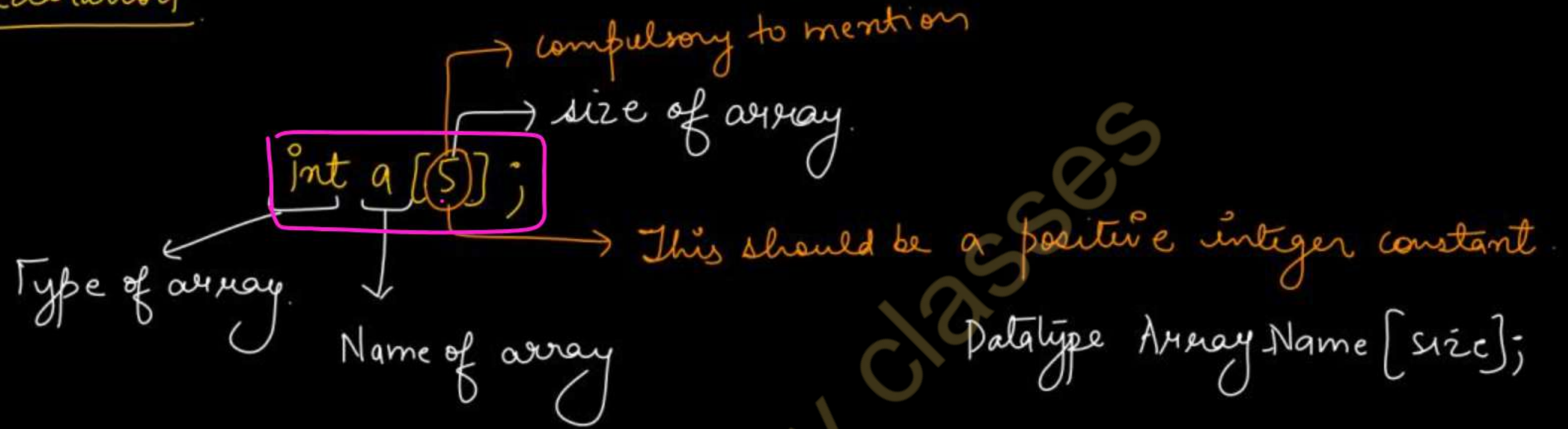
2-D
Array

3-D Array

Multidimensional arrays

One Dimensional Array

Declaration



- `a` is an integer array with size 5.
- OR
- `a` is an array that can store max 5 integer values.

`int a[];`
X compilation error

One Dimensional Array

Initialising 1-D Array

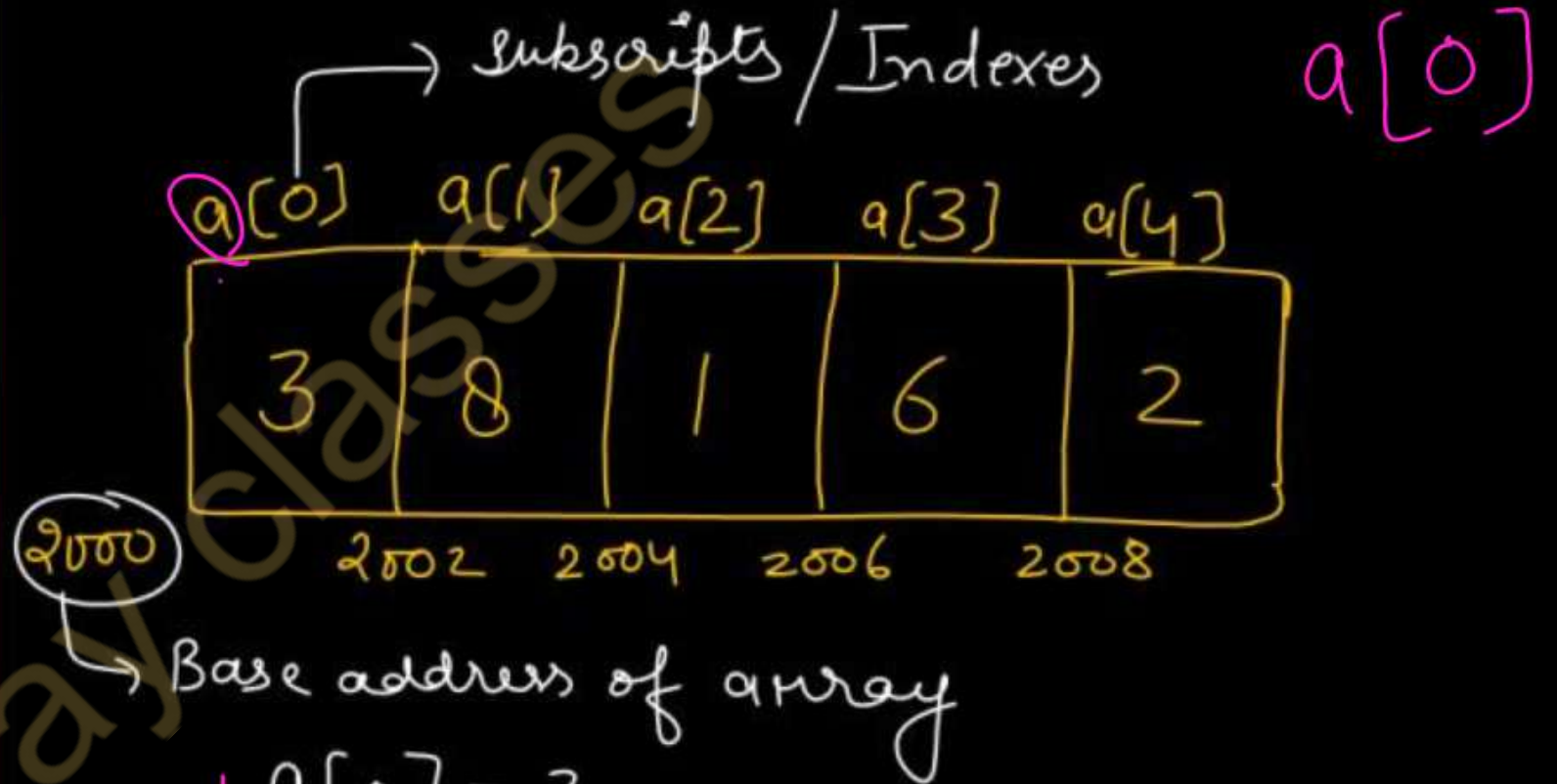
int a[5] = { 3, 8, 1, 6, 2 };
 ↑ optional

OR

int a[] = { 3, 8, 1, 6, 2 };

int a = 10;
 ↑
 1000
 2 bytes

Representing 1-D Array into Memory



a[0] = 3

a[1] = 8

a[2] = 1

a[3] = 6

a[4] = 2

One Dimensional Array (Accessing 1-D Array)

How to Input a 1-D array

int a[5];

a

a[0]	a[1]	a[2]	a[3]	a[4]
2	5	1	8	4

```
{ for (i=0; i<=4; i++)  
{  
    scanf("%d", &a[i]);  
}
```

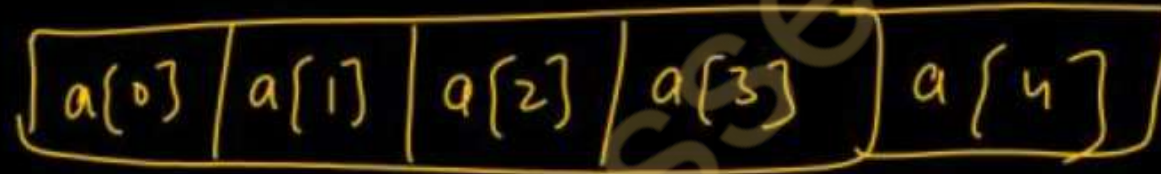
How to print 1-D array

```
for (i=0; i<=4; i++)  
{  
    printf("%d", a[i]);  
}
```


One Dimensional Array

Note :- There is no bound checking in case array provided by "C language".

```
int a[5];
```



```
for (i=0; i <= 5; i++)
```

```
{
```

```
    scanf("%d", &a[i]);
```

```
}
```

// compiler will not produce any error.

✓ Program to input an array of n integers and print them.

```
#include <stdio.h>
int main()
{
    int a[100], n, i;
```

max size of array

```
    printf("Enter the no. of array elements");
    scanf("%d", &n);
```

```
    printf("Enter array elements");
```

```
    for(i=0; i<=n-1; i++)
    {
        scanf("%d", &a[i]);
    }
```

$n=3$

$a[0]$	$a[1]$	$a[2]$
8	1	6

```
    printf("The array elements are:");
```

```
    for(i=0; i<=n-1; i++)
```

```
    {
        printf("%d\n",  $a[i]$ );
    }
```

```
    return 0;
```

```
}
```

output :

8
1
6

Program that takes elements in an array and print sum of all elements of it

```
#include <stdio.h>
```

```
int main()
```

```
{ int a[100], n, i, s=0;
```

```
printf("Enter the no. of elements in array");
```

```
scanf("%d", &n);
```

```
for(i=0; i<=n-1; i++)
```

```
{ scanf("%d", &a[i]);
```

```
  s = s + a[i];
```

```
}
```

```
printf("The sum of all array  
elements is = %d", s);
```

```
return 0;
```

```
}
```

$n=3$, $s=0$

$i=0$

$a[0]$ is input

$s = 0 + a[0]$

$s = 0 + 5 \Rightarrow s = 5$

$i=1$

$a[1]$ is input

$s = s + a[i]$

$s = 5 + 3 \Rightarrow s = 8$

$i=2$

$a[2]$ is input

$s = s + a[i]$

$s = 8 + 4 \Rightarrow s = 12$

$i=3$

$\rightarrow X$

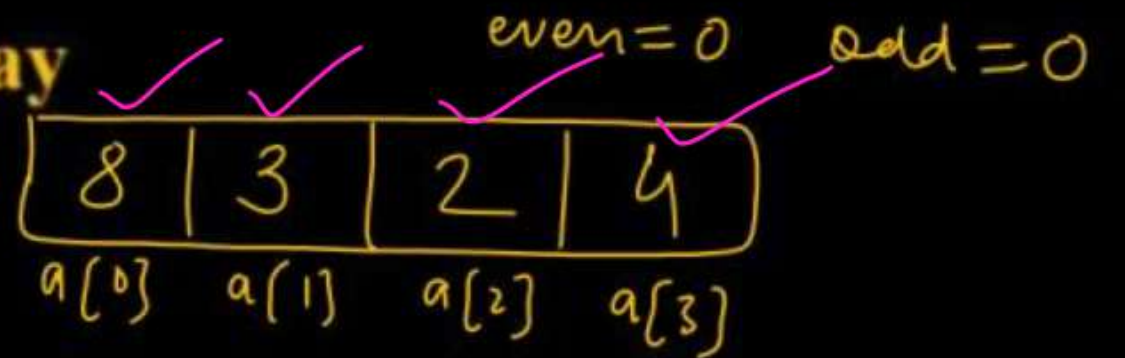
$a[0]$	$a[1]$	$a[2]$			
5	3	4			

$a[99]$

Program to count the even and odd numbers in an array

```
#include <stdio.h>
int main()
{
    int a[100], n, i, even=0, odd=0;
    printf("Enter no. of elements");
    scanf("%d", &n);
    for(i=0; i<=n-1; i++)
    {
        scanf("%d", &a[i]);
        if(a[i]%2==0)
        {
            even++;
        }
        else
        {
            odd++;
        }
    }
    printf("No. of even array elements are = %d", even);
    printf("No. of odd array elements are = %d", odd);
    return 0;
}
```

$n=4$



$i=0$

a[0] is input

$$8 \% 2 == 0 \checkmark$$

$$\text{even} = 0 + 1 \Rightarrow$$

even=1

$i=1$

a[1] is input

$$3 \% 2 == 0 \times$$

$$\text{odd} = 0 + 1 \Rightarrow$$

odd=1

$i=2$

a[2]

$$2 \% 2 == 0 \checkmark$$

$$\text{even} = 1 + 1 \Rightarrow$$

even=2

$i=3$

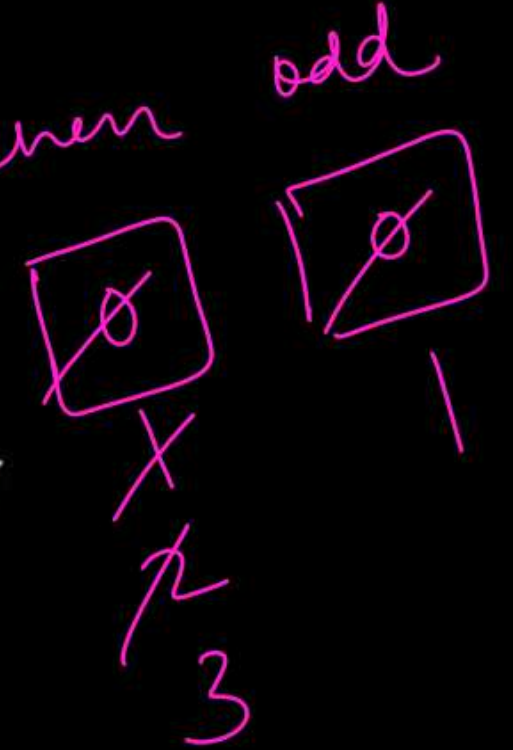
a[3]

$$4 \% 2 == 0 \checkmark$$

$$\text{even} = 2 + 1 \Rightarrow 3$$

even=3

$i=4$



DPP

- 1) WAP to print the sum of all even elements of array.
- 2) WAP to print the sum of all odd elements of array.

```
main()
{
    int a[100], n, Senen=0, Sodd=0, i;
    n ←
    for(i=0; i<=n-1; i++)
    {
        scanf("%d", &a[i]);
    }
    for(i=0; i<=n-1; i++)
    {
        if(a[i]%2==0)
            Senen = Senen + a[i];
        else
            Sodd = Sodd + a[i];
    }
}
```


Program to find the maximum and minimum number in an array(AKTU)

```
#include <stdio.h>
int main()
{
    int a[100], n, i, min, max;
    printf("Enter no. of elements in array");
    scanf("%d", &n);
    for (i=0; i<=n-1; i++)
    {
        scanf("%d", &a[i]);
    }
    min = max = a[0];
    for (i=1; i<=n-1; i++)
    {
        if (a[i] < min)
        {
            min = a[i];
        }
        if (a[i] > max)
        {
            max = a[i];
        }
    }
    printf("The minimum and Maximum values are %d, min, max);
    return 0;
}
```

element $n=4$

$\min=3$, $\max=3$

$i=1$ $a[1] < 3$
 $4 < 3 \times$
 $a[1] > 3$
 $4 > 3 \checkmark$ $\max=4$ $\min=3$

$i=2$ $a[2] < \min \Rightarrow 8 < 3 \times$
 $a[2] > \max \Rightarrow 8 > 4 \checkmark$
 $\max=8$ $\min=3$

$i=3$ $a[3] < \min \Rightarrow 1 < 3 \checkmark$ $\min=1$
 $a[3] > \max \Rightarrow 1 > 8 \times$ $\max=8$

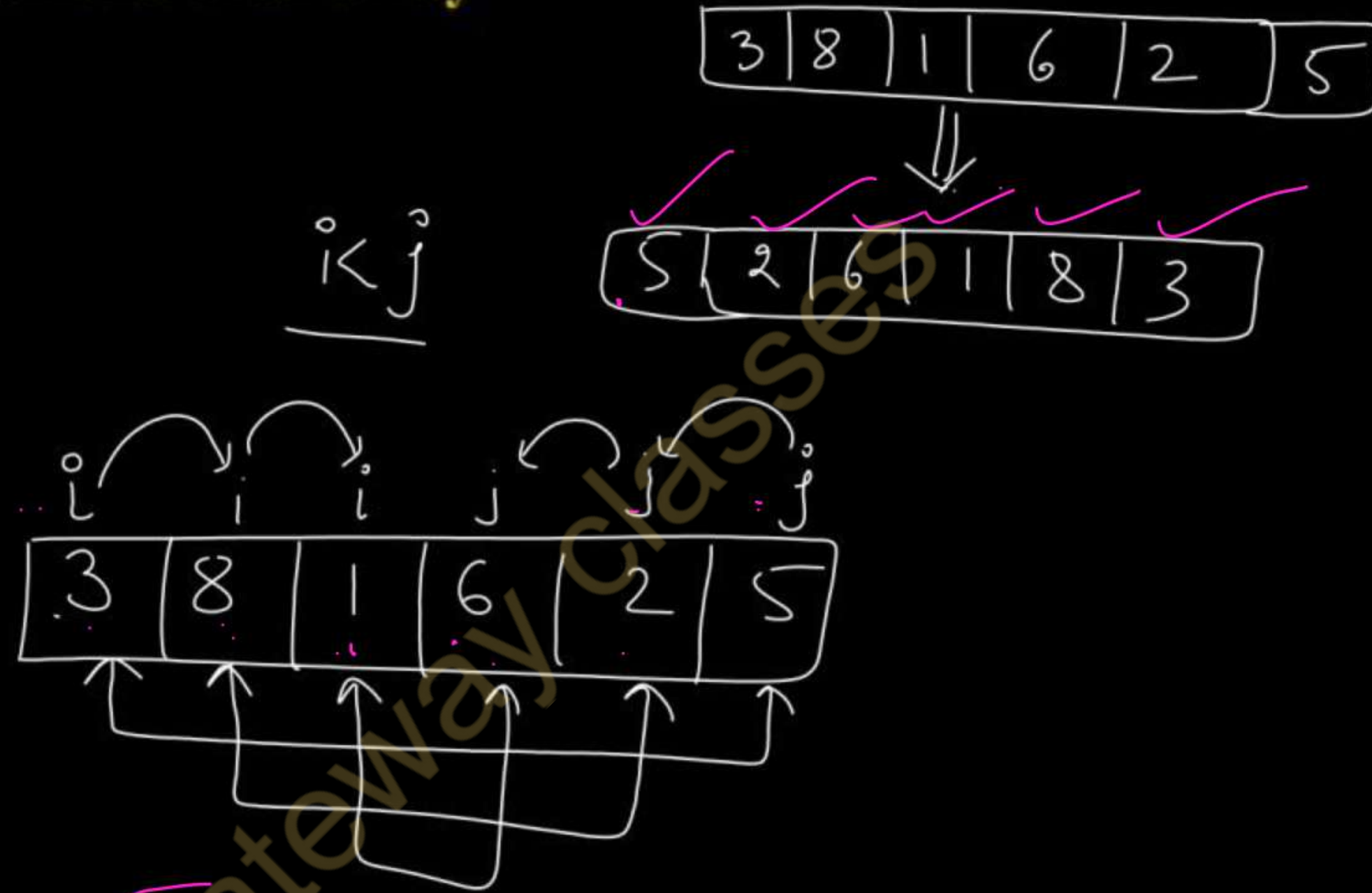
$i=4$ $\times$

$a[0]$ $a[1]$ $a[2]$ $a[3]$

3	4	8	1
---	---	---	---

$\downarrow$ max $\downarrow$ min

Program to reverse the elements of an array



Program to reverse the elements of an array

```
#include <stdio.h>
int main()
{
    int a[] = { 3, 8, 1, 6, 2, 5 };
    int i, j, temp;

    for (i = 0, j = 5; i < j; i++, j--)
    {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }

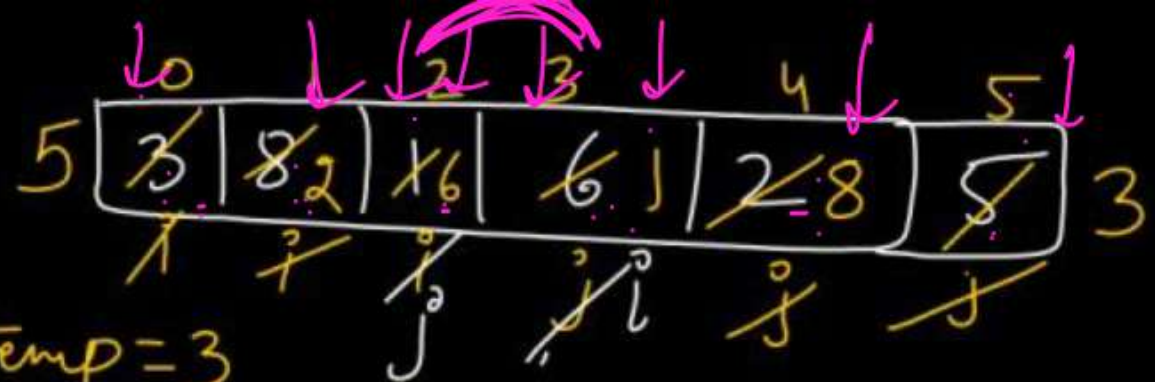
    printf("The reversed array is = ");
    for (i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }
    return 0;
}
```

$$i=0, j=5$$

$$i=1, j=4$$

$$i=2, j=3$$

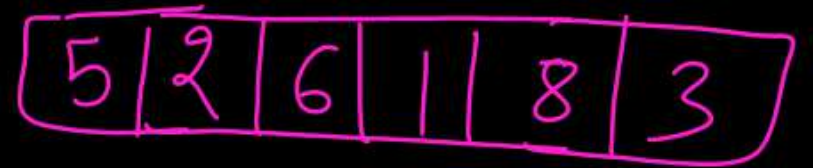
$$i=3, j=2$$



temp = 3

$$a[0] = a[5] \Rightarrow a[0] = 5$$

$$a[5] = 3$$

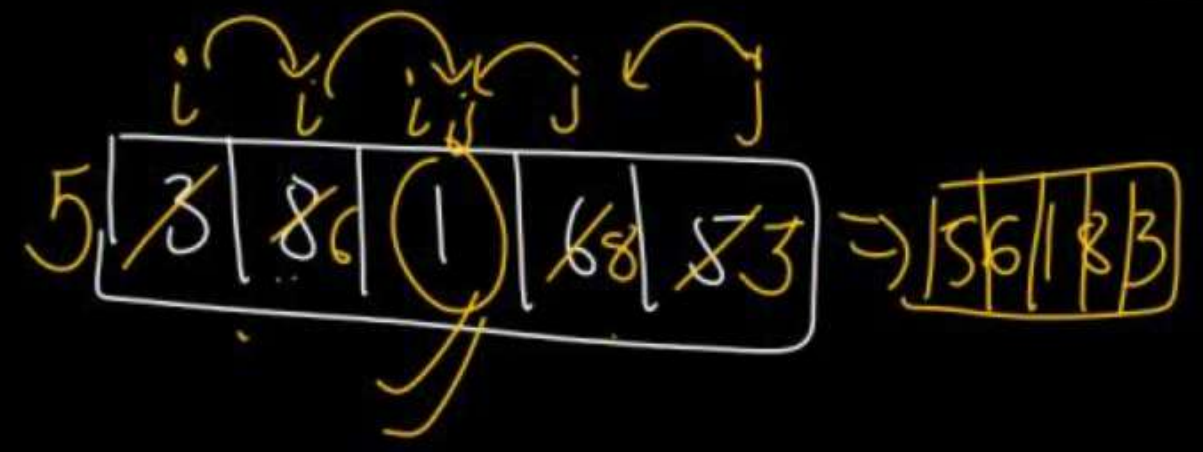


Reversed Array



$$a[2] = 6$$

$$a[3] = 1$$





AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture- 8

Topics to be Covered

- Two-dimensional Array
 - Program to print addition of two matrices.
 - Program to print transpose of a matrix
 - Program to find sum of diagonal elements of a matrix
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

2- dimensional Array

Declaration

`int a[2][3];`

no. of rows → 2
no. of columns → 3
 $2 \times 3 = 6$ elements
Type of array → int
Name of array → a

Note → a is a 2-Dimensional array that can store max 6 integers.

Initialization —

`int a[2][3] = { 2, 3, 8, 1, 5, 6 };`

OR

`int a[2][3] = {`
row 1
`{ 2, 3, 8 },`
`{ 1, 5, 6 },`
Row 2
`};`

2-dimensional Array

Memory Representation

2	3	8	1	5	6
---	---	---	---	---	---

1000 1002 1004 1006 - - -

Row-wise filling
of elements

Row-Major

Column Major

Col-wise filling
of elements

2	1	3	5	8	6
---	---	---	---	---	---

	✓ cols ✓		
rows ↑	2	3	8
	1	5	6

} 1-D array
} 1-D array

2- dimensional Array

Diagram illustrating memory layout and indices for an array a :

1	8	3	4	5	7	3	2	6
---	---	---	---	---	---	---	---	---

Indices (row labels):

- $a[0][0]$ (above 1)
- $a[0][1]$ (above 8)
- $a[0][2]$ (above 3)
- $a[1][0]$ (above 4)
- $a[1][1]$ (above 5)
- $a[1][2]$ (above 7)

Memory addresses (below the array):

- 1000 (below 1)
- 1002 (below 8)
- 1004 (below 3)
- 1006 (below 4)
- 1008 (below 5)
- 1010 (below 7)
- 1012 (below 3)
- 1014 (below 2)
- 1016 (below 6)

How to input a 2-D Array

2- dimensional Array

```
int a[3][2];
```

```
for(i=0; i<=2; i++)  
{  
    for(j=0; j<=1; j++)  
    {  
        scanf("%d", &a[i][j]);  
    }  
}
```

$i=0 \rightarrow j=0, j=1$

$i=1 \rightarrow j=0, j=1$

$i=2 \rightarrow j=0, j=1$

	0	1
0	2	8
1	7	5
2	4	1

How to Print 2-D array

```
for(i=0; i<=2; i++)  
{  
    for(j=0; j<=1; j++)  
    {  
        printf("d\t", a[i][j]);  
    }  
    printf("\n");  
}
```

2	8
7	5
4	1

Program to print addition of two matrices. ($m \times n$)

```
#include <stdio.h>
int main()
```

```
{ int a[10][10], b[10][10], c[10][10], i, j, m, n;
```

```
printf("Enter dimensions of matrices");
```

```
scanf("%d %d", &m, &n); // dimensions of matrix
```

```
printf("Enter first matrix");
```

```
for(i=0; i<m; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ scanf("%d", &a[i][j]);
```

```
}
```

```
printf("Enter second Matrix");
```

```
for(i=0; i<m; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ scanf("%d", &b[i][j]);
```

```
}
```

```
for(i=0; i<m; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ c[i][j] = a[i][j] + b[i][j];
```

```
}
```

```
printf("The resultant addition matrix is:");
```

```
for(i=0; i<m; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ printf("%d", c[i][j]);
```

```
} } printf("\n");
```

```
return 0;
```

Input
2nd matrix

Adding
two matrices

Input
Matrix
a

$$\begin{array}{c}
 \text{A} \\
 \begin{array}{cc}
 0 & 1 \\
 0 & \begin{bmatrix} \underline{1} & \underline{2} \\ \underline{3} & \underline{4} \end{bmatrix} \\
 1 &
 \end{array}
 \end{array}
 \quad
 \begin{array}{c}
 \text{B} \\
 \begin{array}{cc}
 0 & 1 \\
 0 & \begin{bmatrix} \underline{5} & \underline{6} \\ \underline{7} & \underline{8} \end{bmatrix} \\
 1 &
 \end{array}
 \end{array}$$

$$\begin{array}{cc}
 & 0 & 1 \\
 0 & \begin{array}{c} \underline{1+5} \\ - \end{array} & \begin{array}{c} \underline{2+6} \\ - \end{array} \\
 1 & &
 \end{array}$$

```

for (i = 0; i < m; i++)
{
    for (j = 0; j < n; j++)
    {
        c[i][j] = a[i][j] + b[i][j];
    }
}

```

Program to print addition of two matrices. ($m \times n$)

```
#include <stdio.h>
int main()
```

```
{ int a[10][10], b[10][10], c[10][10], i, j, m, n;
```

```
printf("Enter dimensions of matrices");
```

```
scanf("%d %d", &m, &n); // dimensions of matrix
```

```
printf("Enter first matrix");
```

```
for(i=0; i<m; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ scanf("%d", &a[i][j]);
```

```
}
```

```
printf("Enter second Matrix");
```

```
for(i=0; i<m; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ scanf("%d", &b[i][j]);
```

```
}
```

```
for(i=0; i<m; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ c[i][j] = a[i][j] + b[i][j];
```

```
}
```

```
printf("The resultant addition matrix is:");
```

```
for(i=0; i<m; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ printf("%d", c[i][j]);
```

```
} } printf("\n");
```

```
return 0;
```

Input
2nd matrix

Adding
two matrices

Input 1st
Matrix
a

Program to print transpose of a matrix (Imp)

```
#include <stdio.h>
int main()
{
    int a[10][10], b[10][10], m, n, i, j;
    printf("Enter matrix dimensions");
    scanf("%d %d", &m, &n);
    printf("Enter the matrix");
    for(i=0; i<m; i++)
    {
        for(j=0; j<n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
```

Input
Matrix

```
for(i=0; i<m; i++)
{
    for(j=0; j<n; j++)
    {
        b[j][i] = a[i][j];
    }
}
printf("The resultant transpose matrix is =");
for(i=0; i<n; i++)
{
    for(j=0; j<m; j++)
    {
        printf("%d", b[i][j]);
    }
    printf("\n");
}
return 0;
```

Transpose of
Matrix a

(3×2) $\circ$ $\underline{m \times n}$

$$A = \begin{matrix} \circ & & \circ \\ 0 & 1 & 2 \\ 1 & 3 & 4 \\ 2 & 5 & 6 \end{matrix}$$

$\leftarrow \begin{matrix} 0 & 1 & 2 \end{matrix} \rightarrow$ $\underline{n \times m}$ ✓

$$B = \begin{matrix} \circ & & \circ \\ 0 & 1 & 3 & 5 \\ 1 & 2 & 4 & 6 \end{matrix}$$

```

for (i = 0; i < 3n; i++)
{
    for (j = 0; j < 2n; j++)
    {
        b[j][i] = a[i][j];
    }
}

```

$i = 0$ $j = 0$ | $b[0][0] \leftarrow a[0][0]$
 $j = 1$ | $b[1][0] \leftarrow a[0][1]$

$i = 1$ $j = 0$ | $b[0][1] \leftarrow a[1][0]$
 $j = 1$ | $b[1][1] \leftarrow a[1][1]$

$i = 2$ $j = 0$ | $b[0][2] \leftarrow a[2][0]$
 $j = 1$ |

Program to find sum of diagonal elements of a matrix(AKTU)

```
#include <stdio.h>
int main()
{
    int a[10][10], m, n, i, j, s1=0, s2=0;
    printf("Enter matrix dimensions");
    scanf("%d%d", &m, &n);
    if(m == n)
    {
        printf("Enter the matrix");
        for(i=0; i<m; i++)
        {
            for(j=0; j<n; j++)
            {
                scanf("%d", &a[i][j]);
            }
        }
    }
}
```

Diagram illustrating a 3x3 matrix with elements 1 through 9. The main diagonal elements (1, 5, 9) are circled in green. The off-diagonal elements (2, 3, 4, 6, 7, 8) are circled in pink.

```
for(i=0; i<n; i++)
{
    s1 = s1 + a[i][i];
    s2 = s2 + a[i][m-1-i];
}
printf("The sum of main diagonal elements = %d", s1);
printf("The sum of off diagonal elements = %d", s2);
else { printf("Can not generate sum of diagonals");
}
return 0;
```

code for generating sum of diagonals



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture- 9

Topics to be Covered

- Program to multiply two matrices
- 3-Dimensional Arrays
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

AKTU PYQs

1. Write a program to multiply two matrices (read size and elements of matrices from the keyboard). (AKTU 2022-23)
2. Write a Program to print the multiplication of two 2-dimensional arrays and print the result. (AKTU 2021-22)
3. Write a Program to print the multiplication of two-dimensional matrices with $m*n$ dimensions. (AKTU 2023-24)
4. Write a Program to print the multiplication of two $n*n$ matrices and print the resultant matrix along with the transpose of resultant matrix. (AKTU 2023-24) ✓

Program to multiply two matrices with dimensions entered by the user.

```
#include <stdio.h>
int main()
{
    int a[10][10], b[10][10], c[10][10], r1, c1, r2, c2, i, j, k;
    printf("Enter dimensions of First Matrix");
    scanf("%d %d", &r1, &c1);
    printf("Enter dimensions of Second Matrix");
    scanf("%d %d", &r2, &c2);
    printf("Enter the first Matrix");
    for(i=0; i<r1; i++)
    {
        for(j=0; j<c1; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
```

Input
1st matrix

```
printf("Enter the second Matrix");
for(i=0; i<r2; i++)
{
    for(j=0; j<c2; j++)
    {
        scanf("%d", &b[i][j]);
    }
}
```

✓
 r_1 c_1 r_2 c_2 ✓
Input $r_1 \times c_2$
2nd $\downarrow$
matrix $\downarrow$
rows cols

```
for(i=0; i<r1; i++) ✓
{
    for(j=0; j<c2; j++) ✓
    {
        c[i][j] = 0;
        for(k=0; k<c1; k++)
        {
            c[i][j] = c[i][j] + a[i][k] * b[k][j];
        }
    }
}
```

code for
Matrix Multiplication


```

printf("The resultant matrix is = ");
for(i=0; i<n1; i++)
{
    for(j=0; j<n2; j++)
    {
        printf("%d", c[i][j]);
    }
    printf("\n");
}
return 0;
}

```

$$\begin{array}{l}
 A = \begin{matrix} & \begin{matrix} 0 & 1 & 2 \end{matrix} \\ \begin{matrix} 0 \\ 1 \end{matrix} & \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \end{matrix} \quad (2 \times 3) \quad B = \begin{matrix} & \begin{matrix} 0 & 1 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \end{matrix} & \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \end{matrix} \quad (3 \times 2)$$

$$C = \begin{matrix} & \begin{matrix} 0 & 1 \end{matrix} \\ \begin{matrix} 0 \\ 1 \end{matrix} & \begin{bmatrix} ? & ? \\ ? & ? \end{bmatrix} \end{matrix} \quad (2 \times 2)$$

$$\begin{array}{l}
 c[0][0] = ? \\
 c[0][1] = ? \\
 c[1][0] = ? \\
 c[1][1] = ?
 \end{array}$$

```

for( $i=0$ ;  $i < \cancel{2}$ ;  $i++$ ) // rows for C
{
  for( $j=0$ ;  $j < \cancel{2}$ ;  $j++$ ) // cols for C
  {

```

$C[i][j] = 0$;

```

for( $k=0$ ;  $k < \cancel{3}$ ;  $k++$ )
{

```

$C[i][j] = C[i][j] + a[i][k] * b[k][j];$

$i=0, j=0 \quad C[0][0] = 0$

$k=0$
 $C[0][0] = C[0][0] + a[0][0] * b[0][0]$

$= 0 + 1 * 1 = 1$

$k=1$
 $C[0][0] = 1 + a[0][1] * b[1][0]$
 $= 1 + 2 * 3$
 $= 1 * 1 + 2 * 3$

$$A = \begin{matrix} & \begin{matrix} 0 & 1 & 2 \end{matrix} \\ \begin{matrix} 0 \\ 1 \end{matrix} & \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \end{matrix} \quad B = \begin{matrix} & \begin{matrix} 0 & 1 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \end{matrix} & \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \end{matrix}$$

$$C = \begin{matrix} & \begin{matrix} 0 & 1 \end{matrix} \\ \begin{matrix} 0 \\ 1 \end{matrix} & \begin{bmatrix} 1 \times 1 + 2 \times 3 + 3 \times 5 & 1 \times 2 + 2 \times 4 + 3 \times 6 \\ \text{---} & \text{---} \end{bmatrix} \end{matrix} \quad \begin{matrix} C[0][0] = ? \\ C[0][1] = ? \\ C[1][0] = ? \\ C[1][1] = ? \end{matrix}$$

$k=2 \quad C[0][0] = 1 \times 1 + 2 \times 3 + a[0][2] \times b[2][0]$
 $\neq 1 \times 1 + 2 \times 3 + 3 \times 5$

$i=0, j=1 \quad C[0][1] = 0$

$k=0 \quad C[0][1] = 0 + a[0][0] \times b[0][1] \Rightarrow 1 \times 2$

$k=1 \quad C[0][1] = 1 \times 2 + a[0][1] \times b[1][1] \Rightarrow 1 \times 2 + 2 \times 4$

$k=2 \quad C[0][1] = 1 \times 2 + 2 \times 4 + a[0][2] \times b[2][1]$

$C[0][1] = 1 \times 2 + 2 \times 4 + 3 \times 6$

Program to print the multiplication of two $n \times n$ matrices and print the resultant matrix along with the transpose of resultant matrix. (AKTU 2023-24)

```
#include <stdio.h>
```

```
int main()
```

```
{ int a[10][10], b[10][10], c[10][10], d[10][10]; n, i, j, k;
```

```
printf("Enter the value of n");
```

```
scanf("%d", &n);
```

```
printf("Enter first Matrix");
```

```
for (i=0; i<n; i++)
```

```
{ for (j=0; j<n; j++)
```

```
{ scanf("%d", &a[i][j]);
```

```
}
```

```
} printf("Enter the second Matrix");
```

```
for (i=0; i<n; i++)
```

```
{ for (j=0; j<n; j++)
```

```
{ scanf("%d", &b[i][j]);
```

```
}
```

```
}
```

```
for (i=0; i<n; i++)
```

```
{ for (j=0; j<n; j++)
```

```
{ c[i][j] = 0;
```

```
for (k=0; k<n; k++)
```

```
{ c[i][j] = c[i][j] + a[i][k] * b[k][j];
```

```
}
```

```
}
```

Input
2nd Matrix

Multiply 1st and
2nd Matrix


```
printf("The resultant multiplication matrix=");
```

```
for(i=0; i<n; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ printf("%d", c[i][j]);
```

```
} printf("\n");
```

```
} }   
↑   
creating transpose
```

```
for(i=0; i<n; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ d[j][i] = c[i][j];
```

```
} }
```

display
result of
mult.

```
printf("The transpose of resultant multiplication matrix=");
```

```
for(i=0; i<n; i++)
```

```
{ for(j=0; j<n; j++)
```

```
{ printf("%d", d[i][j]);
```

```
} printf("\n");
```

```
{ return 0;
```

Display

Transpose of
Multiplication
matrix.

Declaration

no. of matrices

Type of array

name of array

rows in one matrix

col in one matrix

```
int a[2][2][3];
```

0 th 2D array						1 th 2D Array					
1	2	3	4	5	6	7	8	9	10	11	12

3-Dimensional Arrays

$2 \times 2 \times 3 = 12$ elements

```
int a[2][2][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

or

```
int a[2][2][3] = {
```

mat-1

```
{ { 1, 2, 3 },
```

```
{ 4, 5, 6 },
```

```
},
```

mat-2

```
{ { 7, 8, 9 },
```

```
{ 10, 11, 12 },
```

```
};
```

How to input 3-D array

```
for (i=0; i<2; i++) // no. of matrices
{
    for (j=0; j<2; j++) // for both matrix
    {
        for (k=0; k<3; k++) // rows
        {
            scanf("%d", &a[i][j][k]);
        }
    }
}
```

How to print?

```
for (i=0; i<2; i++)
{
    for (j=0; j<2; j++)
    {
        for (k=0; k<3; k++)
        {
            printf("%d", a[i][j][k]);
        }
        printf("\n");
    }
    printf("\n");
}
```




AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture- 10

Topics to be Covered

- Introduction to Strings
- String Handling Functions ✓
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Strings

- A String is a sequence of characters terminated with a null character '\0'.
- The C String is stored as an array of characters.
- The difference between a character array and a string is that the string is terminated with a unique character '\0'.

→ its ASCII value is 0.

char a[10];

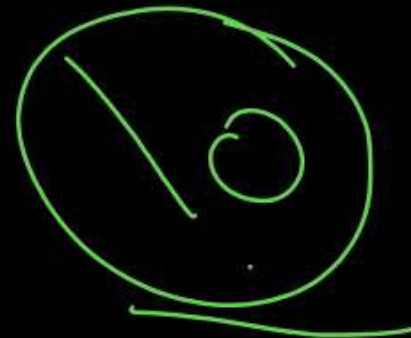


null character

'\0'

char a[10] = "Hello";

↓
string



Strings

Initialization of string

char str[10] = "Hello";

OR

char str[10] = {'H', 'e', 'l', 'l', 'o', '\0'};

'\0' marks
ending of the
str



str[0] str[1] str[2] - - -

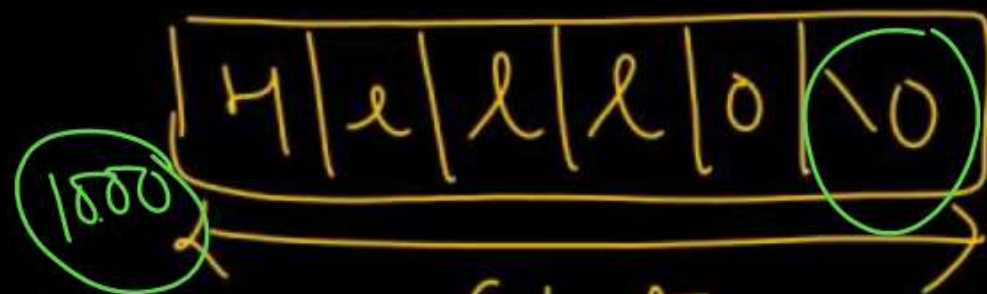
length of the string

→ No. of characters excluding '\0'

length of str = 5

char str[] = "Hello";

str



str

6 bytes

length = 5

size = 6 bytes

Strings

How to input a string from user

%s → format specifier

char str[50];

scanf("%s", str);

It accepts a single word string only. If there is a white space in the string then it will input the string before space only.

Another Method

gets(str); // unformatted Input funⁿ

→ scanf("%[^\n]s", str);

Now it will accept a multiword string until a new line occurs

Strings

How to print a string

```
char str[] = "Gateway classes";
```

1) `printf("%s", str);`

↓
It will print the whole string.

2) `puts(str);`

//

③

i → 0 1 2 3 4 - - - - -
|G|a|t|e|w|a|y| |c|l|a|s|s|e|s|\0

```
for (i = 0; str[i] != '\0'; i++)  
{  
    printf("%c", str[i]);  
}
```

↓
Important to Remember

String Handling Functions

(In-built function present in C-library)

(Header file string.h needs to be included)

1) strlen()

2) strcpy()

3) strcat()

4) strcmp()

5) strstr()

1) char str[] = "Hello";

strlen(str) $\Rightarrow$ it will give 5

$\hookrightarrow$ provides length of the string which is passed inside.

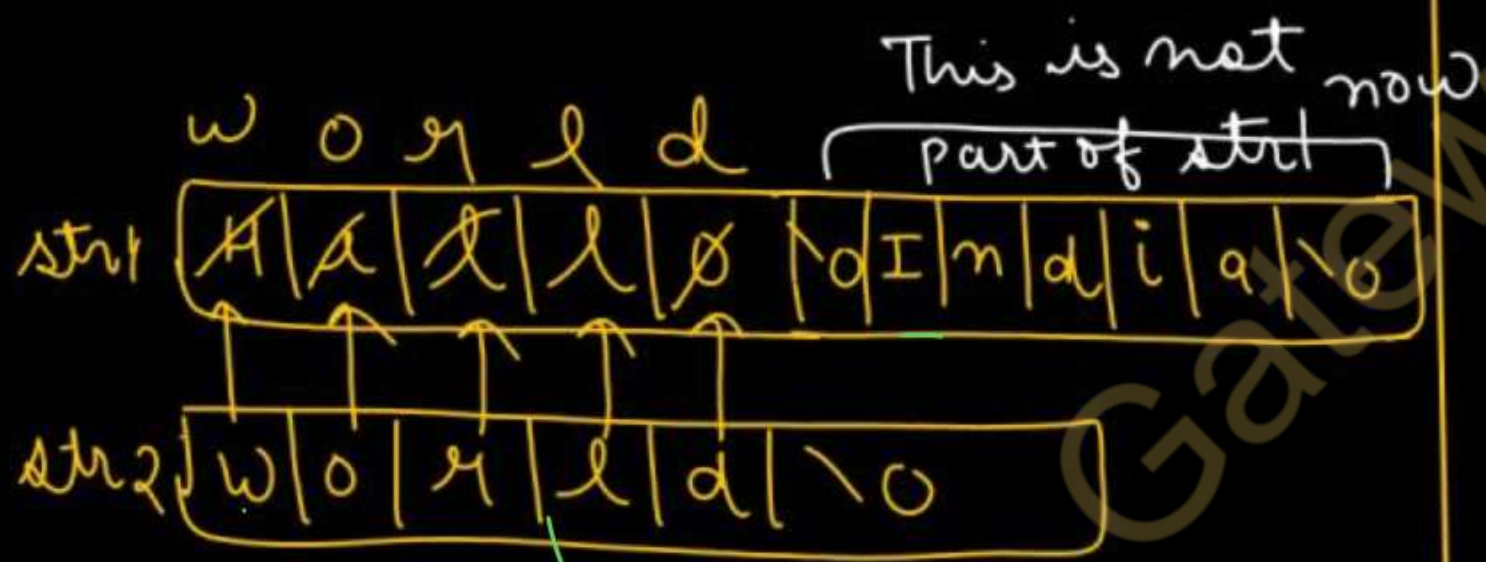
String Handling Functions

② strcpy(str1, str2)

It copies str2 into str1

char str1[] = "Hello India";

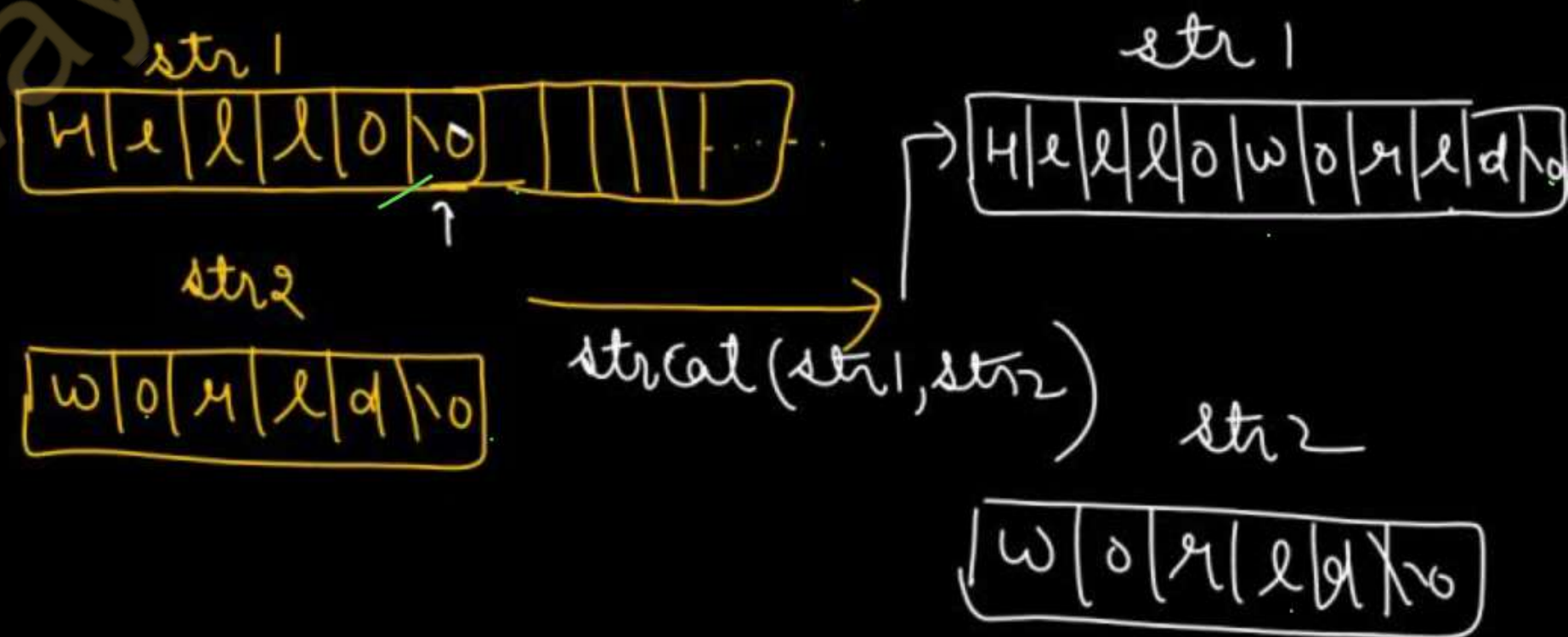
char str2[] = "World";



③ strcat(str1, str2)

It appends str2 after str1.
(concatenates)

Ex. char str1[50] = "Hello";
char str2[] = "world";
strcat(str1, str2);



String Handling Functions

④ strrev(str)

↓
It reverse the string str and stores it back into str only.

Ex. char str[] = "Hello";

strrev(str);

printf("%s", str);

output — olleH

⑤ strcmp(str1, str2)

It compares str1 and str2 if they are identical or not.

Example

char str1[] = "Hello";

char str2[] = "Hellp";

strcmp(str1, str2);

strcmp(str1, str2) returns 0 if strings are identical otherwise any positive/negative value.

str1

H	e	l	l	o	\0
---	---	---	---	---	----

H	e	l	l	p	\0
---	---	---	---	---	----

→ Here strcmp will return the difference b/w. 0 and p ASCII values.

$d = \text{ASCII of } (o) - \text{ASCII value of } p$

$= -1$

String Handling Functions

Program to check whether a given string is palindrome or not using library functions

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str1[50], str2[50], d;
```

```
    printf("Enter string");
```

```
    gets(str1);
```

```
    strcpy(str2, str1);
```

```
    strrev(str1);
```

```
    if (strcmp(str1, str2) == 0)
    {
        printf("String is palindrome");
    }
    else
    {
        printf("String is not palindrome");
    }
    return 0;
```

```
}
```

AKTU PYQs

1. Discuss the following string functions in C with suitable code snippet:
- i) strrev ii) strcmp iii) strcat iv) strlen v) strcpy (AKTU 2022-23)
2. Write a Program to count number of vowels in a string. (AKTU 2021-22)

Gateway classes



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture- 11

Topics to be Covered

- Code Implementation of String Functions
- Some additional string programs
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Program to find length of a string without using library function (Implementation of strlen())

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[50]; int i;
```

```
    printf("Enter the string");
```

```
    gets(str);
```

```
    i=0;
```

```
    while(str[i] != '\0')
```

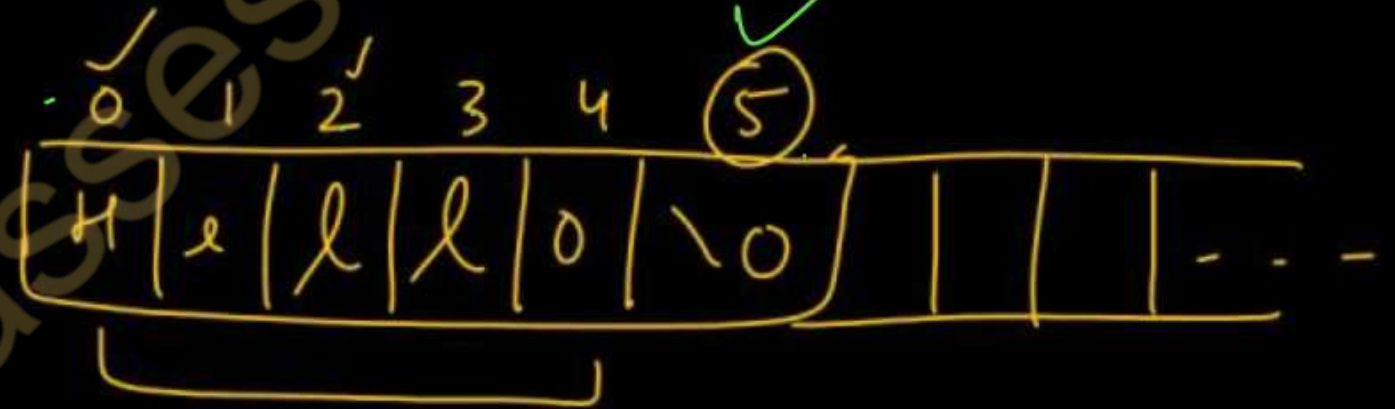
```
    {  
        i++;
```

```
    }
```

```
    printf("length of the string is = %d", i);
```

```
    return 0;
```

```
}
```



Program to copy one string into another string without using library function

```
#include <stdio.h>
int main()
```

(Implement strcpy())

```
{ char str1[50], str2[50]; int i;
```

```
printf("Enter a string");
```

```
gets(str1);
```

```
for(i=0; str1[i] != '\0'; i++)
```

```
{
```

```
    str2[i] = str1[i];
```

```
}
```

```
    str2[i] = '\0';
```

```
printf("The copied string is = %s", str2);
```

```
return 0;
```

```
}
```

str1

0	1	2	3	4	5	6
H	e	l	l	o	\0	

...

str2

0	1	2	3	4	5	6	7	8
H	e	l	l	o	\0			

...

Program to concatenate two strings without using library function

```
#include <stdio.h>
int main()
```

```
{
    char str1[100], str2[50]; int i, j;
```

```
    printf("Enter the strings");
```

```
    gets(str1);
    gets(str2);
```

```
    i=0;
    while (str1[i] != '\0')
```

code for reaching upto the end of str1

```
    { i++; }
```

```
    j=0;
```

```
    while (str2[j] != '\0')
```

```
    {
        str1[i] = str2[j];
```

copying str2 into str1

```
        i++;
```

```
        j++;
```

```
    }
    str1[i] = '\0'; // storing '\0' at the end of str1
```

```
    printf("The concatenated string is = '%s'", str1);
```

```
    return 0;
```

strcat()

str1

0	1	2	3	4	5	6	7	8	9	10	11	12	13	...
H	e	l	l	o	w	o	r	l	d					

str2

0	1	2	3	4	5	...
w	o	r	l	d	\0	

str1

0	1	2	3	4	5	6	7	8	9	10	11	12	13	...
H	e	l	l	o	w	o	r	l	d	\0	.	.	.	.

Program to compare two strings without using library function

```
#include <stdio.h>
```

```
int main()
```

```
{ char str1[50], str2[50]; int i;
```

```
printf("Enter two strings");
```

```
gets(str1);
```

```
gets(str2);
```

```
i=0;
```

```
while((str1[i] != '\0') && (str2[i] != '\0')  
      && (str1[i] == str2[i]))
```

```
{ i++;
```

```
}
```

```
if(str1[i] == str2[i])
```

```
{ printf("The strings are identical"); }
```

```
else { printf("Strings are not identical"); }
```

```
return 0;
```

Diagram illustrating the memory layout of string `str1`. The string is stored in an array of 50 characters. The first 6 characters are 'H', 'e', 'l', 'l', 'o', and '\0' (null terminator). The remaining characters are represented by dashes. A green arrow points to the null terminator at index 5.

Index	0	1	2	3	4	5	6	7	8
Character	H	e	l	l	o	\0	-	-	-

Diagram illustrating the memory layout of string `str2`. The string is stored in an array of 50 characters. The first 6 characters are 'H', 'e', 'l', 'l', 'p', and '\0' (null terminator). The remaining characters are represented by dashes. A green arrow points to the null terminator at index 5.

Index	0	1	2	3	4	5	6	7	8
Character	H	e	l	l	p	\0	-	-	-

Program to reverse a strings without using library function (strrev())

```
#include <stdio.h>
```

```
int main()
```

```
{ char str[50], temp;
```

```
  int i, j, l=0;
```

```
  printf("Enter the string");
```

```
  gets(str);
```

```
  for(i=0; str[i]!='\0'; i++)
```

```
  { l++; } // string length
```

```
  for(i=0, j=l-1; i<j; i++, j--)
```

```
  {
```

```
    temp = str[i];
```

```
    str[i] = str[j];
```

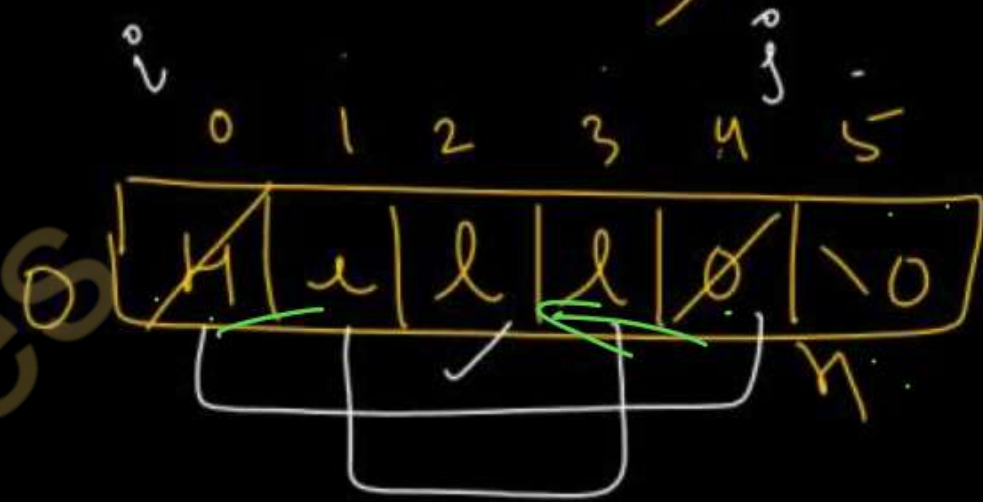
```
    str[j] = temp;
```

```
  }
```

```
  printf("The reversed string is \"%s\"", str);
```

```
  return 0;
```

```
}
```



i=0, j=4

i++, j--

(i < j)

temp = H

str[0] = o

str[4] = H

Program to check if a string is palindrome or not without using library function

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[50];
```

```
    int i, j, l = 0, flag = 1;
```

```
    printf("Enter a string");
```

```
    gets(str);
```

```
    for (i = 0; str[i] != '\0'; i++)
```

```
    { l++; }
```

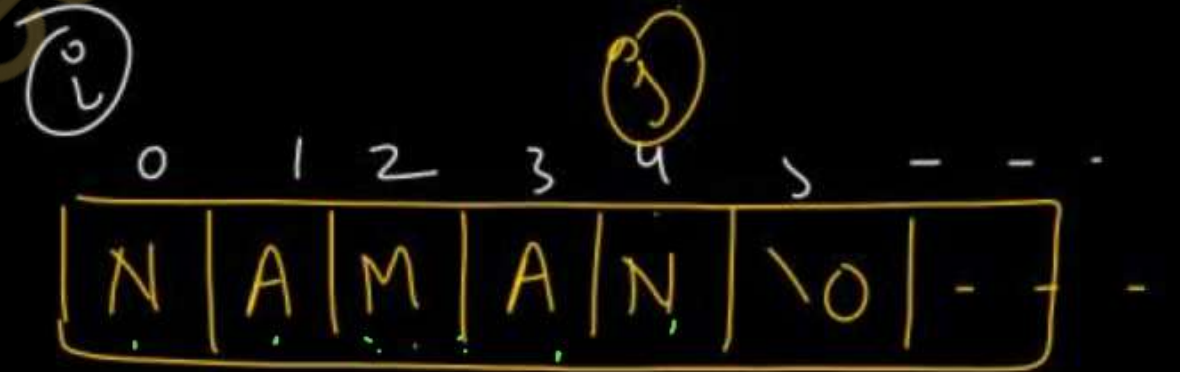
```
    for (i = 0, j = l - 1; i < j; i++, j--)
```

```
    { if (str[i] != str[j])
```

```
        { flag = 0;
```

```
          break;
```

```
    }
```



```
    if (flag == 1)
```

```
        printf("The string is palindrome");
```

```
    else printf("The string is not palindrome");
```

```
    return 0;
```

```
}
```

Program to count no. of vowels in a string.

Ex.

0 1 2 3 - - - - -
G a t e w a y c l a s s e s \0

```
#include <stdio.h>
int main()
```

```
{
    char str[50];
    int i, vowel = 0;
    printf("Enter string");
    gets(str);
```

```
for (i = 0; str[i] != '\0'; i++)
```

```
{
    if (str[i] == 'a' || str[i] == 'e' || str[i] == 'i' || str[i] == 'o' || str[i] == 'u' || str[i] == 'A' ||
        str[i] == 'E' || str[i] == 'I' || str[i] == 'O' || str[i] == 'U')
```

```
    { vowel++; }
}
```

now
return 0;

☒ X Y Z X S
vowel

DPP

- Q- Program to convert a string to uppercase
- Q- Program to count no. of words in a strings

Gateway classes

AKTU PYQs

1. Discuss the following string functions in C with suitable code snippet:

i) strrev ii) strcmp iii) strcat iv) strlen v) strcpy (AKTU 2022-23)

2. Write a Program to count number of vowels in a string. (AKTU 2021-22)

Gateway classes



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays

Lecture- 12

[PART-1 + PART-2]

Topics to be Covered

- Structure in C
- AKTU PYQs



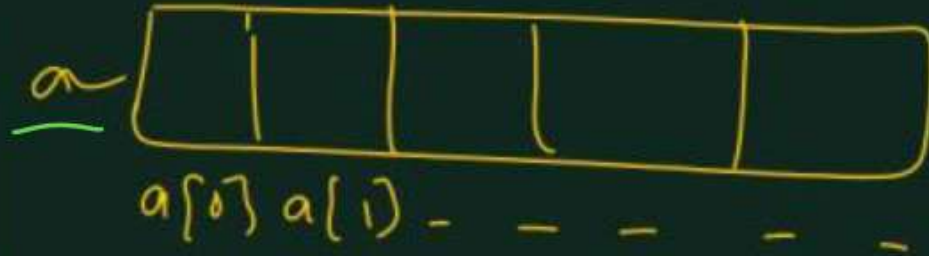
By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

int a;



int a[5];



need some functionality
to store multiple type
of data in one variable



Structure

Why use structure?

- In C, there are cases where we need to store multiple attributes of an entity.
- It is not necessary that an entity has all the information of one type only, It can have different attributes of different data types.
- For example, an entity **Student** may have its name (string), roll number (int), marks (float).
- To store such type of information regarding an entity student, we have the following approaches:
 - Construct individual arrays for storing names, roll numbers, and marks.
 - Use a special data structure to store the collection of different data types.

Structure

Structure

- Structure in C is a user-defined data type that enables us to store the collection of different data types. *elements*.
- In other words, it is capable of storing heterogeneous data. Each element of a structure is called a member. **struct** keyword is used to define the structure.

```
struct structure_name
{
    data_type member1;
    data_type member2;
    .
    .
    data_type memberN;
};
```


How to define a structure

Keyword
↑
struct student → Name of the structure
{
char name[10];
int roll-no;
float marks;
};

Structure members.

Note → Till now no memory is allocated to student. It will be allocated after the creation of a variable of student type.

int a;

How to define a structure

Keyword
↑
struct student → Name of the structure
{
 char name[10];
 int roll-no;
 float marks;
};

Structure Members can not be initialised inside the structure definition.

EX.

struct student

{

 char name[30] = "X42";

 int roll-no = 15;

 float marks = 80;

};

} invalid

How to define a structure

Keyword
↑
struct

```
{  
    char name[10];  
    int roll-no;  
    float marks;  
} s1;  
main()  
{  
    _____  
    _____  
    _____  
    _____  
}
```

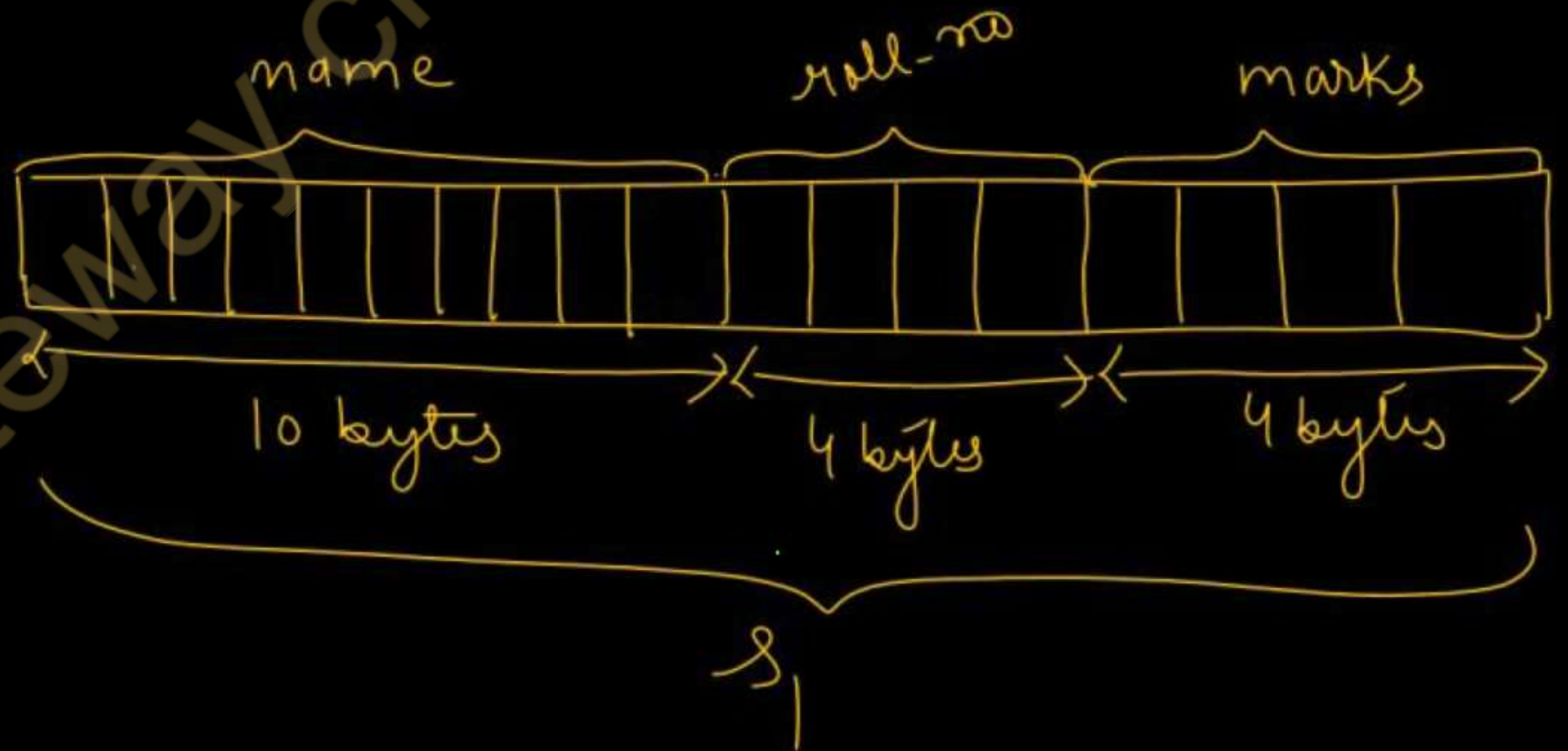
```
struct student ✓  
{  
    char name[10];  
    int roll-no;  
    float marks;  
}  
main()  
{
```

struct student s1, s2;

How to define a structure

Keyword
↑
struct student → Name of the structure
{
 char name[10];
 int roll-no;
 float marks;
} s1;

sizeof(s1)
= 18 bytes



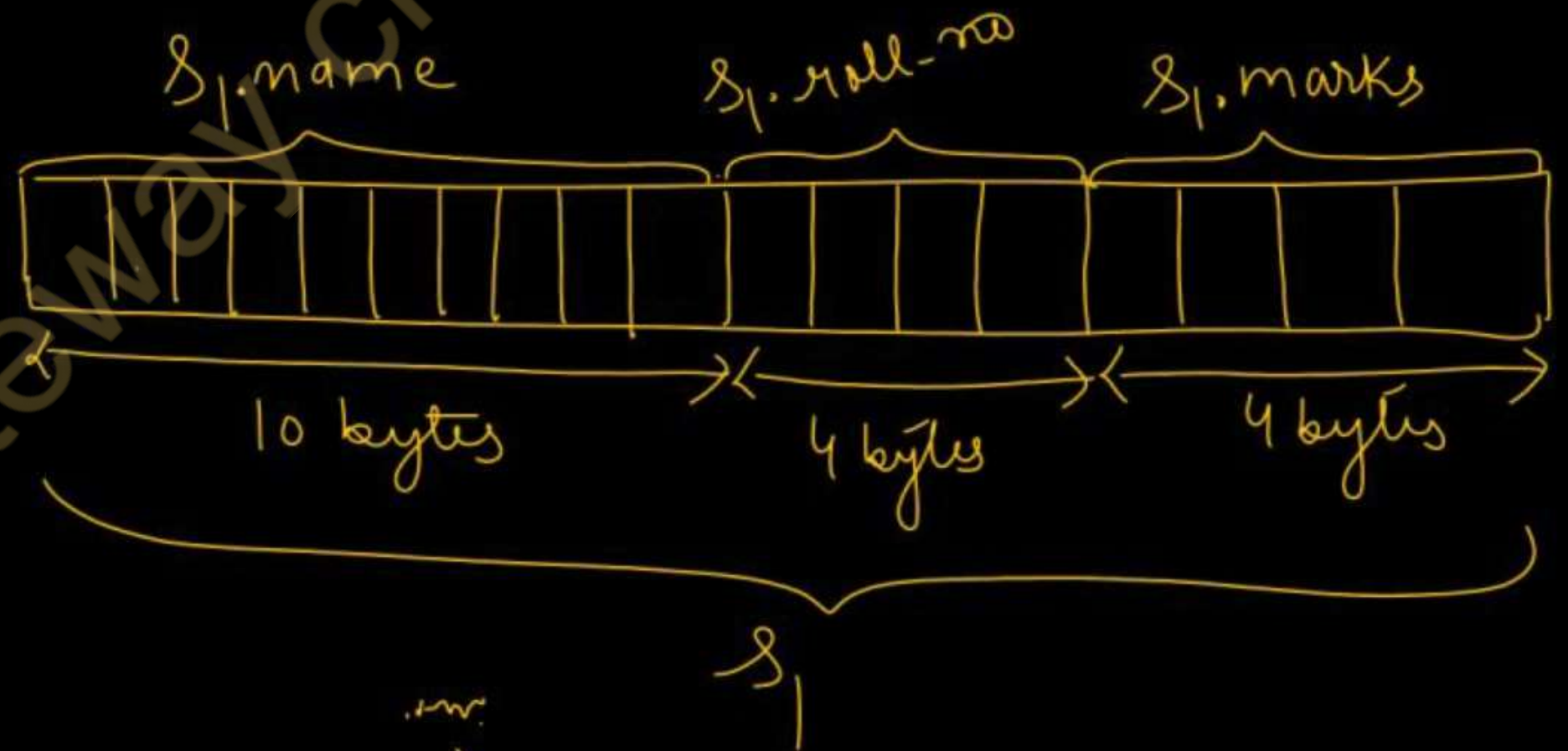
How to define a structure

Keyword
↑
struct student
{
 char name[10];
 int roll-no;
 float marks;
} s₁;

sizeof(s₁)
= 18 bytes

How to access structure elements

- 1) Dot operator (•) ✓
- 2) Indirection operator (→) ✓



How to define a structure

Keyword
↑
struct student
{
 char name[10];
 int roll-no;
 float marks;
} s₁ = {"X42", 10, 85.5};

How to initialize a structure

main ()

{
 struct student s₂ = {"ABC", 15, 90};
}

s₁ name s₁ roll-no s₁ marks

s ₁	X42	10	85.0
----------------	-----	----	------

s₂ name s₂ roll-no s₂ marks

s ₂	ABC	15	90.0
----------------	-----	----	------

Program to show definition, initialization, input and displaying structure.

```
#include <stdio.h>
struct student
{
    char name[30];
    int roll;
    float marks;
};
```

Global structure definition

```
int main ()
```

structure initial

```
{
    struct student s1 = {"A", 10, 80};
```

```
    struct student s2, s3;
```

```
    s2 = s1; // structure copy
```

```
    printf("Enter student details");
```

```
    printf("Enter name");
```

```
    gets(s3.name);
```

```
    printf("Enter roll no");
```

```
    scanf("%d", &s3.roll);
```

```
    printf("Enter Marks");
```

```
    scanf("%f", &s3.marks);
```

```
    printf("The details of first student:");
```

```
    printf("Name=%s \t roll number=%d \t
```

```
marks=%f", s1.name, s1.roll, s1.marks);
```

```
    printf("The details of Second Student:");
```

```
    printf("Name=%s \t roll number=%d \t marks=%f", s2.name,
```

```
s2.roll, s2.marks);
```

```
    printf("The details of Third student:");
```

```
    printf("Name=%s \t roll number=%d \t marks=%f", s3.name,
```

```
s3.roll, s3.marks);
```

```
    return 0;
```

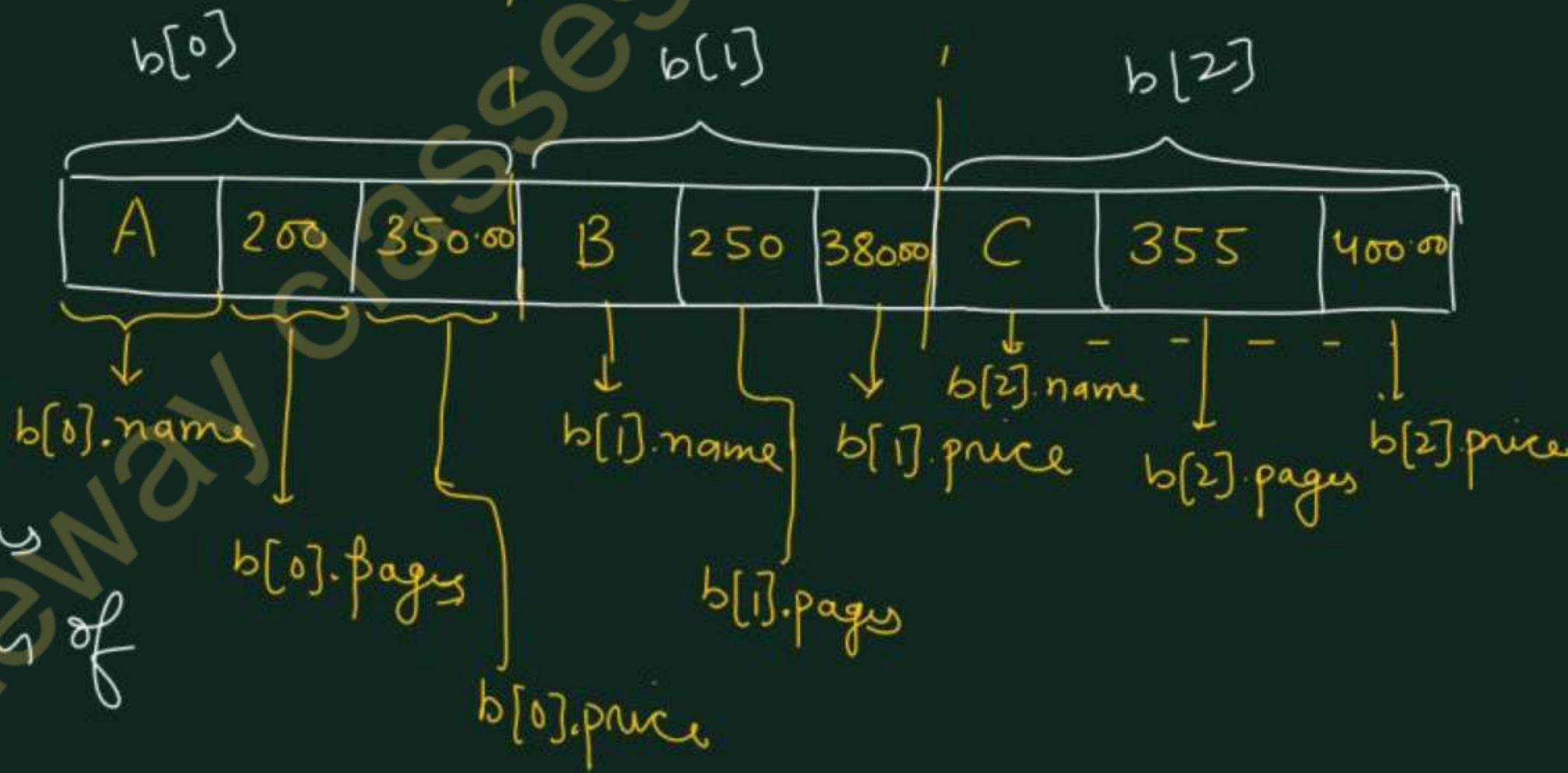
```
}
```


P

Array of structure (Imp)

```
struct book  
{  
    char name[30];  
    int pages;  
    float price;  
}  
b[3];
```

Array of structure means
it can hold the information of
3 books.



Program to create a structure of 100 employees

```
#include <stdio.h>
struct employee
{
    char name[30];
    int id;
    float salary;
};
int main()
{
    struct employee e[100];
    int i;
    printf("Enter details of all employees");
    for(i=0; i<=99; i++)
    {
        gets(e[i].name);
        scanf("%d", &e[i].id);
        scanf("%f", &e[i].salary);
    }
}
```

array of 100
employees

```
printf("Details of Employees are:");
for(i=0; i<=99; i++)
{
    printf("The details of employee %d ", i+1);
    printf("Name = %s", e[i].name);
    printf("Id = %d", e[i].id);
    printf("Salary = %f", e[i].salary);

    return 0;
}
```


Construct a structure to 'customer' specify data of customers in a bank. Data to be stored is Account number, Name, balance. Assume maximum of 200 customers in the bank. Write a program in C to find and print all details of customers having balance less than 100.

```
#include <stdio.h>
struct customer
{
    int acc-no;
    char name[30];
    float balance;
};
```

```
int main()
{
    struct customer c[200];
    int i;
    for(i=0; i<200; i++)
    {
        printf("Enter details for customer %d", i+1);
        scanf("%d", &c[i].acc-no);
        gets(c[i].name);
        scanf("%f", &c[i].balance);
    }
```

```
for(i=0; i<200; i++)
{
    if(c[i].balance < 100)
```

```
    printf("Account No=%d", c[i].acc-no);
    printf("Name=%s", c[i].name);
    printf("Balance=%f", c[i].balance);
}
```

```
return 0;
}
```


Nested Structure (Imp)

```
struct address  
{  
    char city[50];  
    int pin;  
    char ph[15];  
};
```

```
struct employee  
{  
    char name[30];  
    int id;  
    float salary;  
    struct address a;  
};
```

```
struct employee e;
```

```
e.name ✓  
e.id ✓  
e.salary ✓  
e.a.city }  
e.a.pin }  
e.a.ph }
```

variable of Another structure "address"
is created inside "employee" structure definition

```
#include <stdio.h>
```

```
struct employee → outer structure
```

```
{ char name[30];  
  int age;
```

```
    struct date  
    {  
        int mm;  
        int dd;  
        int yyyy;  
    } doj;
```

```
} e;
```

→ inside structure

OR

```
struct date  
{ int mm;  
  int dd;  
  int yyyy;  
};
```

```
struct employee  
{ char name[30];  
  int age;
```

```
    struct date doj;  
};
```


Nesting of structure

```
#include <stdio.h>

struct employee → outer structure
{
    char name[30];
    int age;
    struct date
    {
        int mm;
        int dd;
        int yyyy;
    } doj;
} e;
```

→ inside structure

```
int main()
{
    printf("Enter employee details");
    printf("Enter employee name");
    scanf("%s", e.name);
    printf("Enter employee age");
    scanf("%d", &e.age);
```

```
printf("Enter employee's Date of Joining");
scanf("%d", &e.doj.mm);
scanf("%d", &e.doj.dd);
scanf("%d", &e.doj.yyyy);
printf("The employee details are");
printf("Name = %s", e.name);
printf("Age = %d", e.age);
printf("doj = %d %d %d", e.doj.mm, e.doj.dd,
e.doj.yyyy);

return 0;
```


AKTU PYQs

1. Construct a structure to 'customer' specify data of customers in a bank. The data to be stored is: Account number, Name, Balance in account. Assume maximum of 200 customers in the bank. Write a program in C to find and print all details of customers (i.e. Account Number, Name and Balance) having balance less than Rs. 100. (AKTU 2022-23)
2. Write a program in C language using union to store name, mobile number, year of birth, fees deposited, total marks in class 12th and print only name and marks of class 12th of those students who have deposited fees less than 5000 and are born after year 2005. (AKTU 2023-24)



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture- 13

Topics to be Covered

- Union in C
- Enumeration in C
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Union

- **Union** can be defined as a user-defined data type which is a collection of different variables of different data types in the same memory location.
- The union can also be defined as many members, but only one member can contain a value at a particular point in time.
- If we need to use the same memory location for two or more members, then union is the best data type for that. The largest union member defines the size of the union.

Example

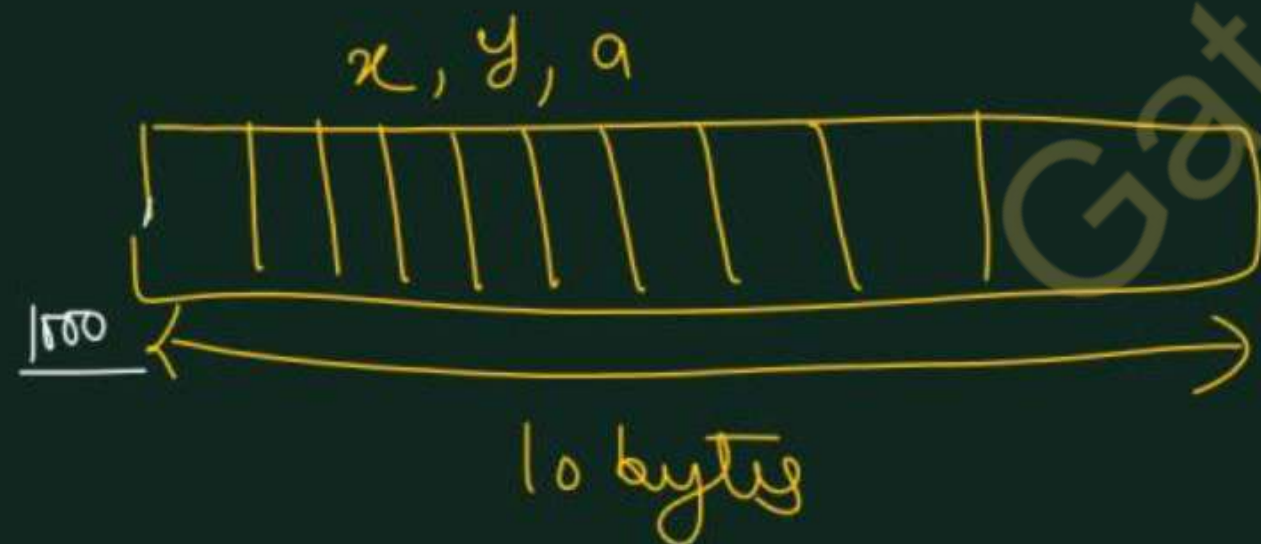
Union

union xyz → Keyword → Name of union
{
 int x;
 float y;
 char a[10];
};

Union Members

union xyz u; → 10 byte

Memory



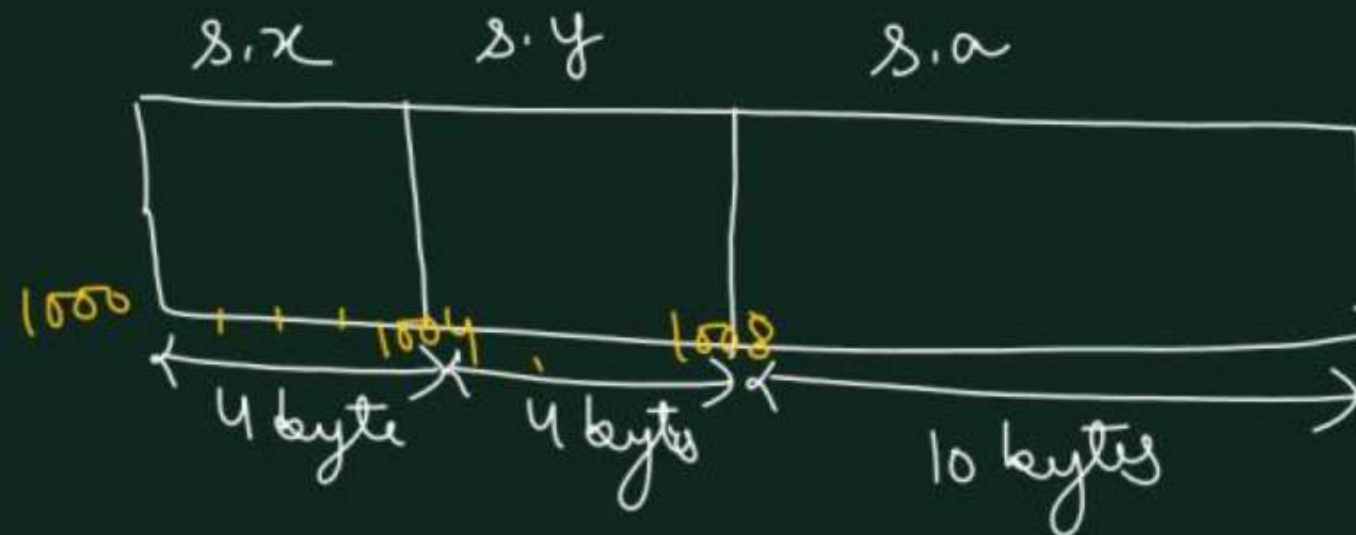
Structure

struct xyz
{
 int x;
 float y;
 char a[10];
};

Structure Members

struct xyz s; → it will take 18 bytes

Memory



Difference between Structure and Union

(AKTU)

C Structure	C Union
Allocates storage space for all its members separately.	Allocates common storage space for all its members. The largest union member defines the size of the union.
Structure occupies higher memory space.	Union occupies lower memory space
We can access all members of structure at a time.	We can access only one member of union at a time.
<pre>struct student { int mark; char name[6]; double average; };</pre>	<pre>union student { int mark; char name[6]; double average; };</pre>
For above structure, memory allocation: int mark – 2B, char name[6] – 6B double average – 8B Total memory allocation = 2+6+8 = 16 Bytes	For above union, only 8 bytes of memory will be allocated Total memory allocation = 8 Bytes

Enumeration (Short Note)

Enumeration (or enum) is a user defined data type in C. It is mainly used to assign names to integral constants, the names make a program easy to read and maintain.

Example 1:

```
#include<stdio.h>
enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};
int main()
{
    enum week day; // creating variable of enum week type
    day = Wed;
    printf("%d", day); // It will print 2.
    return 0;
}
```

Handwritten annotations:

- enum is labeled as "Keyword".
- week is labeled as "Name of enumeration".
- The members Mon, Tue, Wed, Thur, Fri, Sat, Sun are labeled as "enum members" with indices 0 through 6 written above them.
- The variable day in the declaration is circled.

Example 2:

```
#include<stdio.h>
enum year{Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec};
int main()
{
    int i;
    for (i=Jan; i<=Dec; i++)
        printf("%d ", i);

    return 0;
}
```

Handwritten annotations: Arrows point from 0 to Jan, 1 to Feb, 2 to Mar, and 3 to Apr. Another arrow points from 0 to Jan in the for loop. A second arrow points from 11 to Dec in the for loop. A handwritten note says: // it will print 0 1 2 3 4 5 6 7 8 9 10 11

Example 3:

```
#include <stdio.h>
```

```
enum day {sunday = 1, monday, tuesday = 5, wednesday, thursday = 10, friday, saturday};
```

```
int main()
```

```
{
```

```
    printf("%d %d %d %d %d %d %d", sunday, monday, tuesday, wednesday, thursday, friday, saturday);
```

```
    return 0;
```

```
}
```

output — 1 2 5 6 10 11 12

AKTU PYQs

1. Write a program in C language using union to store name, mobile number, year of birth, fees deposited, total marks in class 12th and print only name and marks of class 12th of those students who have deposited fees less than 5000 and are born after year 2005. (AKTU 2023-24)
- 2) Short note on union.
- 3) Difference between structure and union.
- 4) Write short note on enum.



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture- 14

Topics to be Covered

- Pattern Programs
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

①

i \ j	1	2	3	4	5
1	*				
2	*	*			
3	*	*	*		
4	*	*	*	*	
5	*	*	*	*	*

②

1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5

③

(AKTU-2021-22)

1	2	3	4	5
2	2	3	4	5
3	3	3	4	5
4	4	4	4	5
5	5	5	5	5

```
#include <stdio.h>
```

```
int main()
```

```
{ int rows, i, j;
```

```
printf("Enter no. of rows");
```

```
scanf("%d", &rows);
```

→ lets take no. of rows = 5

```
for(i=1; i<=rows; i++)
```

```
{ for(j=1; j<=i; j++)
```

```
{ printf("*");
```

```
} printf("\n");
```

```
} return 0;
```

*				
*	*			
*	*	*		
*	*	*	*	
*	*	*	*	*

i \ j	1	2	3	4	5
1	*				
2	*	*			
3	*	*	*		
4	*	*	*	*	
5	*	*	*	*	*

i \ j	1	2	3	4	5
1	1				
2	1	2			
3	1	2	3		
4	1	2	3	4	
5	1	2	3	4	5

i \ j	1	2	3	4	5
1	1				
2	2	2			
3	3	3	3		
4	4	4	4	4	
5	5	5	5	5	5

(AKTU-2021-22)

```
#include <stdio.h>
int main()
{
    int rows, i, j;
    printf("Enter no. of rows");
    scanf("%d", &rows); → 5
    for(i=1; i<=rows; i++)
    {
        for(j=1; j<=i; j++)
        {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

same

same

printf("%d", j);

same

printf("%d", i);

same

④ (AKTU 2016-17)

1	A				
2	A	B			
3	A	B	C		
4	A	B	C	D	
5	A	B	C	D	E

A-65
B-66

```
#include <stdio.h>
int main()
{
    int rows, i, j;
    printf("Enter no. of rows");
    scanf("%d", &rows);
    for(i=1; i<=rows; i++)
    {
        for(j=1; j<=i; j++)
        {
            printf("%c", (j+64));
        }
        printf("\n");
    }
    return 0;
}
```

⑤

A				
B	B			
C	C	C		
D	D	D	D	
E	E	E	E	E

same

→ printf("%c", (i+64));

same

⑥ (AKTU-2020-21)

5	*	*	*	*	*
4	*	*	*	*	
3	*	*	*		
2	*	*			
1	*				

```
#include <stdio.h>
int main()
{
    int rows, i, j;
    printf("Enter no. of rows");
    scanf("%d", &rows);
    for(i=rows; i>=1; i--)
    {
        for(j=1; j<=i; j++)
        {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```


⑦

5	1	2	3	4	5
4	1	2	3	4	
3	1	2	3		
2	1	2			
1	1				

(AKTU 2022-23)

```
#include <stdio.h>
int main()
{
    int rows, i, j;
    printf("Enter rows");
    scanf("%d", &rows);
    for (i = rows; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d", j);
        }
        printf("\n");
    }
    return 0;
}
```

Floyd's Triangle (Imp)

⑧

1				
2	3			
4	5	6		
7	8	9	10	
11	12	13	14	15

(AKTU 2021-22)

```
#include <stdio.h>
int main()
{
    int rows, i, j, K=1;
    printf("Enter rows");
    scanf("%d", &rows);
    for (i = 1; i <= rows; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d", K);
            K++;
        }
        printf("\n");
    }
    return 0;
}
```

⑨

A				
B	C			
D	E	F		
G	H	I	J	
K	L	M	N	O

same

→ Replaced by

```
{
    printf("%c", (K+64));
}
same
```


16) $\frac{1}{4} \quad 9$
 $16 \quad 25 \quad 36$
 $49 \quad 64 \quad 81 \quad 100$
 AKTU(2023-24)

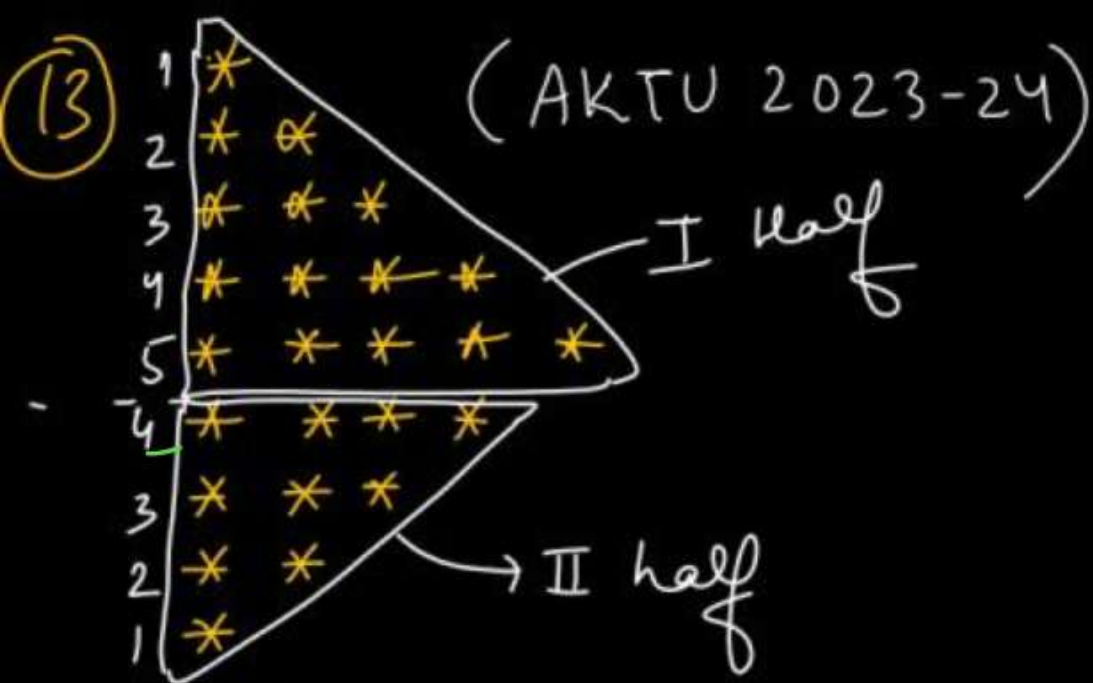
```
#include <stdio.h>
int main()
{
    int rows, i, j, K=1;
    printf("Enter rows");
    scanf("%d", &rows);
    for(i=1; i<=rows; i++)
    {
        for(j=1; j<=i; j++)
        {
            printf("%d", K*K);
            K++;
        }
        printf("\n");
    }
    return 0;
}
```

11) $\begin{array}{c} 10 \\ 9 \quad 8 \\ 7 \quad 6 \quad 5 \\ 4 \quad 3 \quad 2 \quad 1 \end{array}$
 AKTU(2018-19)

```
#include <stdio.h>
int main()
{
    int i, j, K=10;
    for(i=1; i<=4; i++)
    {
        for(j=1; j<=i; j++)
        {
            printf("%d", K);
            K--;
        }
        printf("\n");
    }
    return 0;
}
```

12) $\begin{array}{c} 5 \quad 4 \quad 3 \quad 2 \quad 1 \\ 4 \quad 3 \quad 2 \quad 1 \\ 3 \quad 2 \quad 1 \\ 2 \quad 1 \\ 1 \end{array}$

```
#include <stdio.h>
int main()
{
    int rows, i, j;
    printf("Enter rows");
    scanf("%d", &rows);
    for(i=rows; i>=1; i--)
    {
        for(j=i; j>=1; j--)
        {
            printf("%d", j);
        }
        printf("\n");
    }
    return 0;
}
```

```
#include <stdio.h>
int main()
{
    int rows, i, j;
    printf("Enter rows");
    scanf("%d", &rows);
```

```
for(i=1; i<=rows; i++)
{
    for(j=1; j<=i; j++)
    {
        printf("*");
    }
    printf("\n");
}
```

printing the
upper half

```
for(i=rows-1; i>=1; i--)
{
    for(j=1; j<=i; j++)
    {
        printf("*");
    }
    printf("\n");
}
return 0;
```

printing the
lower half

14 (AKTU 2023-24)

upper half

lower half

```
#include <stdio.h>
```

```
int main()
```

```
{ int rows, i, j;
```

```
printf("Enter rows");
```

```
scanf("%d", &rows);
```

```
for(i=1; i<=rows; i++)
```

```
{ for(j=1; j<=i; j++)
```

```
{ printf("%d", j);
```

```
} printf("\n");
```

```
} for(i=rows-1; i>=1; i--)
```

```
{ for(j=1; j<=i; j++)
```

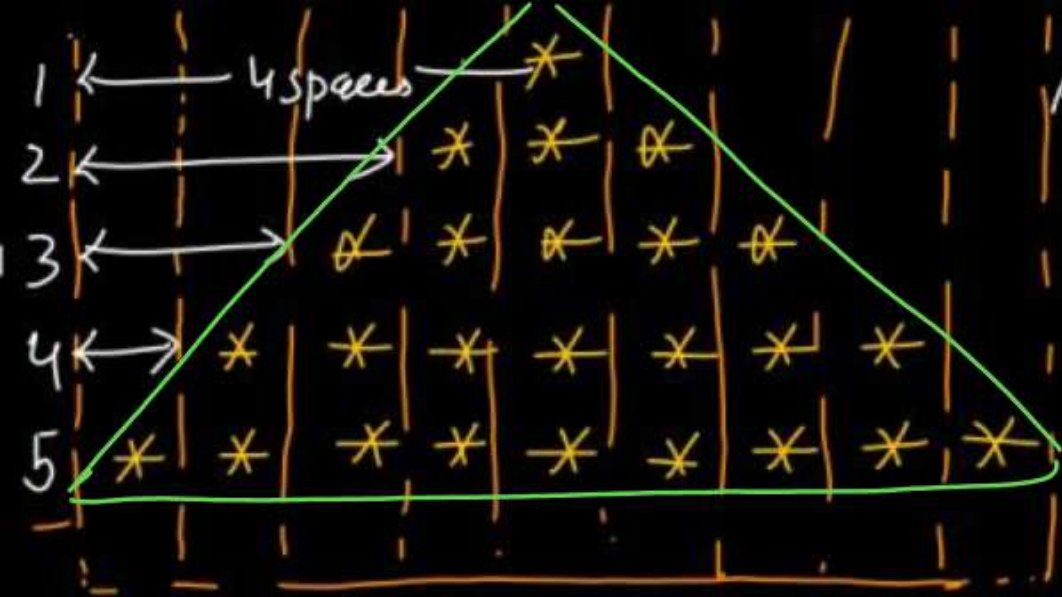
```
{ printf("%d", j);
```

```
} printf("\n");
```

```
} return 0;
```

printing the upper half

printing the lower half



AKTU (2022-23)
(15)

```
#include <stdio.h>
int main()
{
    int rows, i, j, space;
    printf("Enter rows:");
    scanf("%d", &rows);
```

```
for(i=1; i<=rows; i++)
```

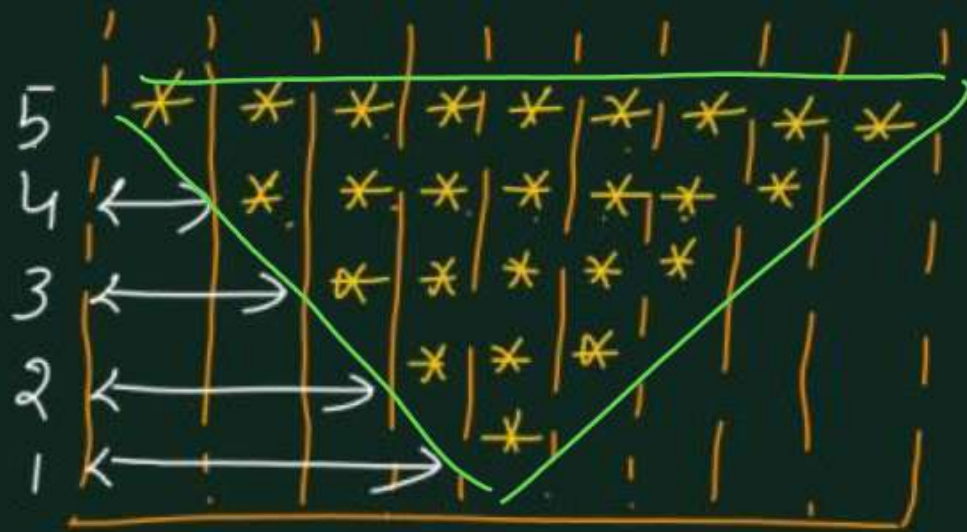
```
{
    for(space=1; space<=(rows-i); space++)
    {
        printf(" ");
    }
    for(j=1; j<=(2*i-1); j++)
    {
        printf("*");
    }
    printf("\n");
}
```

rows	for spaces
i=1	4 (rows-i)
i=2	3 (rows-i)
i=3	2 (rows-i)
i=4	1 (rows-i)
i=5	0 (rows-i)

for * (2*i-1)
1
3
5
7
9



16



rows = 5

spaces

0 (rows - i)

1

2

3

4

*

9 (2 * i - 1)

7

5

3

1

```
#include <stdio.h>
```

```
int main()
```

```
{ int rows, i, j, space;
```

```
printf("Enter rows");
```

```
scanf("%d", &rows);
```

```
for (i = rows; i >= 1; i--)
```

```
{ for (space = 1; space <= (rows - i); space++)
```

```
{ printf(" ");
```

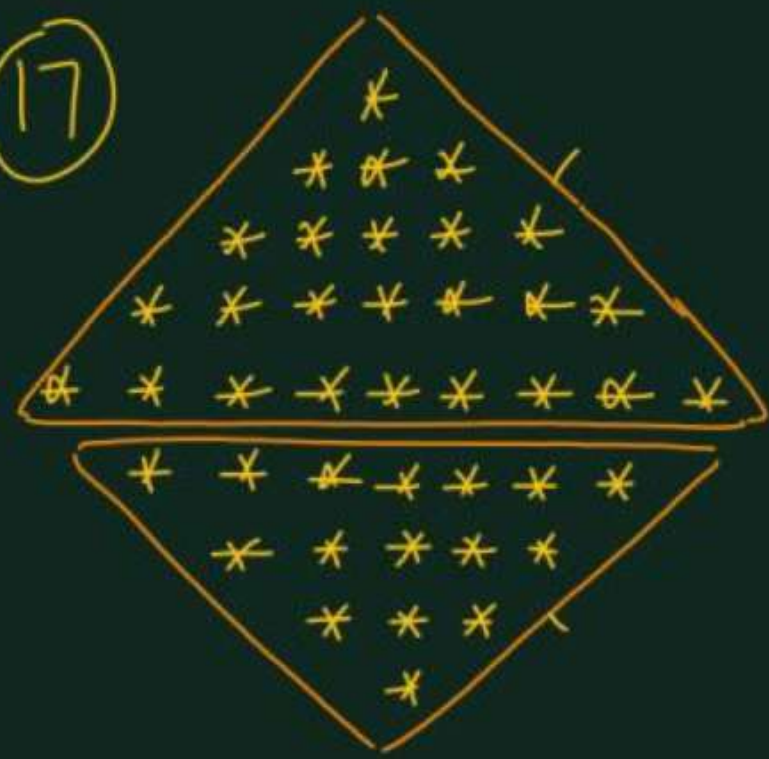
```
for (j = 1; j <= (2 * i - 1); j++)
```

```
{ printf("%d", j);
```

```
} printf("\n");
```

```
} return 0;
```


17



```
#include <stdio.h>
int main()
{
    int rows, i, j, space;
    printf("Enter rows");
    scanf("%d", &rows);
```

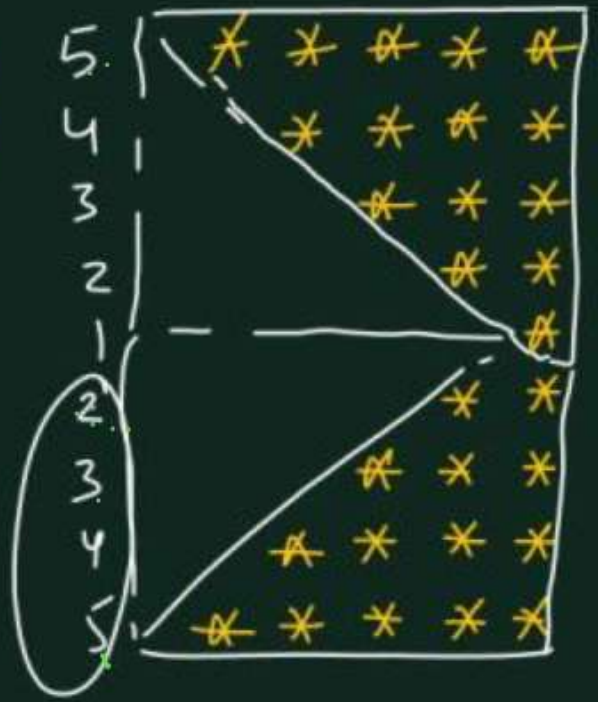
```
for (i=1; i<=rows; i++)
{
    for (space=1; space<=(rows-i); space++)
    {
        printf(" ");
    }
    for (j=1; j<=(2*i-1); j++)
    {
        printf("*");
    }
    printf("\n");
}
```

upper Triangle

```
for (i = rows-1; i>=1; i--)
{
    for (space=1; space<=(rows-i); space++)
    {
        printf(" ");
    }
    for (j=1; j<=(2*i-1); j++)
    {
        printf("*");
    }
    printf("\n");
}
return 0;
```

lower Triangle

18



AKTU (2021-22)
DPP ✓

```
#include <stdio.h>
int main ( )
{
    int rows, i, j, space;
    printf("Enter rows");
    scanf("%d", &rows);
```

```
for (i = rows; i >= 1; i--)
{
    for (space = 1; space <= (rows - i); space++)
    {
        printf(" ");
    }
    for (j = 1; j <= i; j++)
    {
        printf("*");
    }
    printf("\n");
}
```

upper
Triangle

```
for (i = 2; i <= rows; i++)
{
    for (space = 1; space <= (rows - i); space++)
    {
        printf(" ");
    }
    for (j = 1; j <= i; j++)
    {
        printf("*");
    }
    printf("\n");
}
```

lower
Triangle

return 0;



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-3: Iteration and Loops & Arrays Lecture- 15

Topics to be Covered

- Series Programs
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

WAP to print the sum of following series.

$$S = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \dots + \frac{1}{n}$$

```
#include <stdio.h>
```

```
int main()
```

```
{ int n, i;
```

```
float s = 0.0;
```

```
printf("Enter no. of terms");
```

```
scanf("%d", &n); ✓
```

```
→ for (i = 1; i <= n; i++)
```

```
{
```

```
    S = S + 1/i;
```

```
}
```

```
printf("The sum of series is = %f", s);
```

```
return 0;
```

```
}
```

$$i=1 \quad S = 0 + \frac{1}{1}$$

$$i=2 \quad S = \frac{1}{1} + \frac{1}{2}$$

$$i=3 \quad S = \frac{1}{1} + \frac{1}{2} + \frac{1}{3}$$

$$i=4 \quad S = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} \dots$$

WAP to print the sum of following series.

$$1^1 + 2^2 + 3^3 + 4^4 + \dots + n^n$$

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main()
```

```
{  
    int n, i, s=0;
```

```
    printf("Enter no. of terms");
```

```
    scanf("%d", &n);
```

```
    for(i=1; i<=n; i++)  
    {
```

```
        s = s + pow(i, i);
```

```
    }  
    printf("The sum of series is = %d", s);
```

```
    return 0;
```

$$\begin{array}{l} i=1 \\ 0 \end{array}$$

$$\begin{array}{l} i=2 \\ 0 \end{array}$$

$$\begin{array}{l} i=3 \\ 0 \end{array}$$

$$s = 0 + 1^1$$

$$s = 1 + 2^2$$

$$s = 1 + 2^2 + 3^3 \dots$$

WAP to print the sum of following series. (AKTU 2021-22)

$$1! + 2! + 3! + 4! + \dots + n!$$

$$S = 1! + 2! + 3! + 4! + 5!$$

```
#include <stdio.h>
int main()
```

```
{ int i, n, fact, s=0, j;
  printf("Enter no. of terms");
  scanf("%d", &n);
```

```
  for(i=1; i<=n; i++)
```

```
  { fact=1;
    for(j=1; j<=i; j++)
    { fact=fact*j;
    }
```

```
    s=s+fact;
  }
```

```
  return 0;
}
```

Suppose $n=5$

$$\begin{array}{l} i=1 \\ \text{fact}=1 \\ j=1 \end{array}$$

$$\text{fact} = \text{fact} \times j$$

$$\text{fact} = 1 \times 1 = 1 \Rightarrow s = 0 + 1 \Rightarrow \boxed{s=1}$$

$$\begin{array}{l} i=2 \\ \text{fact}=1 \\ j=1 \end{array}$$

$$\text{fact} = 1 \times 1$$

$$\text{fact} = 1 \times 2 \Rightarrow \text{fact} = 2$$

$$s = s + \text{fact}$$

$$\boxed{s = 1 + 2} \Rightarrow 1! + 2!$$

$$i=3$$

$$\text{fact}=1$$

$$j=1, j=2, j=3 \mid \text{fact} = 1 \times 2 \times 3 \Rightarrow 3!$$

$$s = 1 + 2 + (1 \times 2 \times 3)$$

$$\Rightarrow s = 1! + 2! + 3! + \dots$$

$$i=4, i=5$$

WAP to print the sum of following series. (AKTU 2023-24)

$$1 + \frac{1!}{n!} + \frac{2!}{n!} + \frac{3!}{n!} + \dots + \frac{n!}{n!}$$

```
#include <stdio.h>
int main()
```

```
{
    int n, x, i, f = 1, j;
    float fact = 1, s = 1.0;
    printf("Enter no. of terms");
    scanf("%d", &x);
    printf("Enter n");
    scanf("%d", &n);
```

```
    for(i = 1; i <= n; i++)
    {
        f = f * i;
    }
    // finally f will hold n!
```

```
    for(i = 1; i <= x; i++)
    {
        fact = 1;
        for(j = 1; j <= i; j++)
        {
            fact = fact * j;
        }
        s = s + (fact / f);
    }
```

```
    printf("The sum of series is = %f", s);
    return 0;
}
```

WAP to print the sum of following series. (AKTU 2023-24)

$$S = \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!}$$

```
#include <stdio.h>
#include <math.h>
int main()
```

```
{ int i, j, n, x;
```

```
float s = 0.0, fact;
```

```
printf("Enter no. of terms");
```

```
scanf("%d", &n);
```

```
printf("Enter value of x");
```

```
scanf("%d", &x);
```

```
for (i = 1; i <= n; i++)
{
    fact = 1;
```

```
for (j = 1; j <= i; j++)
```

It will
store
factorial
of i values

```
fact = fact * i;
```

```
s = s + pow(x, i) / fact;
```

```
return 0;
```

```
}
```

$$s = s + x^i / i!$$

WAP to print sum of the following series -

$$1) x + \frac{x^3}{3!} + \frac{x^5}{5!} + \frac{x^7}{7!} + \dots \text{upto } n \text{ terms}$$

$$2) \frac{1}{0!} + \frac{x^2}{2!} + \frac{x^4}{4!} + \frac{x^6}{6!} + \dots \text{upto } n \text{ terms}$$

$\frac{x}{0!}$

Thank
you